

VECTOR

APL2001 at Yale ...

• Ray's Conference Diary	48
• Gustafsson's Shipping Simulator	57
• Reiter on image galleries	69
• McDonnell is boggled	91
• JPEG compression in J	103



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

www.vector.org.uk

Vol.18 No.1 July 2001

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2001-2002

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	10	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439788385.

Email: apl385@compuserve.com

Contents

	Page
Guest Editorial	3
Anthony Camacho	
APL NEWS	
Quick Reference Diary	5
British APL Association News	
News from Sustaining Members	14
APL Product Guide	16
Gill Smith	
The Education Vector	
Zark Newsletter Extracts	31
edited by Jon Sandles	
Crossword	39
J-ottings 29: Power Steering Extra	41
Norman Thomson	
RECENT MEETINGS - APL2001 at Yale	
My Yale 2001 Diary	48
Ray Cannon	
GENERAL ARTICLES	
NavSim – a Shipping Simulator: Part I	57
Tomas Gustafsson	
Web Image Galleries Made Routine	69
Clifford A. Reiter	
TECHNICAL SECTION	
Hacker's Corner: Compressing with GZIP	84
Adrian Smith	
Technical Correspondence	89
At Play with J: Boggle	91
Gene McDonnell	
J is for JPEG: Windows Bitmap to JPEG Baseline Compression	103
Richard Dazeley	
Inversion of Boolean Matrices	125
Michel Dumontier	
Matrix inversion modulo p Linear systems modulo p	134
Michel Dumontier	
Index to Advertisers	143



**This year we at HMW celebrate our
20th anniversary.**

Services:

APL Consultancy
Bespoke Development
Project Management (full member DSDM Consortium)

Products

4xtra: Dealing System
Inca: Software Change Management
Maya: Development Suite
Aztec: Task Management shell

HMW Computing

Hamilton House
1 Temple Avenue
London
England
EC4Y 0HA
Tel/ Fax: 0870 1010 469
Email: HMW@4xtra.com
URL: <http://www.4xtra.com>

Nightmare Programming – Nightmare Data

Guest editorial by Anthony Camacho

Once upon a time, when the first computers were still being invented, there was a distinction between a calculating machine and a computer. Both could do more or less any task that would fit in the very small memories of such machines. The essence of the difference was that a calculating machine's program was unchangeable (maybe held on punched cards) whereas a computer had a stored program and the data in the program store could be amended in just the same way as any other data. One could program a branch by storing a different instruction in the sequence next to be obeyed.

Such programs soon became a nightmare of complexity; debugging was *difficult*. When the program kept changing and had thousands of different valid states, a program listing could not represent the program adequately. (APL was invented to solve the problem of representing a multi-million state machine.)

Some of us were outraged when it was suggested as a matter of policy or good programming style that programs should not modify themselves. If a program were made read-only then its state could be fixed and the tested version could be relied on not to change. Of course the problem of testing was still insoluble in the sense that it was not possible to prove that there was no bug left in any program. This soon became received wisdom and is now the rule to such an extent that there are few programs that cannot be run directly from read-only memory (most commonly CD ROM). We used to be able to back up just the data directories.

The total separation between programs and data did not last long.

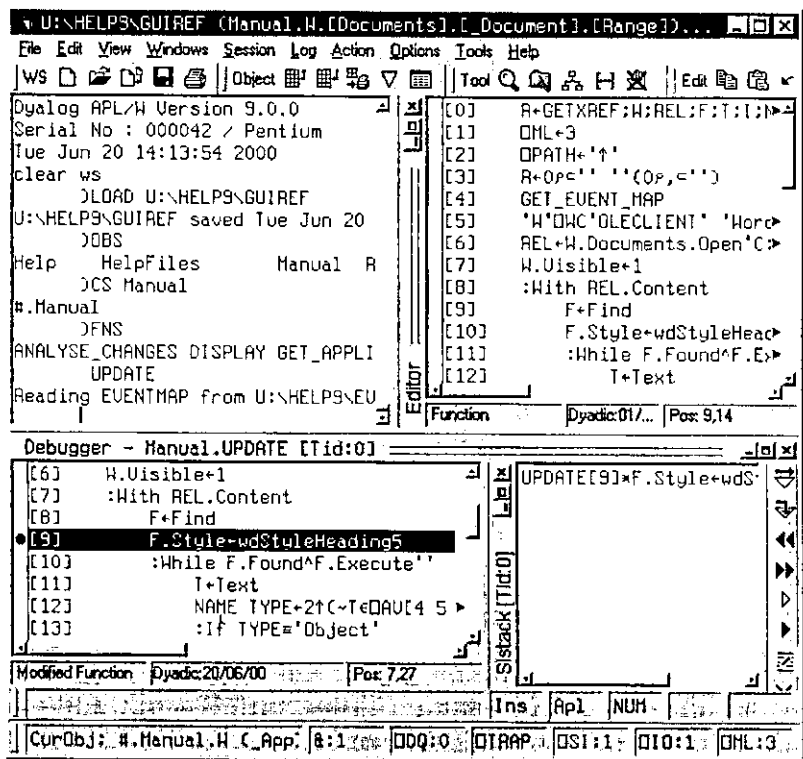
Perhaps it was the invention of the macro that breached the dyke (or the macro that called Visual Basic). Perhaps it was the foolish introduction of the *object* paradigm where programming could be included with the data. The Dynamic Data Exchange Management Library (about 1990) led the way.

It is, in a way, a surprise that computer people who had assented to the proposition that programs should not be treated as data; not be dynamically changed eagerly accepted the proposition that data should contain programs.

The consequences are legion. The worm and the virus for one. Surely this one consequence is bad enough to justify re-introduction of the rule *programs and data shall be rigorously separated*. Maybe we could even have simple back-ups of just the data directories again.

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Quick Reference Diary

Date	Venue	Event
17-18 October	New York	J User Conference
4-7 November	Naples, Florida	APL2000 User's Meeting
22-25 July, 2002	Madrid, Spain	APL2002 Conference

Dates for Future Issues of VECTOR

	Vol.18 No.2	Vol.18 No.3	Vol.18 No.4
Copy date	7th September	7th December	8th March
Ad booking	14th September	14th December	15th March
Ad Copy	21st September	21st December	22nd March
Distribution	October 2001	January 2002	April 2002

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

**Download
from the
web for
free!**

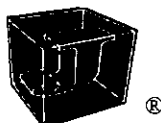
Announcing JsoftwareTM Version 4.06

- ♦ New and Improved Documentation
- ♦ New Data Types, Symbol and Unicode
- ♦ DLL Callbacks Supported
- ♦ New Language Features
- ♦ Improved Locales
- ♦ And more!

This release will make you more productive. We've listened to our users requests, and implemented many technical changes and improvements. We also added new features, for example, significant new data types.

Make sure to watch our web site for the new release. Professional Edition subscribers will receive their copies in the mail after the edition is released on the web.

Jsoftware Inc.
19235 Covington Ct.
Shorewood, MN 55331
Info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202



www.jsoftware.com

CORRESPONDENCE

From: Bill Lam

29 May 2001

I enjoy reading your editorial, it said there is not enough submission on APL. But in the same issue, there is an article "The Development and Q/A Process at Microsoft, US". It does not relate to APL, J, K nor array-thinking in general.

Why is this article included for publication?

[Editor's reply]

Thank you for your compliments. It's not actually a contradiction that, since we are lacking a bit APL contributions, we have to resort to contributions that cross the boundaries of being strictly on topic. Actually, I find hard when talking about a computer language to avoid talking about development in general. And the article, summary of a talk presented at the Finnish APL meeting, is in my opinion quite interesting because it highlights some fundamental points about software development and project management. In particular Extreme Programming seems to be a hot topic and I am trying to learn something about it to see if it can be applied to APL teams as well. Maybe Vector should be a technical magazine devoted to the APL language only (or maybe to Array Oriented Languages with some relationship to APL only) and should leave more general discussion topics to other magazines. I, for one, would find it limiting, the way knowing only one computer language in today's world is limiting. But if you think that we should be more selective about the published papers, please, let us know. While it's true that Vector can be a fun game for us, we are also trying to give a service to our readers and as such we should try and improve the quality of it by listening to the readers' comments.

British APL Association News

Minutes of the meeting of the Committee of the British APL Association held on 18 May 2001 at the Royal Statistical Society

It was agreed that business could be settled by the Committee by email but that the Chairman should insist on the formality of a reply from everyone; proposals should not be agreed without everyone's opinion being on record.

It was agreed that we would propose at the AGM to raise fees for next year (2002-2003) to £20 for individual members, to leave the corporate membership at £100 but reduce the number of Vectors included to 5 (which the corporate could increase by paying more), and increase sustaining membership to £500.

It was agreed that we would revert to a fixed date each year for renewals and, for the members whose year was different, offer to allow them to pay either the fraction or 1+ the fraction of a year's subscription to bring them into synchronisation.

It was agreed that the Committee would be the same as last year except that Ian Clark would be asked to serve as 'minister without portfolio' and the Chairman would ask Norman Thomson to be Education Officer on the understanding he would not be expected to travel to London.

It was agreed that the Web site could carry relevant advertisements with links to, say, APL vendors and it was agreed to leave detailed arrangements to the Webmaster and Chairman.

It was agreed the BAA would help with VikAPL and would look into the practicality of holding a two-day event early in September. The suggested venue is the Lake District – maybe Kendal.

The Committee re-appointed John Sullivan to audit the accounts for 2000–2001.

Anthony Camacho 22nd May 2001

Minutes of the AGM of the British APL Association held at the Royal Statistical Society from 2pm to 2.15 pm on 18 May 2001

Apologies were received from Ian Clark and Stefano Lanzavecchia.

The minutes of the last AGM as printed in Vector Vol 17 No 1 were approved.

The Chairman reported as follows:

British APL Association Chairman's report on the year 2000-2001

The committee for next year is:

Chairman	Adrian Smith
Secretary	Anthony Camacho
Treasurer	Nicholas Small
Editor	Stefano Lanzavecchia
Webmaster	Ray Cannon
Activities	Jon Sandles
Education Officer	Vacant – to be co-opted
Projects & Publicity	Alan Mayer

The subscriptions for next year are to be increased to £20 for an individual, £100 (with entitlement to 5 copies of Vector) for a corporate and to £500 for a sustaining member.

Vector is thriving; it even has more material available than usual. The working group continues to be a friendly and effective way to drum up articles and it is noticeable that the material we print does not even largely come from the committee and the working group.

The project to port A+ is progressing (but we haven't had a completely successful compilation yet) and we hope that we will have an APL-like powerful desk calculator that we can give away to schools and anyone interested before too long.

We gave welcome help to APL Berlin 2000 and as a by-product produced the CD of the working papers and the archive of those Vectors we have in electronic form. This is very useful as an aid to promote membership, particularly in the USA. We are committed to help to at least the same extent at Madrid.

Accounts

The Treasurer, Nicholas Small, presented the accounts (as below). There were no questions.

Outstanding Achievement Award



Adrian Smith presented the BAA award for outstanding achievement to John Daintree in recognition of his work in setting up Dyalog as part of Microsoft's .NET project.

The award recognises that this work has not been at all easy, and has required considerable persistence and patience in interpreting the patchy and ever-changing documentation that goes with any beta software.

John has been able to work with a succession of .NET beta programmes from Microsoft and has been able to integrate the Dyalog APL interpreter into these environments with ever-increasing confidence and reliability.

The award also recognises that this is a major opportunity for the APL world to get back into mainstream development, and that Dyadic Systems as a whole should be thanked for their considerable investment in time and manpower in responding to Microsoft's invitation to participate in the .NET programme.

No member wished to ask a question or to raise any other business. The Chairman declared the AGM over and the meeting continued with the presentations, no doubt reported elsewhere.

Anthony Camacho 22 May 2001

Treasurer's report

Nicholas Small reported: he circulated a short set of accounts:

British APL Association – Summary of Annual Accounts 2000/2001

Summary of income and expenditure/receipts and payments:

	2000/01 (R&P) £	2000/01 (I&E) £	1999/2000 (I&E) £	1998/99 (I&E) £	1997/98 (I&E) £
Income/Receipts					
Subscriptions	9513	9736	9152	10368	11021
BCS services	183	183	278	167	0
Bank interest	2251	2251	1882	2943	3322
Vector advertising (incl. VAT)	2342	2967	3047	2463	3471
Other	394	394	366	264	108
	-----	-----	-----	-----	-----
Total receipts	14682	15530	14725	16206	17922
	-----	-----	-----	-----	-----
Expenditure/Payments					
Meetings	265	265	259	704	0
Administration	1316	1347	1344	1423	1152
BCS services	183	183	278	167	0
Vector production and despatch	16223	16344	15802	17086	14584
Education Vector Supplement	-	-	-	-	108
Projects	1721	1721	470	328	501
Other	307	355	501	426	308
	-----	-----	-----	-----	-----
Total payments	20014	20215	18653	19954	16653
	-----	-----	-----	-----	-----
Assets summary:					
Bank and other balances		39652	44984	49055	52525
Debtors		1912	858	733	2071
Creditors		(5265)	(4858)	(4766)	(5270)
		-----	-----	-----	-----
Net assets		36299	40983	45012	49327
		-----	-----	-----	-----
Written off			(100)	(568)	

Notes to the Accounts

Pence figures have been omitted, so columns may not add exactly.

The value of stocks of Vector have not been assessed, nor has the value of the Association's computing hardware and software.

For 2000/2001, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for earlier years relate to income and expenditure.

Membership report

Nicholas reported on behalf of Rowena:

1. Membership at 30.4.01 (previous year's figures in parentheses)

	UK		FOREIGN		TOTAL	
	Number	Vectors	Number	Vectors	Number	Vectors
Sustaining	5 (4)	25 (15)	3 (4)	66 (76)	8 (8)	81 (88)
Corporate*	7 (6)	50 (37)	1 (1)	10 (30)	7 (11)	47 (87)
Corp. Ind*	19 (20)	19 (24)	2 (2)	2 (1)	22 (23)	26 (27)
Individual	122 (130)	122 (128)	237 (218)	235 (218)	359 (348)	357 (346)
Non-voting	16 (15)	16 (15)	0 (0)	0 (0)	16 (15)	16 (15)
Life	1 (1)	1 (1)	1 (1)	1 (1)	2 (2)	2 (2)
Library	0 (1)	0 (1)	5 (6)	5 (6)	5 (7)	5 (7)
Russians			10 (10)	10 (10)	10 (10)	10 (10)
APL Groups			13 (14)	39 (42)	13 (14)	39 (42)
						586 (573))

*Add the Vector numbers in these rows to get the total subscribed for by corporate members.

2. New members, May 2000-April 2001

Individual UK: 5 Europe: 9 USA/Canada:19*

*including 13 taking advantage of the special \$50 for two years offer.

Library:VUBA Printmedia, Germany

Corporate: Adaytum

3. Non-renewers, May 2000-April 2001

Library: SWETS (Abingdon) and Tieto-Tietopalvelut OY (Finland)

Individual: 29, of which 3 gone away
 8 explicit resignations (usually not using APL)
 18 no contact, just failed to renew

Further business

There were no questions to Officers nor any other business so the Chairman closed the meeting at approximately 2.15 pm and handed over the meeting to Jon Sandles.

Anthony Camacho 22nd May 2001

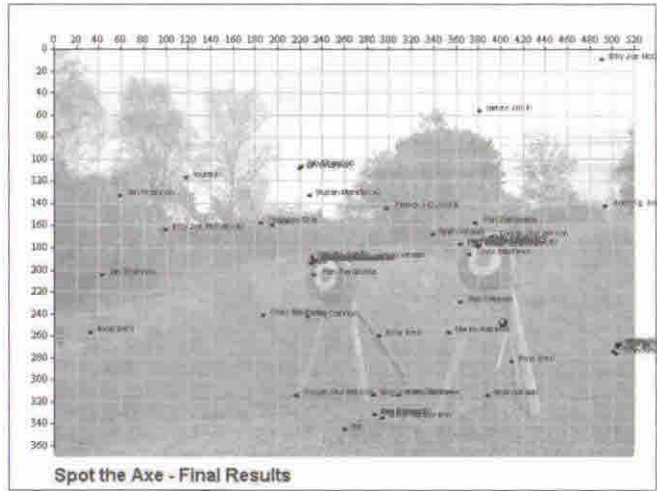
Competition Winner – “Spot the Axe”



Readers of Vector will remember that at VikAPL there were a number of Viking challenges such as Log-sawing and Axe-flinging. We ran a picture of the target, with the axe cunningly eliminated (the Paintshop Pro clone brush is a wonderful thing) and allowed visitors to the website 3 attempts to click the exact spot where the axe was lodged. There were over 80 entries, a few of which were disallowed on the grounds of prior knowledge.

The prize to the lucky winner was a bottle of Aalborg Akvavit, and of course the honour of standing as the target in next year’s contest. The runner-up prize of a life-time’s supply of Gammel Dansk (possibly the most unpleasant drink in the world) was fortunately not awarded.

Where the entries fell



The scatter shows a rather misplaced faith in the chance of actually sticking the axe in the target! As for the actual position (embedded in the rear leg of the support), no-one has any idea how it got there.

Sustaining Members' News

Insight Systems

After almost two years as an independent consulting company, Insight Systems is happy to announce our return to what we love best: development, sales and marketing of a highly successful APL-based planning application.

Effective mid-August 2001, Insight Systems will be a subsidiary of KCI Computing, Inc. of Los Angeles, California. In addition to marketing and supporting KCI's products and services in Europe, Insight Systems' technical team will join the product development group at KCI. KCI Computing, Inc. develops, markets and supports CONTROL[®], an integrated financial management and decision support tool. KCI uses APL as a core component of its technology, to provide analytical and reporting capabilities on relational databases, using VB and C++ to provide GUI, Web front ends and other interface components. For more information about KCI and CONTROL[®], see the web page at <http://www.kcicorp.com>.

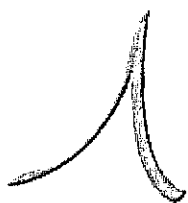
As a member of the KCI group, Insight Systems will continue to provide "one stop shop" services to the APL community in Denmark and internationally. We will continue as resellers of the most popular commercially available APL interpreters, we will continue to provide APL Management Consulting, advising our customers on software development strategies using APL, especially with respect to integration of APL components in "mainstream" software architectures. We will continue provide recruitment services for companies seeking APL developers for employment or contract work.

Insight Systems will remain a sustaining member of the British APL Association. We will continue to develop and maintain our ODBC interface, which is distributed by Dyadic Systems and APL2000. Development of a new release of this product will be concluded during the second half of 2001.

HMW

After breaking our silence in the last issue of Vector, we find we don't have too much to add this time. At least, having admitted to still selling DOS versions of our systems, we can announce that the Windows version seems to be gaining ground on its DOS parent and has generated a lot of interest in the last month or two. Its a great relief to be developing in a more modern platform! It does mean a busy summer for us, so think of us slaving away over a hot computer when you are all on holiday.

Causeway Graphical Systems Ltd



Well, Dyalog may have their DSS but at Causeway we have our CUSP (Causeway Update and Support Programme) and we think we have the prettiest logo!

CUSP is modelled closely on DSS (why change a working formula) and is designed to keep our customers in touch with the latest updates to RainPro, NewLeaf, CausewayPro and HelpStuf. For an annual subscription (the rate varies depending on the level of support you want – see the website for details) you get access to unlocked copies of the latest shipped workspaces and automatic emails whenever anything is fixed or improved. Typically these are going out every two weeks, but the frequency will obviously vary as changes inevitably come in in clumps, separated by long quiet periods. The passworded ZIP files will now be updated annually (and the passwords changed at the same time) so you will still be able to pay for an annual update if that is the approach you prefer.

One of the reasons that RainPro has been developing quickly of late is that (at last) we are able to sell this outside the APL community. Disguised as GraPL.NET it is now available to the world of VB programmers as a scripting tool for making VML and SVG charts in-line in web pages. Many of the improvements are specifically to support interactive charts on the web (for example adding JavaScript handlers to data items for drill-down) but there are some things which our GraPL.NET users are asking for which are valuable in their own right. Recent examples would be multi-line y-labels and key text, and much better support for labelling of time-based axes with styles like 'annual' or 'monthly' to get the tick-marks correctly placed.

NewLeaf has recently learned how to make PDF files 'straight out of the box' with no requirement for 3rd-party tools such as Acrobat Distiller. This allows you to distribute NewLeaf reports (with a nice table of contents made using leaf.Bookmark) as PDF documents that can be viewed and printed with the standard Acrobat Reader (free from Adobe). Of course charts from RainPro can be included in the output, as can bitmap images. In fact I am beginning to prefer NewLeaf to PowerPoint as a tool for making a bunch of simple foils – have a look at the PDFs on the Causeway website (under /pubs/seminars) to see the sort of thing you can make very easily indeed.

It is something of an irony that we have a world lead in one of the most modern 'sexy' technologies (interactive vector graphics with XML) with a bunch of 1992-vintage APL*PLUS/PC functions. Back to the future anyone?

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.

Please help us keep it up to date!

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,656	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERsystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERsystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERsystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Optima	IBM Compatible	poa	Complete networked or stand-alone solutions including configuration installation, maintenance and commissioning.

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.

DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	APL2 PC (US Version)	\$630	Product No. 5799-PGG. PRPQ Number RJ0411. Order from 1-800-IBM-CALL
	APL2 PC (European Version)	£348	Product No. 5604-260. Part number 38F1753. From all IBM dealers, including MicroAPL.
	APL2 for OS/2 Entry Edition	\$185	Part No 89G1556.
	APL2 for OS/2 Advanced Edition	\$650	Part No 89G1697. Contains all facilities of the Entry Edition plus: DB2 interface; co-operative processing TCP/IP interface; tools for writing APs; TIME facility
	APL2 for Windows version 1.0	\$1500	Product No. 5639-d46, part number 4229558.
	APL2 Runtime environment	\$250	Part No. 39L8419 - Packager and runtime CDs. One additional runtime install \$200, 5 additional \$900, 10 \$1,500.
	APL2 for Sun Solaris	\$1500	Product No. 5648-065.
	APL2 for AIX 6000	poa	Product No. 5765-012.
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Envt Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyadic Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
J Software Inc.	J on the Web online registration ...		
	J Educational Edition	\$95	
	J Standard Edition	\$295	
	J Professional	\$895	
	Books and accessories (discounts for reg users)		
	J Dictionary	\$50	
	J User Manual	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Concrete Math	\$40	
	Exploring Math	\$50	
	J User Conference Proceedings	\$35	
	Mugs, T-shirts, Mousepads	\$10 each	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.

	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APL 68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL 68000 Level II Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL 68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small, simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Fully fledged Windows development environment.
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Software\Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.

Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win.
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+Win version and later a LINUX version.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009

	(AGSS)	\$500	for Workstations (OS/2, Aix, Solaris), Product Number 6764-092
		\$2500	for CMS, Product Number 5764-011
INFOSTROY	APL+PLUS/Xbase Interface (11/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark
	<i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>		
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQLAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	AP127/PC	poa	ODBC interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP interface for Dyalog APL/W
	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	ISAP	poa	MS Windows DLL for calling Dyalog APL/W from web pages via MS Internet Information Server. Visit www.lingo.com for a demo.
	AP747	poa	ODBC Interface for Dyalog APL/W
	QWIN	poa	Legacy DOS Windowing support for APL+Win
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +11 v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.

	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book. (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Ajay Askoolum	Consultancy	poa	APL+Win development and migration of actuarial, financial, mathematical applications.
Andrews	Consultancy	poa	APL programming and analysis, Year-2000 legacy systems, algorithms, tree-processing.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
AUSCAN Software	Consultancy	poa	APL software development, training
Carracho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.

DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APLW.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa	Moving applications between platforms. Client/server development. Multilingual user interface.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management; conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APLW. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
George MacLeod	Consultancy	poa	Design and programming of new APL applications. Enhancing and maintaining existing APL applications. Porting existing APL applications from one APL system to another. Supporting users of APL applications. Experienced on both mainframe, UNIX and PC APL interpreters.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting

MicroAPL	Consultancy	poa	Technical & applications consultancy.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning; for all APL versions (10+ years experience)
Object Oriented Ltd	Consultancy	poa	General APL consulting, code recycling - mainframe to PC, performance tuning.
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy	poa	A range of consultants specialising in all areas of pharmaceutical, industrial and financial systems with 5-15 yrs experience on both PC and mainframe.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interlacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 20 years experience. Custom software development & training, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Weighahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.

APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL France		Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG) Bern		Part of Qity SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada	Gimme Arrays!	Monthly Meetings, APL skills database, J SIG, Toronto Toolkit	\$25

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@melbpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 0JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email: sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@apiborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnfeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software/Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: elandau@cais.com
Ajay Askoolum	Ajay Askoolum	42 Hanworth Road, Redhill, Surrey RH1 5HT Tel: 01737-771643 Email: ajay@askoolum.freereserve.co.uk
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@interlog.com
BACUS	Joseph De Kerf	Rocinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@gofis.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Kornemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1, Heifer Mill Cottages, Mosterton, Beaminster, Dorset DT8 3HG, England. Tel: +44 (0)7931 370304. Email: earthspot@aol.com
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10129 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652 Email: codework@codework- it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark

Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mailbox.swipnet.se
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749328 Fax: 01622 749365 E-mail: martin@gsoft.freemove.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4extra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominiq, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept RENF40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOTROY	Alexei Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 312-2673 Fax: +7 812 311-2184 Email: aalm@infotroy.spb.su
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3280-0411 Fax: +81 (03) 3280-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteingasse 13/26, 8010 Graz, Austria. Tel: 0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
J Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (416) 925- 6096; Fax: +1 (416) 488-7559 Email: info@jsoftware.com
KJK-Ileto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56,00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.htv.fi
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 6JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo_allegro@mail.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 28 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: akinloch@globalnet.co.uk
George MacLeod	George MacLeod	37 Newhouse Rd, Bovingdon, Herts, HP3 0EJ, UK. Tel: 01442-831385 Fax: 01442-831388 Email: george.macleod@slmcorp.com

Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263643 Email: Ellis@mrflrm.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Theo Zwart, Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: info@oasis.nl or rijkse@oasis.nl
Object Oriented Ltd	Walter G. Fil	Am Grindel 2, CH-6004 Luzern, Switzerland. Tel: 41 41 418 70 70 Fax: 41 41 418 70 77 Email: info@object-oriented.com
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	1st Floor, 10 Raleigh Court, Priestley Way, Crawley, West Sussex, RH10 2PD, UK. Tel: 01293 562700 Fax: 01293 562699 Email: mailbox@optima-systems.co.uk
Qualedi Inc.	Ann Wilkie (Marketing) Ed Shaw (Technical)	Merritt Crossing, 440 Wheelers Farms Road, Milford, CT 06460, USA. Tel: +1(203) 882-9988. Fax: +1(203) 882-9891 Email: awilkie@qualedi.com; eshaw@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 313, Newtown, CT 06470, USA. Tel: +1 (203) 270-9729 Email: rendata@apbooks.cnxhost.com or orders@apbooks.cnxhost.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eller	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@ret.fi
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoe@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Laurie Howard	Soliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email: sales@soliton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (952) 470-7345 Email: sales@jsoftware.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulthielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 C.Serve: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org

Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adaytum Limited	www.adaytum.com/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/rb/apljourn.htm
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
COSY (Bob Armstrong)	www.cosy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de/ ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.olap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.insight.dk/infostroy/
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
J Software Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	www.mackaykinlochltd.co.uk/~admkl/
MicroAPL Ltd	www.microapl.co.uk/
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/

Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualifiedi.com
Renaissance Data	www.aplbooks.cnchost.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.jsoftware.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

ZARK Newsletter Extracts

introduced by Jon Sandles

Here is another Zark newsletter extract. We were running short of issues to work from, but many thanks must go to Gary Bergquist who has once again supplied us with some more copies so that we can continue this series of reprints for some time to come. Below we reprint the solutions to the COMBIN problem, which throws up some immense variety, particularly in the resulting timings. Note: Jonathan Barman points out to me that whilst writing this article up he came across a dynamic function that is shipped with Dyalog APL version 9.0 (cmat is shipped in the dfns workspace), which performs the same task. Jonathan found that it was about 4 times slower than the fastest solution presented below. (It is however faster than the other solutions.) However, this proves once again that there is always worth in revisiting old problems with new technology at hand. There may be new, more elegant ways of solving an old problem, which may also turn out to be faster. Please don't be afraid to try and shed new light on the problems being discussed in this column.

Utility Corner: Combinations

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In last issue's LIMBERING UP column, we asked you to write a function named *COMBIN* that returns a matrix of all combinations defined by its arguments. For example,

```
3 COMBIN 5
1 2 3
1 2 4
1 2 5
1 3 4
1 3 5
1 4 5
2 3 4
2 3 5
2 4 5
3 4 5
```

In this example, *COMBIN* returns all combinations of 5 items taken 3 at a time. In general, for *M COMBIN N*, the result is an integer matrix with $M!N$ rows and M columns, in which each row is a distinct combination of the numbers $1..N$.

What makes this problem interesting is its hidden complexity. While simple to explain, it's not so simple to solve.

We received a wide variety of solutions. For this problem, there seems to be an inverse relationship between simplicity and efficiency. The simplest functions were slowest. The fastest functions were obscure. Take your pick: clarity or speed.

(Note: In the following presentation, we look liberties. The functions received have been changed to make them work in either origin, to be consistent with one another, and in some cases to be a bit faster or a bit clearer [we think].)

The simplest approach was to recurse. Here's a recursive nonlooping solution from Howard Peelle:

```

      ▽ R←M COMBIN N
[1]  ⌈ Doubly recursive algorithm.
[2]  →(M=1,N)/L1,L2
[3]  R←⊖IO,1+(M-1)COMBIN N-1
[4]  R←R,[⊖IO]1+M COMBIN N-1
[5]  →0
[6]  L1:R←(N,1)⌈N 1!N
[7]  →0
[8]  L2:R←(1,N)⌈N N!N
      ▽

```

Maria Wells sent us another recursive solution that includes a bit of nesting logic to keep you on your toes:

```

      ▽ R←M COMBIN N;A;P
[1]  ⌈ Recursive nested algorithm:
[2]  →(M=1,N)/L1,L2
[3]  A←+ \ (P+1+N-M)⌈1
[4]  R←+, [⊖IO] / (⌈P), "A+(M-1)COMBIN" N-A
[5]  →0
[6]  L1:R←(N,1)⌈N 1!N
[7]  →0
[8]  L2:R←(1,N)⌈N N!N
      ▽

```

As mentioned above, these simple functions paid the price in efficiency. Howard's function was 14 times slower and Maria's 10 times slower than the fastest function submitted. (All functions were timed on an APL*PLUS II system for the expression

4 COMBIN 15, which returns 1365 rows. Timings vary for different arguments and different APL systems.)

The next function, also submitted by Howard Peelle, uses a "truth table" algorithm. Here's the approach: each row in the result of COMBIN can be viewed as the result of the expression $T/\iota\rho T$, where T is an N -element bit vector with M 1s. Find all the different combinations of N bits (there are 2^N of them); select just those with M 1s; and use the resulting bit vectors to compress the vector ιN .

```

      ▽ R←M COMBIN N
[1]  ⍝ Truth table algorithm.
[2]  ⍝ All combinations of N 0s and 1s:
[3]    R←(Nρ2)⊔(⊔2*N)-⊔IO
[4]  ⍝ Pick those with exactly M 1s:
[5]    R←,⊔⊔ϕ(M=+R)/R
[6]    R←(M:N),M)ρR/(ρR)ρ⊔N
      ▽

```

Because of the 2^N component in this function, it slows down dramatically as N gets larger. It's also prone to *WS FULLS*. For 4 COMBIN 15, the function was 92 times slower than the fastest function received. (When a function gets that slow, you find yourself rooting for the *WS FULL*.)

Another "select-from-all-combinations" approach, which was used in a function from Craig Tien, is suggested by viewing the result of COMBIN after subtracting ιM from each row. For example,

```

      (3 COMBIN 5)-(3!5),3)ρ⊔3
0 0 0
0 0 1
0 0 2
0 1 1
0 1 2
0 2 2
1 1 1
1 1 2
1 2 2
2 2 2

```

The result is a table of base-3 digits (or, in general, base- P digits where $P+1+N-M$). Translate the vector $\iota P*M$ (in origin 0) into its base- P digits. Select just those sets of digits that are in ascending order (eliminating, for example, 0 1 0, 0 2 0, and 0 2 1).

Add each selected set to $\mathbf{M}$.

```

v R←M COMBIN N;P
[1]  a The nonlooping counting method.
[2]  P←1+N-M
[3]  a Create a table with elts between 0
[4]  a and P-1 to add to vector  $\mathbf{M}$ .
[5]  R←(MpP)T(1P*M)- $\square$ IO
[6]  a Keep only those in ascending order:
[7]  R←q(^(1 0+R)≥ $\bar{1}$  0+R)/R
[8]  a Add vector  $\mathbf{M}$  to each one:
[9]  R←R+(pR)p $\mathbf{M}$ 
v

```

This function, like the one before it, has a tendency to get slow and is *WS FULL* prone on large problems. However, it grows according to $(1+N-M)*M$ rather than $2*N$, so it's not too bad for values of N and M that don't differ much. For the 4 *COMBIN 15* test, it was 17 times slower than the fastest function received.

Henri Brudzewsky used a similar approach but looped to avoid the *WS FULL*.

```

v R←M COMBIN N;A;I;II;J;MM;P
[1]  a The looping counting method.
[2]  MM←M!N a Number of combinations
[3]  A←1M a First combination
[4]  R←(MM,M)pA a Prepare the output mat
[5]  I← $\square$ IO a Row counter
[6]  P←M/1+N-M a Base values to increment
[7]  J←P+II+0 a Unpacked counter
[8]  MM←MM+ $\square$ IO a Termination inded
[9]  L←(I≥MM)/0 a Exit when done
[10] R[I;]←A+J a E.g. 1 2 3 4+0 0 1 2
[11] II←II+1 a Next increment
[12] J←P+II a Unpack increment
[13] J←[ $\backslash$ J a Eliminate previous values
[14] II←P+J a Re-adjust increment
[15] I←I+1 a Inded of next row
[16] →L
v

```

On line 13, $\backslash$ is used to quickly ignore all sets of bas-P numbers that are not in ascending order. While *WS FULL*-resistant, this function is somewhat slower than the previous one (for 4 *COMBIN 15*), running 27 times slower than the fastest function submitted.

The last two functions are the fastest and the most obscure. What they have in common is their *lop-by-column* approach. Instead of working on the rows of the result of *COMBIN* (as do the previous three solutions) or on the submatrices of the result (as do the two recursive solutions), they build one column at a time.

The function from Jack Zinn loops from the first column to the last. Jack noticed that each column in the result of *COMBIN* contains numbers from the set $C+1:P$, where $P+1=N-M$ and C is the column indeed (in origin 0). For example, from *3 COMBIN 5*, the first column contains numbers from 1:3, the second from 1+1:3, and the third from 2+1:3.

The first column from *3 COMBIN 5* is 6 3 1/1:3. Notice that 6 3 1 is 2!4 3 2. The second column is 3 2 1 2 1 1/1+1:3. The numbers 3 2 1 come from 1!3 2 1, and their pattern from the indices 1 2 3, 2 3, 3 or $(0+1:3)$, $(1+1:2)$, $(2+1:1)$. From these observations, and the power of induction (or luck), Jack developed the following function:

```

▽ R←M COMBIN N;C;I;IP;P
[1]  ⍝ Working from first column, formula:
[2]  ⍝      R←A[I]/B[I]
[3]  ⍝ where A is say 3!5 4 3; B is say
[4]  ⍝ 1+1:3; I starts at 1P and then
[5]  ⍝ becomes a function of the previous
[6]  ⍝ element of I, where the segment
[7]  ⍝ starts at that value of I and
[8]  ⍝ continues to the end of 1P
[9]  IP←(I+1P+1+N-M)-⊞IO ⍝ Origin 0 inds
[10] R←((M!N),M)ρ0 ⍝ Build all-0 result
[11] M←M-~C+⊞IO ⍝ Index of last iteration
[12] L1:R[C]+((M-C):⊞IP+M-C)[I]/(IP+C)[I]
[13] →(M<C+1)/0 ⍝ Exit if done
[14] I→,/(I-⊞IO)+1''(P+⊞IO)-I ⍝ Next inds
[15] ⍝ I←ε(I-⊞IO)+1''(P+⊞IO)-I ⍝ If APL2
[16] →L1
▽

```

This function is more than twice as fast as any of the previous functions (on *4 COMBIN 5*), but is still 5 times slower than the fastest function submitted.

The function from Ken Gradney loops from the last column to the first. Given APL's right-to-left orientation, it's perhaps fitting that this function should be the fastest. The moral: screw your head on backwards before working with APL.

For *3 COMBIN 5*, here's how Ken's function proceeds:

```

3      2 3      1 2 3
4      2 4      1 2 4
5 →    2 5      1 2 5
        3 4      1 3 4
        3 5 →    1 3 5
        4 5      1 4 5
                2 3 4
                2 3 5
                2 4 5
                3 4 5

```

In each iteration, you replicate the previous result by row-indexing, and then stick a new column to the left of it. For example, the second array above comes from the first via:

```
R←(3 2 1/1+13),R[1 2 3,2 3,3;]
```

Again, after a healthy dose of induction, the general pattern becomes apparent (to Ken anyhow). Here's the fastest function:

```

▽ R←M COMBIN N;D;E;F;G;P
[1]  A Formula, working from last column:
[2]  A R←(F/D),R[(1+F)-F] PLUSIOTA F;]
[3]  A where D starts as (M-1)+1+N-M and
[4]  A is decremented by 1 after each col;
[5]  A F starts as (1+N-M)ρ1 and is cumu-
[6]  A latively summed from the end after
[7]  A each column. The algorithm for
[8]  A E←A PLUSIOTA F is:
[9]  A E←F/A-+1+0,F
[10] A E←E+1ρE
[11] A giving (A[1]+1F[1]),(A[2]+1F[2]),...
[12] A The +1+0,F term for this col hap-
[13] A ens to be the A term for the next.
[14] E←(1F+N-R+M-1)-1IO
[15] D←R+1P
[16] R←(P,1)ρD
[17] P←Pρ1
[18] L1:→(1IO>1+D+D-1)ρ0
[19] P←+1P
[20] G←+1+0,F+φP
[21] E←F/E-G
[22] R←(F/D),R[E+1ρE;]
[23] E←G
[24] →L1
▽

```

When asked what insight led to this algorithm, Ken said, "PLUSIOTA. The MONIOTA and PLUSIOTA function are the two most versatile and efficient utility functions in my bag of tricks."

For those of you who haven't used these functions, here's a listing of each:

```

▽ R←MONIOTA LEN
[1]  A R←(1LEN[1]),(1LEN[2]),(1LEN[3]),...
[2]  A In APL2: R←1`LEN
[3]  R←LEN/1+0,+1LEN
[4]  R←(1ρR)-R
▽

```

```

V R←A PLUSIOTA LEN
[1]  a R←(A[1]+1LEN[1]),(A[2]+1LEN[2]),...
[2]  a In APL2: R←εA+1LEN
[3]  R←LEN/A-1+0,+LEN
[4]  R←R+1pR
V

```

```

MONIOTA 4 3 5
1 2 3 4 1 2 3 1 2 3 4 5

10 20 30 PLUSIOTA 4 3 5
11 12 13 14 21 22 23 31 32 33 34 35

```

Both functions work properly in either origin.

LIMBERING UP: Polyadic Functions

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

To buy a house, you need to borrow 90000. The bank will lend you the money at 9% interest, provided you agree to repay the loan in equal monthly payments for the next 30 years. What will be payment be?

Your task is to write a function named *MORTGAGE* that returns the monthly repayment amount for any specified loan amount (90000), interest rate (.09), and term (30). (In this example, the monthly amount is 724.16.)

So you don't have to waste time looking up the formula, here it is:

```

V←÷1+INT÷12 a Monthly discount rate
PMT←PRIN÷(V-V*1+12×TERM)÷1-V

```

INT: Nominal annual interest rate
PRIN: Principal (amount borrowed)
TERM: Number of years of loan
PMT: Monthly payment

Wait a minute!! Since the APL code is already written, where's the limbering?

Glad you asked. This is a design task, not a coding task. This function requires three arguments (*INT*, *PRIN*, *TERM*), not one or two. Since APL permits monadic or dyadic functions, but is a bit weak at triadic functions, what function syntax should you use?

Whatever design you choose, test your function on the following problems:

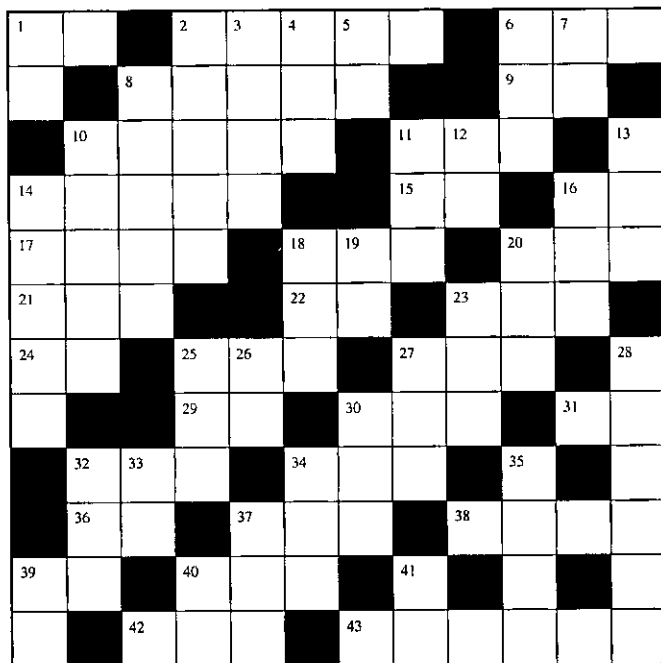
1. Interest is 9%; principal is 90000; term is 30.
2. Interest is 9%; principal is 90000; term is 15, 20, 25, 30 (4-element vector result)
3. Interest is 8%, 9%, 10%; principal is 90000; term is 20, 25, 30 (3-element vector result).
4. Interest is 8%, 9%, 10%; principal is 90000; term is 15, 20, 25, 30 (3 by 4 matrix result).
5. You get the idea.

Mail your solution to:

Vector Production
Brook House
Gilling East
YORK YO62 4JJ
UK

Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated 23 Ketchbrook Lane, Ellington, CT06029, USA

Crossword



Across

1. All done
2. $A \geq \sim \rightarrow C[D]$ for Boolean A
6. The remainder of $B \div A$
8. How many elements in each row of D do not match the corresponding element in vector B?
9. Round C up to the nearest integer
10. $A \equiv B$ for same length vectors A and B
11. N formatted to D decimal places
14. The largest number in the vector $E \times B$
15. $1 \rho C$ for scalar C
16. How many
17. How many numbers in vector A are bigger than B (omit the A)?
18. From $1 B$ select N random numbers, without repeats
20. The Boolean vector B with all but its first 1 converted to 0s
21. $\otimes (*C) \times (*B)$

22. $, +/B=B$ for vector B
23. $(+/V[1\ 3\ 5\ 7\ \dots])-(+/V[2\ 4\ 6\ 8\ \dots])$
24. $B[]$ to put B in sorted order
25. $(\sim V \in D)/V$
27. Once V is true
29. $(+\backslash C \rho C=C) \rightarrow \neg \neg \neg I O$ for scalar C
30. Product of V
31. $(\phi \iota \rho \rho N) \delta N$
32. $(N+B), (N \div B)$ for positive N on vector B, where $N \leq \rho B$
34. 1-poles in Boolean vector V to 1-maps:
 $0\ 1\ 0\ 0\ 0\ 0\ 1\ 0\ 0 \rightarrow 0\ 1\ 1\ 1\ 1\ 1\ 0\ 0\ 0$
36. $\times / \iota C$ in origin 1
37. $(\times / V[1\ 3\ 5\ 7\ \dots]) \div (\times / V[2\ 4\ 6\ 8\ \dots])$
38. Half of V, rounded to an integer
39. A times pi
40. The leading true bits in Boolean V
42. The smallest number in vector V
43. $M \wedge \sim \wedge / V$, for matrix M, vector V

Down

1. Go to line L
2. $+ / A \times B$ for vectors A and B
3. $0 \neq + / A \in B$ for scalar B (omit the A)
4. $(D < B) \vee (D > B)$
5. What's hiding in the scalar A?
6. The values in A, but not less than N
7. The absolute value of C
8. Which columns of matrix B have numbers that are all less than the scalar D?
10. $A + \uparrow / B$ for scalar A, vector B
11. Stick vector D onto the left side of matrix B
12. The number C in characters that look like it
13. The largest number in vector B (or negative infinity if B is empty)
14. The indices to extract from $(E \rho 0)$, D the elements of D sorted according to the collating sequence C
16. Cumulative sum of V
18. $D[N \rho \iota \rho D]$
19. $(\iota 0) \rho 1 \div B$
20. $V[1] < V[2]$ for two-element vector V
23. Cumulative alternating sum
25. Where B is first seen in vector V
26. $C=0$ for Boolean C
27. Are any of V true?
28. $(\rho N \rho V[1]), (\rho N \rho V[2])$ in origin 1
30. Cumulative product of V

32. The number of different combinations of A items taken N at a time
33. $C[(\square IO + \rho C) - \iota \rho C]$
34. Parity of Boolean vector V, i.e. $2 \mid + / V$
35. $, + / V \neq 0$ for vector V
37. Cumulative alternating product
39. Circumference of circle of diameter D
40. All
41. M, enclosed into a nested scalar

Solution to Crossword in 17.4

[Errata: There were a couple of typos in the last crossword. In 6 down Lawrence Welk was mis-spelled as Lawrence Wall; 8 down should have been The greater of BV or M; and 19 and 20 down were marked one square too far left on the grid.]

V	~	N		(	A	1	v	B	)	/	C
¶	K	C		+	/	^	\	V	>	9	9
N	L		-	/	B		S	Γ			ρ
	€		(	v	/	V	⊆	M	)	ι	1
	T	-	+	\	M	R	V		S		7
2	5			φ	A	A		□	A	V	
0		T		V	T	+	.	=	V		1
?	1	0		≠		V	5	-	E	*	2
X	1	A	,	B	,	C		/		1	5
S		M		)	E	R	A	S	E		↓
	R	←	M	↑	R	,	P	φ	D		Z
		0		V	A	R	I	A	B	L	E

[Vector Production] Following an interesting run-in with Word2000 and shaded table-cells, the crosswords are now being generated in NewLeaf and linked to the Word document as metafiles! Moral: never upgrade your word-processor under time-pressure.

J-ottings 29: Power Steering Extra

by Norman Thomson

With the last chords of the Euro-song well wafted on their way back to Estonia, memories of an earlier era come to mind in which such things seemed to have had more "stickability" than now. One song in particular, rendered by a certain Mr. Richard, promised "power to all our friends", which seems to me not a bad epithet for J, and perhaps for other members of its family, namely A and K. (As an aside, since C and Z are already spoken for, I presume that there are 21 available slots left, unless of course there are some languages floating around which nobody has bothered to tell me about.)

All of which is a roundabout way of saying that the power conjunction is one of the most versatile computational tools in J. At the same time there are traps for the unwary which must be avoided. Consider the following statement:

$2^{*^4}1$ and $1^{*^4}2$ mean "multiply 1 by 2 four times" and "multiply 2 by 1 four times" respectively, which are the same thing because multiplication is commutative, that is, 1 times 2 is the same as 2 times 1.

Obviously true? No – the answer is that the left operand is glued more closely to the "times" so that the first of these expressions is properly interpreted as "multiply-by-2" applied four times to 1 giving 16, and the second is "multiply-by-1" applied four times to 2 which gives 2. J is in this sense "left-handed", or, as the more technically aware would say, left binding is stronger than right.

Next, here is the standard matrix product verb to allow some work with matrices:

`mp=.+/ .*` NB. Matrix Product

To self-test your understanding of the power conjunction, cover up all but the next two or three lines and answer the questions: what do the following do to a matrix

(a) $mp^{~^2}$ and (b) $mp^{~^3}$?

If your answer was that it raises it to the fourth and eighth powers respectively – well done! In general $mp^{~^n}$ raises a matrix to the power 2^n on account of the way in which each result is "fed back" to the next exponentiation step.

Suppose you didn't want to go up in quite such big leaps, how about

(a) $m \wedge 2$ m and (b) $m \wedge 3$ m ?

This time the answers are the third and fourth powers of m. In general $m \wedge n$ raises m to the power n+1, which is irritatingly "one out" if you are looking for a tidy verb for m^n .

To find such a verb, go back to the scalar case $2 \wedge (4) 1$ which raised 2 to the power 4. The key ingredient is the 1, that is the identity of multiplication. A verb which gives the identity matrix of the same shape as any square matrix is $id = . = @i. @\#$, following which a verb which raises a matrix x. to the power y. is

```
power=.dyad def 'x.mp^:y. id x.'
```

in which x. is a matrix and y. may be a vector, so that several powers of a matrix can be obtained simultaneously. For example

```
m2
4 1
_2 1
m2 power 10 9
117074 58025
_116050 _57001

38854 19171
_38342 _18659
```

Now compute the ratios of two successive powers:

```
ratio=.%/@power
m2 ratio 10 9
3.01318 3.02671
3.02671 3.05488
```

This demonstrates that the items in the above matrix all tend to a value which is in fact the value of the largest eigenvalue of the matrix, which for m2 happens to be 3. Eigenanalysis at this early stage? – heady stuff! Press on and use this value 3 to obtain a value for the smallest eigenvalue:

```
3+(m2-3*id m2)ratio 20 19
2 2
2 2
```

You don't have to be exact with the multiplier 3, but the closer you are to the true eigenvalue the faster is the convergence:

```

5+(m2-5*id m2)ratio 20 19
1.9994 1.9997
1.9997 1.99985

```

As an aside – in the verb definition `id.=@i.@#` you might think that the `=` is “equals”, but bearing in mind that the left conjunct of `@` must be monadic, `=` in this definition is *self-classify*, which returns an equals table for the *nub* of a vector *y*. versus itself.

The above experiments need not be confined to two by two matrices, for example:

```

m3
1 1 1
1 2 1
1 1 3
m3 ratio 10 9
4.21425 4.21415 4.21442
4.21415 4.21392 4.21456
4.21442 4.21456 4.21418
4+(m3-4*id m3)ratio 20 19
0.3249388 0.3244097 0.3258422
0.3244097 0.3278868 0.3184167
0.3258422 0.3184167 0.3383134

```

showing that the largest and smallest eigenvalues of *m3* are about 4.214 and 0.325. Given that the sum of eigenvalues is the sum of the leading diagonal, the third eigenvalue is readily calculated as 1.461. As for the eigenvectors, it is convenient first to define a verb based on a simple hook `%>./` which normalises a vector so that its largest item is 1, or in the case of a matrix, it does this for each row separately, hence the *transpose* and the rank conjunction:

```

norm=.(%>./)"1%|:
norm m3
      1      1      1
    0.5      1 0.5
0.3333333 0.3333333 1

```

Now observe the behaviour of the successive (normalised) powers of *m3* as they converge:

```

norm m3 power 10
0.5254314 0.6889001 1
0.5254368 0.6889113 1
0.5254221 0.6888809 1

```

Each row when fully converged is simply an eigenvector corresponding to the largest eigenvalue, and repeating the previous “smallest eigenvalue” wrinkle:

```

norm 4+(m3-4*id m3) power 20
_2.07862      1  0.4032745
  1 _0.4818991 _0.193451
  1 _0.4797006 _0.1949656

```

results in rows each of which converge to the eigenvector corresponding to the smallest eigenvalue.

It is not of course suggested that this is the basis of a systematic way of calculating eigenvalues and eigenvectors – for one thing there are numerous special cases which have to be looked out for, and the choice of adjustment factor for the smallest eigenvector must in general be made with some care.

The value of the power conjunction for obtaining, say, quick population estimates is readily apparent. Suppose that `transm` is a transition matrix giving the proportions of juveniles, workers and elderly who change from one status to the next within any one year, so that, for example, 94.71% of juveniles remains so at the end of the year, the birth rate for the workers group is 5.52% and the death rate within the elderly group is 5.7%.

```

transm
0.9471 0.0552  0
0.0379 0.9682  0
  0 0.0207 0.943

```

If the current population (in millions) is 11 38 and 9 for the three sectors, the predicted values in 20 years time are

```

trans mp^:20 11 38 9
26.7554 31.9539 11.1334

```

As before the ratio between successive powers shows convergence to the largest eigenvalue:

```

transm ratio 100 99
1.00458 1.0046  0
  1.0046 1.00459 0
  1.0049 1.00457 0.943

```

This is approximately 1.005, which is the instantaneous overall rate of population increase.

For a matrix which has a column with only non-one zero item, that item is necessarily one of the eigenvalues, and the others are the eigenvalues of the matrix obtained by removing its row and column. Thus 0.943 is an eigenvalue of `transm` and the remaining eigenvalue is the smallest eigenvalue of

```
0.9471 0.0552
0.0379 0.9682
```

Returning to the power conjunction, there is another smart feature of J, namely "function raised to the power infinity". As observed earlier, the function values are repeatedly fed back as input, and if this process ultimately leads to convergence, then $\wedge: _$ will deliver it. For example given

```
cos=.2&o.
```

successive values of $\cos y$. $\cos(\cos y)$. $\cos(\cos(\cos y))$... starting from an initial value of 0. are given by

```
cos^:(i.6)0
0 1 0.5403023 0.8575532 0.6542898 0.7934804
```

and where these converge, as they do for **cos**, the converged value is the solution of the equation $\cos(y)=y$:

```
cos^:_(0)
0.7390851
```

This feature is useful in the sort of population study which concerns, for example, the spread of AIDS within a third world population. Suppose that the current population is 60 million and an 8% birth rate is assumed, a percentage which is subject to a downward adjustment of 10^{-7} per unit of population to account for pressures of space, food shortage and so on. Assume also that there is a net annual emigration of 10,000. Forecasting future population levels is a matter of repeated exercising a polynomial defined by

```
p=.#.&_0.000000001 1.08 _10000
```

so that the projected population in 20 years time is:

```
p^:20(60000000)
7.50634e7
```

The infinity option gives the ultimate equilibrium population when the forces of environmental pressure and migration exactly balance the birth rate:

```
p^:_(60000000)
7.98748e7
```

that is equilibrium is attained at about 80 million.

This sort of thing is what I mean by saying that there is a lot of power in J – don't forget to convey it to all your friends!

RECENT MEETINGS

APL2001 AT YALE

Acknowledgements

Thanks to Ray Cannon for writing up his notes, and to both Ray and Bob Hoekstra for the photographs.

My APL 2001 at Yale Diary

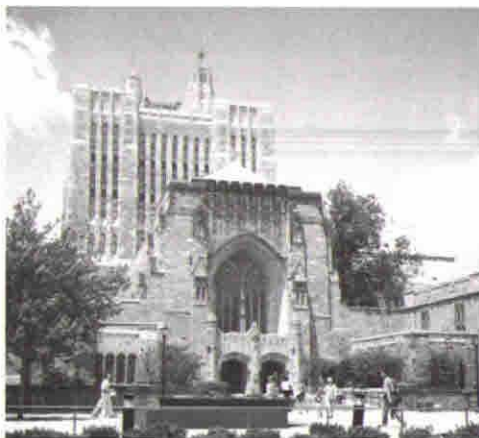
by Ray Cannon

This year's annual international APL conference was held in the "Ivy League" university of Yale in the town of New Haven, Connecticut. Being a mere two hours by (commuter) train from New York, I started and ended my conference week in New York, which increased the contrast between the university and the city.

Sunday, June 24

I started the conference by arriving late for the opening reception, which was held in the grounds of the New Residence Hall, where I had booked a room for my stay.

The opening reception was well attended by both delegates and mosquitoes. But having spent an interesting day in Greenwich Village, watching the annual New York Gay Parade, a most entertaining event, I was very late arriving only 30 minutes before its scheduled end and missed out on most of the food. All that was left at the end was two trays of assorted cheese. These were not wasted however, and reappeared at several subsequent "evening seminars" where they complemented the Russian Vodka.



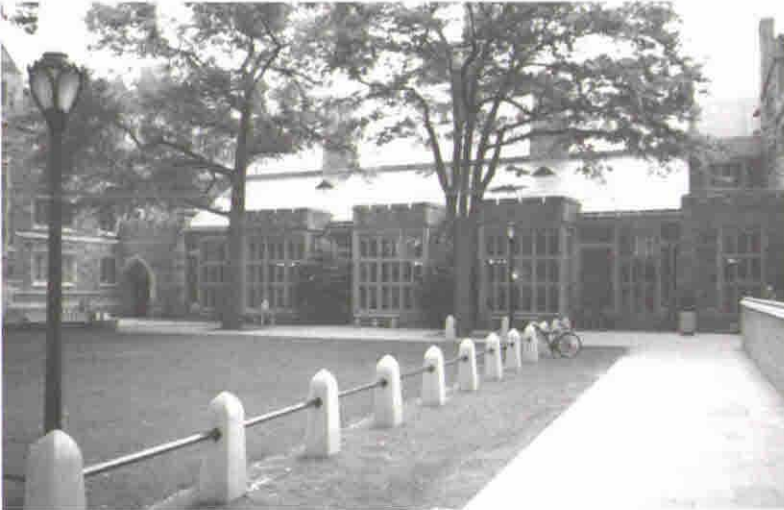
The conference accommodation was split between three locations, the New Residence Hall, the Holiday Inn and the Omni Hotel, all within a 10-minute walk from the main conference site located around the Davis Auditorium. The halls were modern and well equipped, comprising of sets of two bedroom suites each with shared bathroom, kitchenette and living area. (I was sharing with Bob Hoekstra.) These were more than adequate for our needs and far better than the facilities I have seen at any other

university. The nearby Holiday Inn was the location for the main "evening seminars" sessions, which I attended without fail.

Monday, June 25

The opening session was a talk on "Digital Fusion" given by the computer graphics expert and distinguished ACM lecturer, Judson Rosenbush. His talk started on the theme of "mould" in analogue media. That is printing uses "type" which is a mirror image of the printed result, vinyl records are pressed in a mould, and even photographs are produced via a negative. He then went on to explain how modern digital techniques were reducing production costs. If you buy a printed item, the publisher bears the cost of the paper and ink, and the distribution of it. If you purchase the same information off the Internet, you download the data and print it yourself, and YOU bear the cost of the paper and ink, and the cost of distribution ("on-line" costs while downloading.) The handling of digital information is independent of the type of information (image, text, sound, film). What we still have to digitise is taste, smell and feel. The final note I made from this talk was "power now is in the hands of who can turn off your IP number."

I next attended a tutorial given by Gary Bergquist entitled "APL and the WWW". Gary's session, was as ever, an enjoyable romp, with its feet firmly placed in the idea that you can "do it in APL" via the right set of utilities.



Lunch was provided as part of the conference in the Berkley Dining Hall, alongside some of Yale's other summer students. The food was simple but plentiful and quite let down the mock medieval splendour of the dining hall. The seven-minute walk between the conference centre and the dining hall passed through several of Yale's "quads" and lawns with interesting buildings all around.



After lunch was the first of the Vendor Forums. This one from IBM APL2, presided over by a very laid back David Liebttag. David ran through the new features added to APL2 since the last conference, including such items as ODBC support ("as AP127 for DB2, but for ODBC"), partial line marking (copy and pasting partial lines), allowing lower case APL names on the mainframe service, and

allowing bitmaps on buttons. David has and still is working hard on the APL2 product across its range.

The next tutorial I attended was given by Steve Abell (a self confessed "Java Evangelist at Netscape"). Steve's talk was both entertaining and thought-provoking and included the following quotes (as best I can remember them):

APL "an intellectual achievement of the 1st magnitude"

ALGOL60 "a wonderful improvement on its successors"

"Multiple inheritance is too dangerous for many people" but then "APL is too dangerous for many people"

The final session of the day was booked as the Dyadic Vendor Forum, but was actually part one of three, one-hour sessions on "DotNet Technology". This first session given by John Daintree was on "The Concepts and Implementation of .NET classes". I think this is the third time I have heard John give this talk, for which I am grateful, as it is now only just starting to sink in.

The day's formal proceedings were completed with the Opening Reception held in the President's Room in the Rotunda of Woolsey Hall, where we were wined and dined till 9pm.

I then joined a group who were "seeking further liquid refreshment" and wandered towards the New Haven town centre until we found a suitable watering

hole. As pleasant as this was, we were sadly dismayed when we discovered the CT liquor laws meant that all bars closed at 10pm. So we then made our way back to the Holiday Inn and Monday's Evening Seminar, complete with Vodka and Cheese.

Tuesday, June 26

Somehow, I don't think breakfast at 7:30 and session starts at 8:30 quite fit in with the Evening Seminar. Thus I seem to have missed three very good conflicting sessions: Tilman Otto's tutorial on "Compiling APL" (which Bob Hoekstra informed me was great), Eric Baelen on "APL and Excel" (which came complete with an excellent handout, which might make it as a future article in Vector) and Steve Mansour's paper on "A Dynamic APL GUI Equation Solver" (which I kept hearing people talk about in impressed tones).

I did however take this opportunity to talk with John Daintree, who gave me some advice on how to extract some information from Windows about the width of the client area in the Dyalog APL/W version 9 session. (Which I will write up as a "Hacker's Corner" for Vector if I ever get it working <grin>.)

The first session I can remember was the featured speaker Richard Krafchin on "AnEmotion: Animating Email". This talk described how PackageWorks (a well established leader in "providing dimensional direct mail solutions and marketing expertise to major corporate clients") have developed with the aid of APL2000 (and in particular Fred Waid) high calibre animated e-mail marketing campaigns. The APL system used (patent applied for) allows "graphic artists to use a variety of multimedia tools to produce award-winning interactive presentations and animated e-mail". (If the term "junk mail" comes to mind, equate it to "fish paste" and then equate AnEmotion to the most expensive caviar.)

Lunch was followed by the Soliton Vendor Forum of which I can remember very little of either.

I next attended Dave Liebttag's tutorial on "Using Namespaces" in APL2. Here Dave showed how easy they were to use (and to me how different they were from namespaces in Dyalog). One memorable highpoint was Dave telling us to ignore the many pages in the manual just do it "this way", then when asked who wrote the manual, he said that he did.

Cory Skutt's tutorial on "APL Renovation" included an overview of a prototype of a modified APL called Genera. Running under both Dyalog APL/W and IBM

APL2, Genera is "intended to seriously propose and evaluate qualitative design enhancements". All in all, an interesting and thought-provoking session.

Peter Donnelly then gave the second session from Dyadic on DotNet Technology "Calling Web Services from APL". Notable from its absence, was the lack of "ducks" at Yale, but this was completely made up for by the inclusion of Peter's "Booking System for Golf Clubs" featuring remote processing across the net using the "DotNet" technology.

Following the only formal evening activity laid on (a short walking tour of the University) a group of us descended on an excellent Thai restaurant where we enjoyed a splendid meal and several bottles of wine. It was here that I was persuaded that I should not miss this years APL2000's User Conference to be held in a beach side golf-resort-hotel at Naples, Florida in early November.

Unfortunately, due to the strict CT liquor laws, Evening Seminar back at the Holiday Inn finished early, when in the first time in sober memory, the "Vodka ran out". <gasp> , but then so did the cheese. ("Ran" might not be quite accurate, but I am quite sure by this time the cheese was well able to get up and walk out by itself.)

Wednesday, June 27

I only just managed to get in on time for the Morgan Stanley Vendor Forum on their Open Source APL system A+, hosted by John Mizel and Jon McGrew.



In the past, I have never thought of Morgan Stanley as an APL vendor, more of a client of APL vendors but, with the release of A+, they could be said to have joined that small group. However, vendor implies selling, and since A+ is not actually being sold, more being given away, I think they should be classified as APL suppliers.

No matter, the session was excellent, and it gave me a good insight into this extremely fast dialect of APL. The

session was complemented with two tutorial handouts and free copies of A+ on CD.

After the morning coffee break, Bo Gehring was the featured speaker, for the impressive "Audio Immersive Environments".



The program stated "Bo, founder of a leading computer animation studio in Hollywood during the 1980s, will talk about communication models which apply graphics analytics to 3D audio displays, the acoustic side of immersive environments. He will demonstrate an advanced Internet user interface for a fully immersive, 360-degree window into large databases, which may work alone or to enhance a graphic display". This intrigued me so I searched the Internet for "Bo Gehring". I found he worked on graphics for many films including "2001" and "Star Trek - The motion picture". So here was a leading player in computer graphics, talking about 3D sound.

His talk was fascinating and included some amusing sound clips including a helicopter attack. A few of his key points were: "Sound is DATA"; "Hearing is the only truly immersive sense"; we need to "move display to include audio"; headphones need to be equipped with gyroscopes to counter head tracking (that is the virtual sound source should NOT move as your head moves when wearing headphones). He also talked about the military use of 3D sound, say for giving pilots audio warnings of a threat with "built in" direction component. One last note I made was "Moved sound with a mouse".

Lunch was followed by the APL2000 Vendor Forum. As always, this was informative, interesting and realistic. APL2000's latest release (APL+Win v4.0) was shown. This release includes :

Enhancements to Replicate

- Larger Files
- Enhancements to Common Controls
- Improved Syntax for Working with Active Objects
- Grid Object

Eric Baelan put up one slide entitled "State of APL in My Humble Opinion". On it he suggested the different strategies adopted by the different vendors:

- APL200 – Business Applications, Consulting
- Dyadic – Partnership with Microsoft .Net and VB converts
- Sharp – TimesSquare and Linux
- IBM – Cross Platform commonality

Other slides of particular interest were about APL2000's presence on the Internet.

"OpenHere"

OpenHere is now ranked in the top 50 search engines by MediaMetrix and Nielsen, based on unique users each month.

OpenHere spiders add or remove approximately 1 million sites each six months.

OpenHere has over 250,000 categories and over 1.7 million sites listed, making it one of the three largest search/index sites on the net.

Spidered over 100,000 times a day.

As to APL+WIN scope:

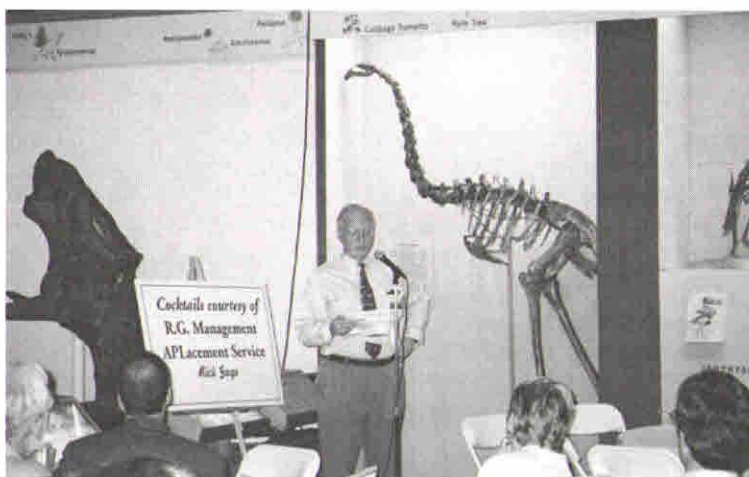
- Millions of pages per month
- Stability – Server up for over 4 months
- Millions of users require a system with constant uptime
- Scalability – load changes by time of day, week month and year
- Managing databases over 10 gig in size
- Approximately 1/3000 of entire internet.

Eric finished off with an open invitation for us to attend this year's APL2000 User Conference, to be held in November in Naples, Florida. I for one have already got my family's airline tickets booked.

The next item I attended was "A Cannon for the Representation of Multivariate Arrays In the APL2 Computing Language" presented by Richard Stockbridge

about self documenting data. Unfortunately the only notes I made on it were written in the margin of my conference program – “Did not understand this”. I should have read the paper first as published in the Proceedings.

The final session of the day was given by John Daintree of Dyadic, on “DotNet Technology: Developing Web pages using APL. Vector really must get John to write it up. I certainly can’t explain it, even after hearing it 3 (or is it now 4) times.



The final formal proceeding of the day was the Conference Banquet, which was held in the Dinosaur hall of the Peabody Museum of Natural History. It was a very apt setting, all these fine old relics of a by-gone age in static poses, complete with a backdrop of Dinosaur bones. (Well that joke went down no better now than it did during the banquet.) The one down point of the evening was the APL2001 conference committee singing. Please, I beg you, don't give up your day jobs.



During the evening, Jon McGrew was awarded the “Kenneth E Iverson Award for Outstanding Contributions to APL”.

Thursday's evening seminar was very well attended, and well supplied with lubricants. Live music was supplied by Brown and Mansour, an up and going “one night stand” folk/rock group featuring Mansour on Mandolin and Jim (virtual Bob) Brown on Guitar. I also seem to recall at one stage, a very nice young lady joined the party, who claimed to be a roadie from a pop group who were performing in town.

that night. However, the only photographic evidence I have is of the 4 or is it 5 empty Vodka bottles.

Thursday, June 28

My Thursday started of with a tutorial on "Using A+", given by John Mizel and Jon McGrew. Although excited by it at the time, I still have not loaded the CD and tried it out at home, but once I have finished writing this diary, I might have time.

The closing session of the conference was given by Paul Judah, the Chairman of the Yale Computer Science Department, on "Computer Research at Yale", a quite interesting talk. I particularly enjoyed the slides on "Perlis Epigrams", a few of which I will repeat here:

Everything should be built top-down, except the first time.

Every program has (at least) two purposes: the one for which it was written, and another for which it wasn't.

It's easier to write one incorrect program than understand a correct one.

It is easier to change the specifications to fit the program than vice versa.

(See <http://www.cs.yale.edu/homes/perlis-alan/quotes.html> for the complete list.)

Following the formal end of the conference, I returned to New York City with Bob Hoekstra (BH) and Bob Armstrong (BA), where we all went to BA's Manhattan apartment near the Brooklyn Bridge. BH only stayed the one night but I remained at BA's place for a few evening seminars, and the Coney Island Mermaid parade. However, that's another story.

NavSim – a Shipping Simulator: Part I

by Tomas Gustafsson

This paper was previously published in the FinnAPL newsletter, having been presented at the Finnish Forest Seminar in February 2000

Background

This is a hobby project that I started to work on already some years ago. Due to various other tasks that appeared during the time, there have been interrupts of as much as 18 months in development. Even so, a great interest in navigation, boating and shipping, plus some previously gathered experience in the area keep me determined to make the output of this work something "very nice".



Ships going through a narrow passage. The frigate in the middle has in its attempt to catch the Russian spy turned too much to port, and may touch the sea bottom. To left a building, to right a light-house. The 3D models are downloaded for free from the Internet.

As this system as a whole will be more or less huge, I will not be able to describe all of it in a single paper. Therefore, I will in this "Part 1" cover some basic goals and way of implementation. More parts will follow as the project goes on.

The Purpose

Nobody will be surprised to hear that there is in fact no purpose with this project. It is pure hobby work. On the other hand, going on the sea has, and will always be, an essential way of transportation. There is no reason why a hobby project like this could not spin off various applications for educational, commercial and leisure usage later on.

I have heard that the Finnish archipelago is the second-largest in the world, after the one of Greece. The waters are full of islands and rocks, thus indeed being an interesting subject to simulate in a computer. The waters are also filled with shipwrecks, thus also being a sensible subject for simulating.

The Goal

1. To create a virtual reality with static and moving objects, nicely rendered in 3D, with colours, lighting and smooth movement.
2. To make the world "stay alive" without any particular interaction by the user.
3. To make the objects much smarter than "the locomotive".
4. To give the user freedom of movement.

Let us take a closer look at the goals.

1. **To create a virtual reality with static and moving objects, nicely rendered in 3D, with colours, lighting and smooth movement.**

Uuuhhuh. Said in one sentence, that statement covers maybe one year of work. In addition, it should have said "authentic", that is; there is some point in having an imagined landscape, but it makes certainly more sense to have a real-world landscape, with islands at their correct positions and in their correct shapes. This should be possible in the near future, as the Finnish Maritime Board and Karttakeskus ["map central"] are working on "vectorizing" the sea charts. What would be more nice than inserting the CD with 2D map data and getting a real 3D image of it on the screen?

That may sound like dreaming, but the fact is that we need the underlying data in any case. Collision-detection, path selection and optimizing, depth figures, lighthouse positions etc. are all a fundamental part of the system functionality.

Current commercial PC implementations of GPS-aided navigation use bitmaps for visual presentation of the chart. Objects may then be inserted by absolute coordinates and by using appropriate paint routines. The bitmap is there only for its visual appearance – I used to say that there should as well be a picture of Lenin instead. Inserted objects like polylines and circles are painted on top of the bitmap but they are in no way directly connected to the objects seen in the underlying bitmap.

And that makes it a question, or rather a problem of bitmap accuracy. The Finnish Maritime Board has a huge database with measurement data, upon which they paint the sea charts. These are stored in special conditions to maintain their physical accuracy. Various commercial charts have then, at least until now, been produced from the original ones and put into distribution. PC applications have used scanned images of the paper charts.

Some vendors have, in frustration at the lack of vectorized charts, decided to do the vectorization work themselves, and by using low-wage workforce in Estonia and Russia created numerical chart databases. Despite the mathematical description of the landscape they gain, the source of data is still the potentially inaccurate paper chart. When in addition considering the maintenance problem connected to this method, it is not unfair to say that we have a completely dead end here.

Once the numerical charts are available, we have the ultimate data source. One must note, that the visual appearance of the virtual reality will be inherited from the chart data and only a limited part of that will actually be used for landscape creation. Sea charts contain quite some information that is only there for navigational purposes. The intention has never been to describe what the landscape looks like when seen with the human eye. The 'most static' part of the landscape, the islands, can be converted to 3D objects by using contour vectors to form height meshes. If there will be actual height curve information available, we can create the meshes according to that. Otherwise, it will not be 'too wrong' to estimate the heights. Anyone who has been navigating in the Finnish archipelago can tell that all the islands look the same when seen from a distance.

'Pleasure for the eye' we want to create! Chart data includes positions of certain static "landscape objects" like buildings, bridges, ferry stands. This simulator must definitely show these, because we know the user is hungry for them. In practice this means that we must have a library of 3D models, choose among those and place them into the world.

In some cases the numerical data may contain information about the quality of the coast line. This will affect the selection of textures. Where the cliff extends out to

the water, we must use a brown texture (or white during winter time), where grass and water meet, we use a green texture for the outmost edge of an island.

The archipelago contains lots of objects for traffic control. These are lightshouses, rummers, line-markers etc. We want to show them in the simulator, thus again performing picking of objects from a 3D object library. In addition, these objects have a meaning for the run-time functionality of the simulator. Some of them have dynamic behaviour, like change of light sequence or colour, or small movement like the one of an anchored rummer. More details about these follow later in this text.

To complete the overview of items in the virtual world, we have all the moving objects. Birds, boats, ships, airplanes, balloons, etc. etc. The boats and ships, of course, are of primary interest with respect to the artificial intelligence. Other things are for viewing pleasure only, but even so, do form an important part of the whole, as we want to imitate something real. All these will be handled by introducing different classes of objects, each with own logic of behaviour.

Other things affecting the visual appearance

- **Time of day**
Ships are slow, and the daylight changes quickly. In this kind of an outdoor environment, the sun is the primary and only main light source. In addition, there are lots of small point lights attached to various objects, but these have no meaning for the overall light. The sun acts primarily as source of ambient light but on clear days, it must also have the character of a point light.
- **Time of year**
In northern Europe, the landscape looks completely different in summer and in winter. The summer is green, the winter is white. In addition, we have ice in the winter. These variations can be solved by using textures, like using a white texture instead of a blue one to show frozen water.

Also, we need to take into account the angle to the sun. This affects directly the gross amount of light. By fairly simple approximations we will be able to get real-looking outdoor lighting conditions.

- **The weather**
The weather dominates, especially at the sea. The weather is generated by a "weather generator". It is a timer-driven utility that uses some user-defined settings, the time of day and year, and pre-defined (hard-coded) limits to vary things within. The weather generator does not work on the basis of true meteorological phenomena like low pressure, high pressure, moving air masses, etc. Instead, we maintain sets of key weather data like density of clouds, fog, rain etc. A weather data set is a bunch of numbers sufficient to describe the weather at a location. These data sets exist at different locations in

the outdoor environment and they move more or less at random. To get the weather at any location, we interpolate using the few closest weather data sets. For example, if we are at a location where one data set is located 10 kilometres to the north, another 20 kilometres to the south and a third 15 kilometres to the west, we will be able to create a "current location data set" by interpolating – the data set to the north will have the biggest influence and so on. By using this method, we get smooth changes in weather as we move around in the world. We also get changes in weather at a static location, as the data sets move. In addition, we can manipulate the data in the sets according to certain rules, as for example creating lots of fog in early morning.

2. To make the world "stay alive" without any particular interaction by the user.

This is the chosen way of implementing the artificial intelligence (AI). Each intelligent object reacts on the data it is able to gather, that is, it acts from its own point of view. Each object is given the same data about the surrounding world, e.g. where are the ships, which is any ship's current orientation, direction, speed and so on. If the user has taken control over one ship, the other ships still see it as 'any ship' – they do not realize that the captain on that ship actually is a real human mind, not coded AI.

Any intelligent object has a primary goal. Depending on the AI class of that object (is it a bird, is it a ship?), it may be able to re-conclude its primary goal. For example: a bird does not have any certain primary goal, other than "just flying around" or "just hang on to that ship". If the goal is to just fly around, we can let the object move by the lines of a polygon. From point A to point B to C to D to E to A, for example. To make it look more realistic, we add distortion to the "next direction value". Say the current direction to point B is 0 degrees, which is to the north. Let the code add an inaccuracy to the value, and make the (erratic) conclusion that the correct direction is 38 degrees. If the bird flies in that direction for 30 seconds, it will get out of path. When 30 seconds have gone, and a new iteration of the direction happens, the system may conclude that the correct direction to B is 352 degrees. Maybe this time, the system adds no error to the value, and so the bird begins to fly in 352 degrees' direction. Eventually it gets within range of B, or in program code within a circle surrounding B, and the system concludes that it has reached B and now will have to fly towards the next point, C. And so on, until the circle is completed; then do the same thing again.

Another way to specify movement of a bird is to let it fly behind a ship. We can achieve that by having an imaginary cube behind the ship. Let the bird always fly within that cube; whenever it hits a wall, change the direction to the one of a reflection of a beam. That is, if the bird hits a wall in the imaginary cube with an incoming angle of 10 degrees in a vertical plane (XY or ZY plane), change its direction to -10 degrees counted from the same plane – it kind of bumps away

from the wall whenever it hits it, by the rule of "incoming angle equals outgoing angle". Of course, it is all the time facing the ship it is following.

A ship may be free to find out a new primary goal, when it has finished the current one. Say the ship arrives in harbour A. Now it is free to go to one of harbours B to Z. By using simple "economical rules" (yes, money is always involved) it may find out that the most profitable thing to do now is to get peanuts from harbour F and ship them to harbour K. It must, of course, be able to ship peanuts and it must be able to go in to the harbours F and K. Now, the next task is to find the path to harbour F – in program code a series of waypoints concluded on the basis of sea chart data. When that is done, the ship will start to go towards the first waypoint and begin a journey full of surprises, problems and adventures.

The natural question raised now is: what "drives" all this? The answer is: timer objects. For a bird, the timer may trigger every 30 seconds. Every 30 seconds the program code re-calculates the direction to fly, based on the current location and the target location. Then the bird flies in that direction for 30 seconds, and so on.

For a ship, which carries a "captain object", the re-calculations need to be much more dense, mostly with intervals of one to two seconds.

3. To make the objects much smarter than "the locomotive"

Maybe not the best choice of words here, but the intention is to say that a locomotive goes in one direction only, and that is "ahead – where the rails go". Doing that for a ship would equal giving it the intelligence of a bird. This is not at all rare in PC applications, for example in a PC game called Command and Conquer, which has artificial intelligence, many of the moving objects advance on hard coded paths.

Giving higher levels of intelligence means more data, more iterations – and more errors. As the world contains hundreds of objects, we must classify them. There is no reason to re-calculate the direction of a bird every second, since it is an object of less importance, there for visual appearance only. And there is certainly no reason to have complex calculating algorithms allocated to it, while there are many reasons for doing that for a ship. We must have a good level of organization and classification in code, to be able to keep everything "in hand". Thus we say that a bird and a ski-jet may belong to the "dumb AI class" while a small craft and a ship belong to the "high-intelligence" class. There are objects in between, which have certain behavioural patterns making them belong to their own classes. For example a lighthouse does nothing else than turn on and off its light, but still it may be considered to have some kind of intelligence, since it is timer-driven and may even malfunction occasionally!

So how do we make the ships intelligent? The answer is that we put human brains on them, namely captains. The ship itself has no intelligence, only characteristics like engine power, displacement, maximum speed etc. A captain is a class of objects with no visual appearance, mostly it is a set of numerical values for age, aggressivity, experience, eye-sight, politeness and so on. The ship provides, or does not provide, means for the captain to get input data. If the ship is equipped with a radar, the captain knows the exact distance to a ship. Otherwise he has to rely on the eye, and that may be quite inaccurate.

The captain executes his actions by using the ship's manoeuvring devices. These are for example the rudder, the engine output and likewise. It is actually the captain who makes the decision to go from A to B. If the ship stands still at point A, the captain will issue a command to the ship, to increase speed to for example 5 knots, or he will just say "give me 30 % output". Then the ship's dynamics functions take care of things like acceleration, rotation, speed, direction. The captain has these figures handy all the time, depending of course on what electrical devices the ship carries. More details about dynamics vs. AI functions will follow later in the text.

4. To give the user freedom of movement

Freedom of movement, that is a task purely for the 3D engine to handle. When talking about virtual realities, what is seen on the screen is said to be the "camera view". The 3D engine is able to handle at least one camera. That camera should have the same type of behaviour that any other object has; it can have a location, a direction, a speed, an acceleration etc. By attaching a camera to the bridge of a ship, we get onto the PC screen what the captain sees if he stands on that bridge. The attaching means only, that the camera moves with the ship, although they are two different objects.

Freedom of movement (FOM) is one of the basic requirements for this simulator. Examples of simulators with limited FOM are battle tank simulators, where one can indeed drive around with the tank, but one can not move within that tank. First person shooting games like Quake usually allow full FOM, but only within the pre-specified arena. MS Flight Simulator again gives FOM like in a tank simulator, but in addition the altitude can vary, i.e. the camera does not follow the ground.

What we want to achieve here is the superset of the above. The ship can go anywhere on the XZ plane (the horizontal plane); the ship can have slight changes in altitude and elevation by reacting to waves; the user (or the captain) can walk on the ship bridge, and may even be able to walk on different decks, all depending on how the ship object was created with the modelling tool.

This simulator must allow the user to "take over the command" of any ship, or alternatively to just let the user go there to take a look at the world from the bridge, while the AI carried on that ship still has the command and takes the actions.

The programming tools and hardware

Dyalog APL

APL is hopefully the only language for the application code. Dyalog APL is very fast, and as long as there are no expensive repeated calculations, like rotating lots of vectors between each rendered frame, we will solve every coding issue with APL only.

A "3D engine"

The engine is (or should be) a complete set of functions for setting up and manipulating three-dimensional objects at run-time. 3D engines are available as third-party commercial products, at a variety of prices and qualities. Most of the engines are created for C programmers only, and thus not directly suitable for an APL programmer with no C experience. The preferred solution is a set of DLL or ActiveX functions to handle it all.

It is the responsibility of the engine to handle cameras, objects, lights and special effects like fog, rain, waves etc. We can use many different programs to create objects like ships, for example AutoCad or 3D Studio. The engine must be able to load these objects and show them with correct shading.

The most affordable ones cost around 100 US\$, the expensive ones are 10,000 US\$ or more. Clearly, a hobby project like this cannot allow an investment of that kind, and I have urged some vendors to create "light" versions of their engines. Indeed have I also been given promises to get such and at the moment I am waiting for one with lots of features, like multi-monitor support, at a cost of 1000-2000 US\$.

The engine must allow APL to manipulate objects with simple commands, like:

```
SetOrientation [object] [Xorientation] [Yorientation]  
[Zorientation]
```

or

```
SetVelocity [object] [Xvelocity] [Yvelocity] [Zvelocity]
```

Windows (with DirectX or OpenGL, maybe also other platforms)

All versions of Windows contain Microsoft's DirectX implementation for 3D graphics rendering. Windows 95, 98 and 2000 utilize hardware with 3D acceleration support, while Windows NT does not support 3D acceleration in hardware.

All versions of Windows support OpenGL, either directly or with upgraded components. Preferably, the 3D engine should be independent from rendering technology, thus supporting the most common present and future platforms. It is important for an APL programmer to not get involved with the rendering technology directly! Instead, the 3D engine must offer a common API for the application code and by reacting to user calls, or by investigating the hardware, be able to select appropriate rendering technology.

Hardware acceleration (3D graphics cards are now very affordable)

The computer used for developing this simulator is a 650 MHz Athlon with a Voodoo 3 graphics adapter. When writing this, it is six months old, but already one can buy 900 MHz processors. The speediest graphics adapter is currently the Nvidia Riva based "Leadtek Geforce" card, which can be bought in Finland for about 300 US\$.

Some basic rules

This is complex, and it contains artificial intelligence. We cannot run ahead in blindness, if we do, we get quickly into serious trouble. Here are some basic rules that we must follow:

1. OOPS

We must structurize and classify everything. The virtual world contains objects, as APL programmers we are used to working with objects. APL objects must be "individuals hanging around", that is, data sets in shared memory. Any object must be given relevant environmental data upon request. At any point in time, an object must be able to "look around" and make a conclusion.

2. Definable variables

During development it becomes apparant, that initial values are wrong. It must be easy to manipulate practically all data for an object. This means that there must be a GUI available for that. We cannot go into code each time we want to change a property of an object. This means lots of input data forms, but it definitely pays to have them. This also makes it easier to specify and document data structures. A

good clue here is to implement "data aware controls", like the way Causeway can handle edit fields and ensure that a named variable always gets updated with any new value the user enters in the field.

3. Data structures

Objects have usually 10's of single numerical values describing various things. A ship may have 150 numbers describing everything, starting from 'length over all', 'displacement', 'engine type'. Thus, we cannot keep all individual values in separate variables, even if they reside in different namespaces. We must use data structures. But, during code development the programmer will find numerous new things to define or keep track of. There must be appropriate cover functions to handle structures with flexibility, so that an addition to a structure does not invalidate existing code anywhere.

4. Functions also as objects

There will be objects that use common functions for common purposes. For example, ship dynamics for all ships may be handled by a single "dynamics function"; a ship object calls that function once a second to update speed, rotation, acceleration etc. But, it will turn out that small ships need different calculation algorithms than big ships. Thus, when handling object movement, we must be able to let an object call a function, but we must also be able to easily change that function into another one. And we do NOT want to go into hard code to do that change. The preferred way is to define that function in a GUI form, maybe select from a list of functions. Thus some APL functions are more object-like by nature than others.

5. Timer driven

"How does it all happen?" I think you, as the reader, are still questioning yourself that. Timers do the triggering, for example: once every 30 seconds, a Timer calls an APL function which evaluates the position of a bird, and takes or does not take actions. Once a second, a Timer calls a dynamics function for a ship, checks the current engine output, the speed, the water resistance, and determines whether the speed should be increased or decreased. Then it sets the new speed value for that object, and the 3D engine moves it slightly between each rendered frame. The timers give us excellent options to maintain continuous calculation and fine-tune it.

6. "I give the orders, you (the 3D engine) execute them"

Object movement is basically to show a picture of it; move it and show the next picture; move it again and show the next picture, and so on. This could be done in APL code, but it should happen in the 3D engine code, due to performance reasons. Thus, as the APL coder, we do not want to get closer to the 3D engine than saying "5 metres per second", or "1 degree per second". Once we have said that, we expect the 3D engine to execute the order until we say differently, even if we go away for a cup of coffee.

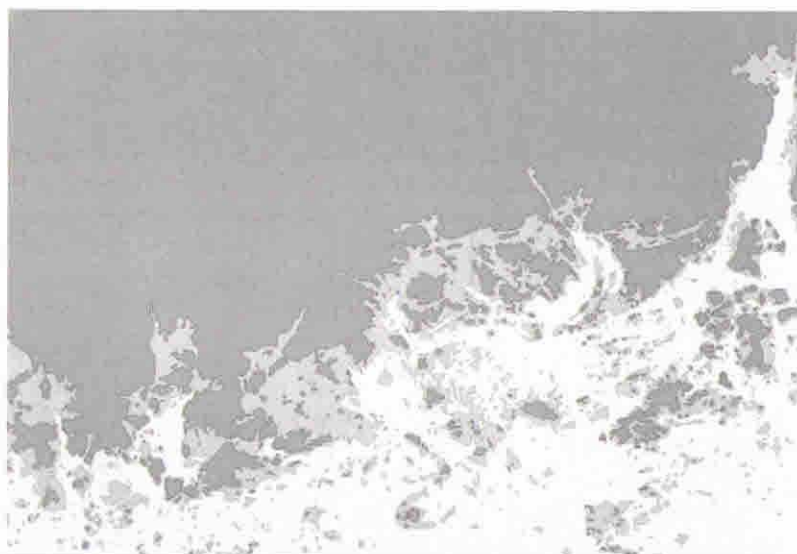
7. Think 2D, show 3D

Luckily, we move on the water surface only, with the exception of special effects like sinking. When looking (and programming) from above, it is much easier to understand and handle movement. Also, the sea chart is a two-dimensional thing. The 3D is there to make it nice to look at; but much of the program logic can happen in 2D.

8. Buy the objects, do not manufacture them unless completely unavoidable

How much time do I need to create a ship with AutoCad? Maybe one week, or three weeks. This is not how an APL programmer should work, instead he should go into the Internet and search for free objects. Eventually, when he is ready to release the product, he may consider actually paying for objects that need a fancier look.

The next part of the story will cover details of data structures and contain code examples...



Extract of the 2-dimensional view: sea chart scanned, converted into a bitmap and then vectorized with APL. The elements in the picture are not bitmap colour values but filled polygons. The point data resides in a nested APL variable. Execution time to find the around 4000 BLOBs: A few seconds. To left Lauttasaari, to right the inlet to Porvoo. Actual colours are: green, yellow, white (darkest → lightest)

Web Image Galleries Made Routine

by Clifford A. Reiter (reiterc@lafayette.edu)

When contemplating the creation of lots of Adirondack hiking web pages, the ability to easily create image galleries was an essential consideration. Part of the goal of those web pages was to create so many hiking galleries that the sheer magnitude of the collection was meant to make viewers smile in bewonderment. If a typical hike has 50 good pictures and we create the corresponding 50 thumbnails and we have ten Adirondack hikes per year for ten years then that will be 10,000 hiking pictures. The "Take a Hike with Cliff" web pages [5] don't have 10,000 hiking pictures yet, but there are thousands of images there. Most of those were created using early versions of the scripts discussed here.

This note gives an introduction to the "image" addon. The addon contains facilities for accessing 24-bit images and creating html galleries; it also contains a lab which introduces the addon facilities. The access to images in various formats means the ability to convert between three dimensional J arrays of RGB triples and files in some conventional format, such as JPEG.

Making web galleries routine means doing most of the work under program control. Even if you don't have massive collections of hiking pictures to organize, the J html gallery building tools can be handy for a lot of boring, but nice, projects: for sharing pictures from department picnics, of students giving talks at maths conferences, and of family vacations. We begin by creating an example web gallery and then regress to more basic utilities.

Creating A Gallery In One Line

First we need to download and install the image addon from [1]. The html building script that comes with that addon is loaded as follows.

```
load 'addons\image\html_gallery6'
```

The directory '\hike\' contains a working copy of the images for the gallery created below (I save a backup of the original images). A web-linked sequence of image gallery pages showing all the images in that directory is created via the following J expression.

```
6 4 mk_html_gallery_seq 200 175 images_in '\hike\'
```

There were 73 jpeg images in the directory. Running the above line resulted in 73 thumbnail images being created and four linked html gallery pages being created. The first gallery page is shown in Figure 1. Notice that the path name had a closing back-slash. While there is much more to say, the resulting galleries are quite useful.

It took about three minutes to create the above gallery sequence on a 750 MHz PC. Each source image is a 2048 by 1536 by 24 bit jpeg image which is typically slightly larger than 1MB. As uncompressed data, the image collection amounts to around 70 MB. Galleries of web-size images are processed much more quickly. For example, creating a similar gallery for 73 web images with an average size of 35k only took 14 seconds.

What Did I Get With One Line?

The above line that created the image gallery has two main parts. The function `images_in` results in a list of all the images in the specified path. The conjunction `mk_html_gallery_seq` creates html galleries containing those images.

The result of the function `images_in` is a boxed list of all the files with *jpeg*, *jpg*, *png*, *tga*, or *bmp* filename extensions in the specified directory. The filenames are in grade-up order (alphabetic except for lower versus upper case and special symbols); also, filenames beginning with "th_" are excluded since this is our default prefix used for thumbnails. If we only wanted the jpeg files in the directory, we could use the function `jpegs_in` instead of `images_in`. There are similar functions for the other file formats. All these are based on the adverb `im_in`. Users needing to match other patterns should look at that adverb and the similar adverb `files_in`.

Note that `mk_html_gallery_seq` is a conjunction. The left conjunction argument specifies the page layout; in the example it was 6 4. The right conjunction argument is the thumbnail bounds; in the example it was 200 175. The right argument of the resulting function is the list of images to include in the gallery. There is an optional left argument specifying the names of the gallery pages to be used, in which case the left conjunction argument only specifies the number of columns to use in the thumbnail array.

The command created four html "index" image galleries named *index00.html* through *index03.html*. The one line expression included 6 4 on the left; that is a request for each page to use a 6 row by 4 column table of thumbnail images. Fewer rows are used if that balances the pages better. Here we had 73 images total and if 24 images were placed on most pages, then the fourth page would have just one image. Instead, once it was determined that four pages were required, the number

of image rows was adjusted to five. The result is that each of the first three html pages has a 5 by 4 array of thumbnails while the fourth has 13 images arranged in four rows. The first html gallery page is titled "gallery" and the subsequent ones are titled with their pathless filenames.

The thumbnails displayed in each gallery maintain the aspect ratio of the original images and are the maximal size fitting into a 200 by 175 pixel window subject to that constraint. Most of my 24-bit images are landscape with a 4 to 3 aspect ratio. However, there usually are a few portrait images. Using 200 by 150 as the thumbnail size bound would make the portrait images noticeably smaller than the landscape images. Using 200 by 200 as the thumbnail bounds makes all the thumbnails the same size, but would leave a lot of white space near rows with portrait thumbnails. Using 200 by 175 is a happy compromise.

Clicking on a thumbnail image brings up the full size version of the image. Each image has a caption that is a centred, short (pathless) filename. The "alt" information associated with each thumbnail is the size, in kilobytes, of the full size image. Height and width information about each thumbnail is included so that browsers can quickly render the web page layout.

What Did I Miss In One Line?

Several things really should be changed in order to get a hiking gallery suitable for public posting. In our example, the digital camera was also used to create some *.mov animations. These are not of a supported image format and hence they are not included in the file list created by the `images_in` function. Since there are only a few of them, they are easily included by using software or a screen capture to obtain a frame. Save the frame as an image and allow a thumbnail to be created from the frame. Then edit the html file to change the reference to the frame to a reference to the animation. That is, handle special links interior to the thumbnail gallery by creating a dummy image whose thumbnail can serve as a guide to the special link.

Second, a few of the images were taken in a portrait orientation. They need to be rotated 90 degrees. This is easily accomplished. For example, our function for rotating an image file clockwise by 90 degrees, `rot_cw_90`, is given below. Notice that the main processing line is designed to rotate either 2 or 3-dimensional arrays and they may either be raw binary or numeric arrays.

```

    rot_cw_90=: 3 : 0"1
z=.raw_read_image y.
z=(1 0&((0 1))@i.@$@$ |: ]) |. z
z write_image y.
)

    (1 0&((0 1))@i.@$@$ |: ]) |. i. 4 2
6 4 2 0
7 5 3 1

    <"1 (1 0&((0 1))@i.@$@$ |: ]) |. i. 4 2 3
+-----+-----+-----+-----+
|18 19 20|12 13 14|6 7 8 |0 1 2|
+-----+-----+-----+-----+
|21 22 23|15 16 17|9 10 11|3 4 5|
+-----+-----+-----+-----+

```

Compression issues arise when dealing with jpeg images since writing a file in jpeg format loses information. We will say more about jpeg quality settings later. However, an easy way to maintain the full quality of the original image is to archive the original. This can be done using the `and_archive` adverb which puts a copy of the original image in a subdirectory named "archive". The following command would rotate and archive an image and thumbnail.

```

    rot_cw_90 and_archive '\hike\dscn0284.jpg'
832391

```

Rotating multiple images is relatively straightforward as we will illustrate in the next section.

The choice of 4 columns of images that are 200 pixels wide might not be optimal for viewing. The time consuming portion of creating the html galleries is creating the thumbnails. Thus, there are facilities for avoiding the recreation of thumbnails. Namely, we can use `update_html_gallery_seq` instead of `mk_html_gallery_seq` or simply use "0" for the thumbnail size bounds. That is, the following command will run in a flash.

```

5 6 3 update_html_gallery_seq images_in '\hike\'

```

The result would be five linked gallery pages. All the width, height and image size information was updated. The rotated image is now properly shown with its portrait orientation. However, the thumbnail has the size of the original thumbnail rotated, 150 by 200, not the 131 by 175 size thumbnail that would have been produced to fit into a 200 by 175 size window.

Note that we could also have constructed a single huge gallery page using the following.

```
3 mk_html_gallery 200 175 images_in '\hike\'
```

That gallery page maker, `mk_html_gallery`, is the actual core html page maker. There are global variables that allow for html code to be inserted before and after the table that gives the image gallery created by that conjunction. The default prolog gives the default gallery heading and the default postlog is empty.

```
ht_table_prolog
<font size = 4><B><center>Image Gallery</center></B>
```

```
ht_table_postlog
```

The conjunction `mk_html_gallery_seq` manipulates the definitions of `prolog` and `postlog` in order to create the gallery sequence. These facilities allow the creation of templates for different styles of web image galleries.

If the html gallery is primarily used for browsing an image directory, the filenames are handy information and the galleries are useful as they are. Of course, usually we will edit the captions: changing the filenames into descriptions of the images. That is the fun part of making a web page and the intention of this script is to make it easy to get to the fun part.

The Complications Of Reality

Here is an outline of a process that illustrates dealing with even more realistic complicating details. Suppose the directory `'\hike'` contains my working directory of a fresh set of digital hiking images. I keep a sample script with the following steps. However, each gallery creation seems to have its own particular features, so I tend to change the path name and run a line at a time from the script.

- I would obtain a sample frame from each movie and add it as a jpeg image to the directory.
- I would name the path.

```
path=':\hike\'
```

- I would make all the filenames in the directory lower case. I find this convenient for gradeup alphabetization and it helps me avoid case errors when uploading files to our web server.

```
dir_to_lc path
```

- I would get the list of images, checking the first couple of filenames.

```
2{.im=:images_in path
+-----+
|\hike\dscn0255.jpg|\hike\dscn0256.jpg|
+-----+
```

- I would make one large gallery (not a sequence) of all the images with large thumbnails and 3 or 4 columns, depending on whether I was working on my notebook or desktop computer.

```
4 mk_html_gallery 400 350 im
```

- I would review the gallery and decide which images needed to be rotated 90 degrees and I would create a list of those files. Often that can be done by direct selection from `im`. Sometimes I rebuild the names as below. This process is repeated for other rotation angles if necessary. Note the filename rebuilding uses `nfmt`, which is defined in the script, to give file type numeric formatting. The left argument gives the number of digits; the default is three.

```
nfmt=: 3&$: :({&'0123456789'@:([ # 10"_ ) #: ]))
]nf=:4 nfmt 255 263 264
0255
0263
0264

2{.to_rot=: (path, 'dscn')&,@(, '&'.jpg')"(1) nf
\hike\dscn0255.jpg
\hike\dscn0263.jpg
```

- I would then rotate and archive the images.

```
rot_cw_90 and_archive to_rot
```

- I would update the html gallery without recreating thumbnails (and subsequently hit refresh on the browser).

```
4 mk_html_gallery 0 im
```

- I would then select the images I want on the public web pages. Probably less than half of the total number of images. I would create a list of their names leaving out the path either by modifying a selection from `im` or by direct construction.

```
nf=: 4 nfmt 87 89 100 101 120 122
sfn=: ('dscn')&,@(, '&'.jpg')"1 nf
```

- I would name and create a directory for the web images.

```
pathw=:path,'www\'
11:5 <pathw      NB. create directory
```

- I would pick simplified names for the web size images (2048 by 1536 is too big) in the web directory.

```
2{.wim=: (pathw,'w'),"1 ] 4}."1 sfn
\hike\www\w0255.jpg
\hike\www\w0264.jpg
```

- I would create the web size images (these jpeg files are about ten times smaller than the originals).

```
1024 1024 mk_th_image"1 (path,"1 sfn);"1 wim
```

- Finally, I can make my web gallery files in a line, with the thumbnail shapes I prefer.

```
10 3 mk_html_gallery_seq 320 280 wim
```

- I then edit the link to the movies and I can have the fun of adding captions to my web hiking images.

Admittedly there is some work involved in selecting which files to rotate and which files to include in the web version, but the other steps are essentially automated in the sense that they can be run unmodified from the sample script.

Beneath the Thumbnails

Much of the html file creation depends upon being able to find the size of an image and creating the thumbnails. Much of what remains is formatting. The image size information is usually a fairly simple decoding of select portions of the information in the file. Thus, in some sense, the only computational content in the gallery creation is the creation of the thumbnails.

The `mk_th_image` function basically reads the image source file, applies the function `resize_image` to the image array and writes the resulting array to the thumbnail file. The function `resize_image`, shown below, resizes an array so that the aspect ratio of the first two axes remains the same, but so that those axes fit into the width and height specified by the left argument.

```
resize_image=: 4 : 0
szi=.2{.$y.
szo=.<.szi*<./([.x.)%szi
sc2=.] #~ [: <.&.(+/\) #@] # [ % #@]
b=.y. sc2~ {.szo
b sc2"2~ {:szo
)
```

Here are some examples of resizing an array with the shape of a 200 pixel wide by 100 pixel high image. Each of the results preserves that aspect ratio.

```
$100 100 resize_image i. 100 200 3
50 100 3
```

```
$100 400 resize_image i. 100 200 3
50 100 3
```

```
$400 100 resize_image i. 100 200 3
100 200 3
```

Within the function `resize_image` the desired resulting height and width, `sco`, are computed. They are used as arguments to the local function `sc2` which replicates the given data enough times (often 0 when making thumbnails) to result in the desired length.

```
sc2=.] #~ [: <.&.(+/\\) #@] # [ % #@]

10 sc2 2 33 114 255 11 15
2 33 33 114 114 255 11 11 15 15

4 sc2 2 33 114 255 11 15
33 114 11 15
```

What If You Don't Like My Thumbnails? (or, The Basic HTML Utilities)

While the `html` utilities were constructed with the idea of using similar sized thumbnails in the image gallery, there are times when that would not be desirable. For example, we would probably not want to change the relative size of the images in a gallery of windows forms captured from the same screen. See Figure 2 for an image gallery of screen captures. Thus, for various tasks, we may want to use more basic `html` building utilities. The details of each definition are in the script. Here is a summary chain of the main functions used to build the `html` galleries.

- `mk_html_gallery_seq` modifies the `prolog` and `postlog` and makes a sequence of calls to:
- `mk_html_gallery` which organizes input image file list, and writes the result of a call to:
- `html_fmt_gallery` which adds header and tail formatting to:

- `ht_table_fmt_images` which adds html table formatting to the table row formats obtained by:
- `ht_tr_fmt_images` which adds html row formatting to the html table data formats obtained by:
- `ht_td_fmt_image` which adds captions and formatting to the href format of an image and its thumbnail obtained by:
- `ht_href_fmt_image` which adds the href details to the html image created by:
- `ht_fmt_image` which is illustrated by the following.

```
400 300 10000 ht_fmt_image 'temp.jpg'

```

So there are just seven levels of minor details from formatting an image to formatting a gallery sequence.

Returning to our example, if `im` is a boxed list of the four images shown in Figure 2, and if we want to create a 2-column gallery without making thumbnails, then we can do the following. Note that we treat the images as their own thumbnails; we can use a 0 on the right to indicate thumbnails should not be created; and we give a named result file *gallery.html*.

```
'gallery.html' 2 mk_html_gallery 0 .~ im
```

Thus, we actually can accomplish our task with our high level functions. Nonetheless, the more basic html utilities can be useful in other contexts.

The Basic File Utilities

The core image reading and writing utilities along with image size utilities are contained in the script *addons\image\image1.ijs*. Users not needing the html utilities can access a fairly clean set of basic image utilities by using that script. The 24-bit versions of *jpeg*, *png*, *bmp* and *tga* image file formats are supported. Image data may either be read as raw data or as integer data.

For example, `B=:raw_read_jpeg 'filename'` would result in `B` being a height by width by 3 array of raw binary data assuming 'filename' was the name of a jpeg image file. Similarly, `B=:read_jpeg 'filename'` would yield an integer array with shape height by width by 3. Each rank one cell has length 3 and those cells specify the red, green and blue values of the corresponding pixel. The analogous functions, such as `raw_read_png`, are defined to handle the other

image formats. If the filename includes an extension **.jpg*, **.jpeg*, **.png*, **.bmp* or **.tga* then `raw_read_image` automatically reads the file in the corresponding format. Likewise, `read_image` works in the same manner but results in an integer array. Raw or integer arrays `B` may be written using any of the desired formats with calls such as `B write_bmp 'filename'`. If the filename has a supported extension, then `B write_image 'filename'` works fine. Png files have optional flags indicating whether to maximize compression or to run fast. See the script for details. Jpeg files have optional quality and progressive scan options. The options can be controlled globally or by using an optional left argument. For example, `85 1 B write_jpeg 'filename'` specifies that `B` be written as a progressive jpeg image with jpeg quality 85. The default settings are nonprogressive and quality 75. High quality settings give better quality images, but are larger. Here we illustrate manipulation of a hiking image and the impact of several choices for the jpeg quality setting upon the image size.

First we read an image as raw data. It is a digital photograph from the top of East Dix. This is where my wife suggested we review our life insurance needs.

```
image_wh path,'dscn0155.jpg'
2048 1536

$B=:raw_read_image path,'dscn0155.jpg'
1536 2048 3
```

We trim away the left and right sides of the image.

```
$c=.448}."2] _448}."2 B
1536 1152 3
```

Now we resize it to fit inside a 768 by 768 window.

```
$d=.768 768 resize_image c
768 576 3
```

Next we write the data with several choices of jpeg quality. Notice the file size gets much smaller as the quality decreases. You should try this with an image of your own.

```
95 d write_jpeg path,'edix95.jpg'
199395

85 d write_jpeg path,'edix85.jpg'
102424
```

```
75 d write_jpeg path,'edix75.jpg'
71205
```

```
50 d write_jpeg path,'edix50.jpg'
44167
```

```
20 d write_jpeg path,'edix20.jpg'
22529
```

```
10 d write_jpeg path,'edix10.jpg'
13410
```

```
5 d write_jpeg path,'edix05.jpg'
9475
```

In practice, quality 75 is often a good compromise between quality and size for nice quality web images. To maintain high quality without much concern for size, a setting such as 95 might be used. That would be appropriate for archiving images that might be printed someday. Casual viewers are unlikely to notice the difference between quality 75 and quality 95. However, a careful observer will notice the difference. Even a casual viewer would probably find quality 50 noticeably lacking. Lower qualities are humorous, but not usually considered appropriate for storing an image. Figure 3 shows the quality 95 and 5 versions of the image.

The script *image1* contains functions for converting between *.jpeg and *.bmp and between *.png and *.bmp. For example,

```
(jq,prog) bmp_to_jpeg 'file.bmp';'file.jpg'
```

would convert the specified *file.bmp* into the *file.jpg* using jpeg quality *jq* and using *prog* (0 or 1) to specify whether to use a progressive jpeg format. An inverse, *jpeg_to_bmp* is also defined as follows.

```
jpeg_to_bmp=: 3 : 0
inout=.'bmp'&ch_ext d_fn_ck y.
conv_jpeg_to_bmp inout
fsize {:inout
)
```

The function 'bmp'&ch_ext changes the extension of a given file to *bmp* and that function is applied, if needed, by the adverb *d_fn_ck* which does default filename checking. Thus, if only the input file jpeg name is given, then the output would have the same name with *bmp* extension. Also notice that the function

further calls `conv_jpeg_to_bmp` which is the conversion function that directly calls a dll.

```
conv_jpeg_to_bmp =: (jpeg_dll, ' jpeg_to_bmp n *c *c')&cdd
```

Thus, many of the functions in the image scripts are built indirectly upon dll calls that convert images between jpeg and bmp, or between png and bmp.

Where Did This Come From?

The image utilities are based upon a few directly defined functions and some dll's used for file conversion. The *bmp* and *tga* files are read and written using updated 24-bit versions of the bitmap utilities from the scripts associated with [4]. The jpeg utilities are based upon Intel Jpeg Library [2]. The library uses the Intel provided *ijl15.dll*. Examples in the documentation for the Intel library compared their library calls with those of the Independent Jpeg Group's. Those examples were modified and used to create *j_jpeg1.dll* which is the dll called by J. That gives support for image conversion between jpeg and bmp and jpeg image size information. However, in combination with the **.bmp* utilities, jpeg images may be read and written. The png conversion utilities were based upon Miano's examples [3]. Modified versions were used to create *j_png1.dll*.

These simple file conversion utilities make it easy to work with images in J and make html image galleries routine.

References

- [1] Image Addon, Jsoftware, see download, addons, at <http://www.jsoftware.com>.
- [2] Intel Jpeg Library (IJL), *ijl15.dll*, see <http://support.intel.com/support/performancetools/libraries/ijl/index.htm>
- [3] John M. Miano, *Compressed Image File Formats*, Addison-Wesley-Longman, 1999.
- [4] C. A. Reiter, *Fractals, Visualization and J*, 2nd ed., Jsoftware, Inc., Toronto, 2000
- [5] C. A. Reiter, Take a Hike with Cliff, <http://www.lafayette.edu/~reiterc/hikes/index.html>.

Clifford A. Reiter
Department of Mathematics
Lafayette College
Easton, PA 18042 USA

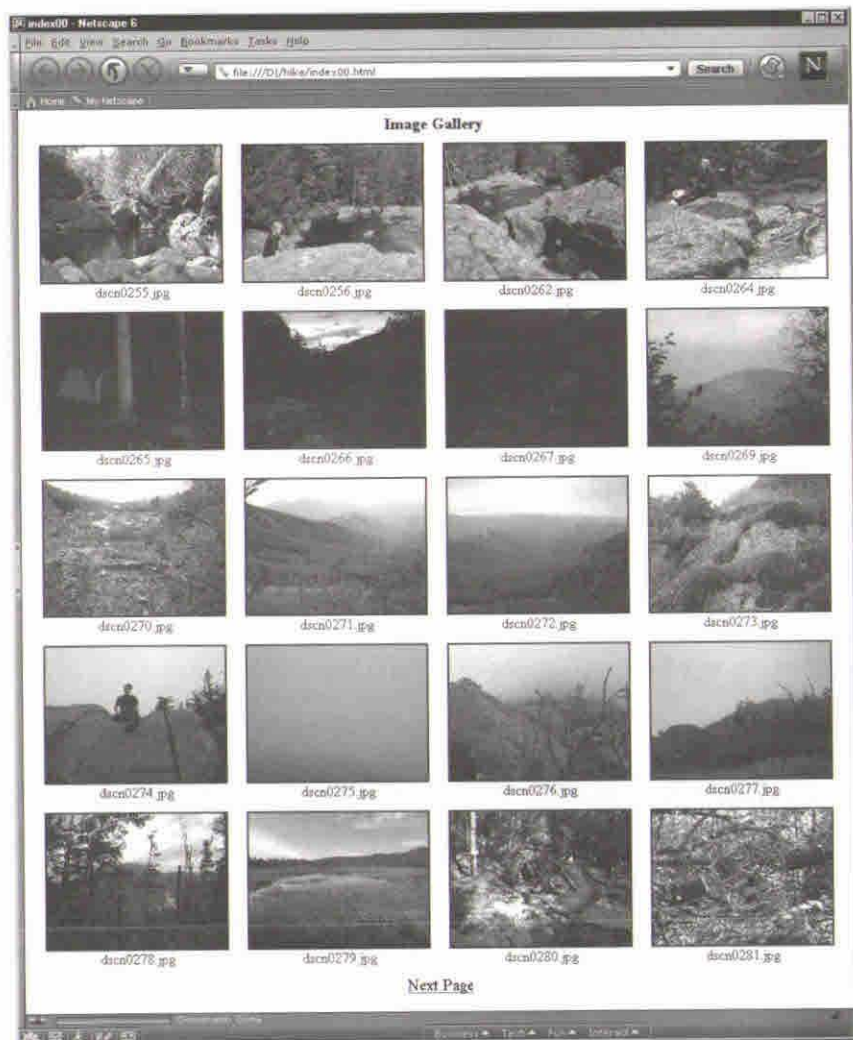


Figure 1. A Web Gallery Page Created With One Line.

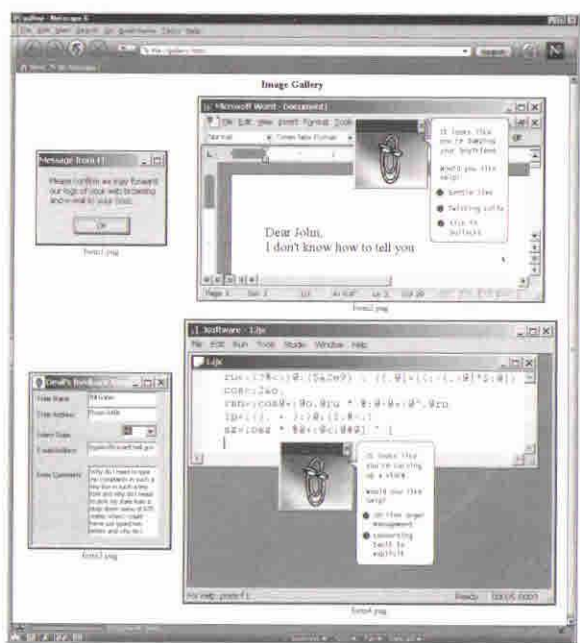


Figure 2. Screen Captures Shown without Resizing.

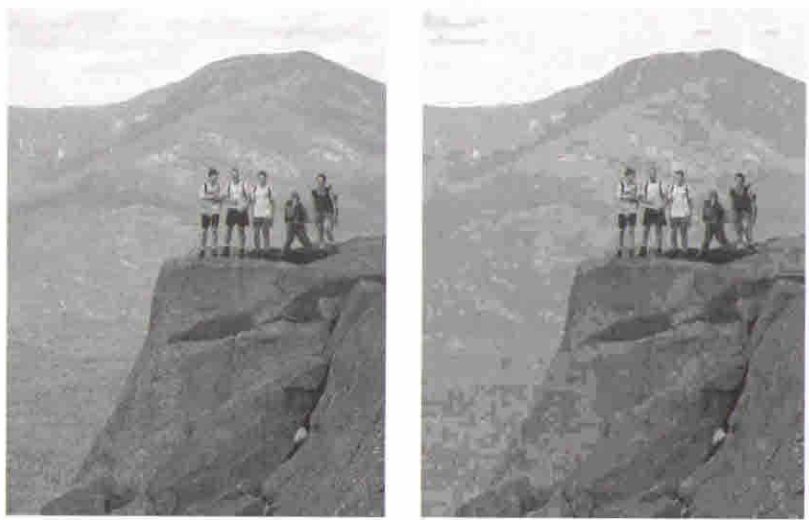


Figure 3. Images with JPEG Quality 95 and 5.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hacker's Corner: Compressing with GZIP	Adrian Smith	84
Technical Correspondence		89
At Play with J: Boggle	Gene McDonnell	91
J is for JPEG: Windows Bitmap to JPEG Baseline Compression	Richard Dazeley	103
Inversion of Boolean Matrices	Michel Dumontier	125
Matrix inversion modulo p Linear systems modulo p	Michel Dumontier	134

Hacker's Corner: Compressing APL arrays with GZIP

Experiments by Stefano, notes by Adrian Smith

Background

OK, we all have 256Mb of memory, so workspace is infinite and disks are huge. Why would you want to bother taking a perfectly good APL vector and reducing it in size by a factor of 10 or so? Just so you can make it bigger again. Well, the IT world is moving in some very mysterious ways at the moment and just as we are learning to use the infinite power of the desktop, along come SOAP and .NET and suddenly we are expected to create 'interoperable' applications with a really slow network separating the components.

What is more, we are supposed to exchange data using a wonderfully redundant scheme called XML (Jonathan Barman's article in Vector 17.4 is an excellent introduction) which can describe pretty well anything in plain ASCII text but takes huge numbers of extra bytes to do it.

OK, various components along the way attempt to compress things, but if you want to really make this stuff fly, you probably should consider compressing the data yourself as you generate it, and saving things like timeseries and budgets (which tend to be full of zeros) in their compressed state.

Disk Speed vs CPU Power

The major performance bottleneck on a modern machine (particularly a portable) is usually the disk access time. I was quite surprised to notice on my old P90 Dell laptop that when we compressed the disk to make some space, everything started running faster! Even with a P90 the time taken to uncurl a compressed 'dyalog.exe' was so small that it was easily beaten by the saving in time to get the (much smaller) file off the disk. If you are storing big timeseries on component files, then the chances are that the Redstone shares didn't move much last week and your data will compress really well. Of course you may not want to compress the entire file (like me you probably keep the first few components as documentation) so a couple of easy to use functions that compress and decompress vectors might come in handy.

Maintaining Portability

Sometimes you only need to do the compression – if you are publishing a report in a portable format like Adobe's PDF you can leave it up to the 'reader' application to decompress the data – but only if you have used a compression scheme that it knows about. The same applies if you are trying to share data with (say) a PERL application on a Unix-based web-server. For some time now I have been avoiding APL component files in favour of a simple 'reverse polish' array representation which is entirely character-based and can be written to a text file. This was specifically designed to be easy to parse by non-APL applications, but of course it is a little wordy and could also be selectively compressed very nicely. As long as I use a compression technology which is readily available to PERL and Delphi programmers, then I am much better insulated against future change.

What is this thing called 'Flate'?

I think all you can say here is that lots of very clever people have been working on recognising redundancy in data and transforming large redundant representations into minimal non-redundant equivalents. The algorithms used in PKZIP are very typical, and the .gz format used by Unix systems is another example. These schemes (and the patented LZW algorithm used in GIF pictures) all give very similar results, and the good thing about the public domain Deflate/Inflate algorithm is that it fractionally outperforms all of them, being an intelligent synthesis of all the best ideas. Basically it looks for repeating patterns in the inbound data (so a vector of random integers will compress rather badly), makes a table of patterns and replaces the pattern by a pointer into the table. Typically the compression algorithm will be quite slow and the decompression lightening fast, which is fine for your average PDF (written once) or dyalog.exe (written once a week) or a corporate budget (written once a year) but is not so sensible for a market tracker that is noting prices every few minutes.

So how do I make it work?

Basically you need a single DLL (freely redistributable) and a bunch of functions to call it. A copy of the C++ header file is rather necessary to work out how the calls should be built, and what all the defined constants are. You can find the DLL and the Dyalog workspace which covers it in the resources section of the Vector web site. All the code is by Stefano – I have added a few comments as notes to myself as I was reading through it.

Firstly – copy zlib.dll somewhere Dyalog can find it (mine is in /winnt/system32) and load the supplied workspace. This has a couple of examples and an Abstract

variable in root, and a ZLIB namespace to do the serious work. The kind of thing it can do is nicely illustrated by:

```

      qq+32768 ZLIB.DeflateIV 2345p15
      pqq
31      qq+32768 ZLIB.InflateIV qq
      pqqq
2345      qq=2345p15
1

```

Obviously this is a very artificial example, but it is surprising how often planning and scheduling systems throw up huge arrays of mostly zeros

```

      pmat+234 106p0  A 2 years planning for 234 products
      pmat[18;5 6 7 67 68 69 70]+12
      pmat[28;98 99]+14
      qq+32768 ZLIB.DeflateIV ,pmat
      pqq
64      qq+32768 ZLIB.InflateIV qq
      pqqq
24804      pmat=234 106pqqq
1

```

Well, it may be a 1-trick pony but it plays this trick pretty well. Let's see if the buffer size has any noticeable effect on a big array:

```

      tt+ai[2] o qq+1024 ZLIB.DeflateIV ,pmat o ai[2]-tt
4166
      tt+ai[2] o qq+4096 ZLIB.DeflateIV ,pmat o ai[2]-tt
1161
      tt+ai[2] o qq+32768 ZLIB.DeflateIV ,pmat o ai[2]-tt
331
      tt+ai[2] o qq+(2*20) ZLIB.DeflateIV ,pmat o ai[2]-tt
201
      p,pmat
641680
      2*20
1048576

```

Basically, we are timing the speed of the APL interpreter, and the catenation to the intermediate result vector. I would be inclined to round up the buffer size to the nearest 1k block above the data size and take it all in one lump always!

Just for interest, how do these times compare with the reconstruction phase:

```

2965      tt+ai[2] ◊ qgg+1024 ZLIB.InflateIV qq ◊ ai[2]-tt
932      tt+ai[2] ◊ qgg+4096 ZLIB.InflateIV qq ◊ ai[2]-tt
291      tt+ai[2] ◊ qgg+32768 ZLIB.InflateIV qq ◊ ai[2]-tt

```

So you can begin to judge the trade-off. Round about 300ms to reduce rather over a megabyte of integers to 64 bytes, but of course we need to keep note of the shape of the original array somewhere. Something which would be very nice here would be for Dyalog to implement `⊞OR 'qq'` such that it returned the internal representation of any array as a vector of bytes. Then we could compress anything and (with something equivalent to reconstruct the array from its OR form) put it back again. The only way I know to do this is to connect back-to-back TCP sockets, set one to 'APL' and the other to 'RAW' and throw the array around the loop! I seem to remember that something very like this made it into +Win 3.6, so maybe someone can follow this up and convert the internal calls appropriately.

Let's Have a Look at the Code

```

v Init;z:strncpy;DLL;ZSTREAM
[1]  a Set up basic DLL calls and find out the version info
[2]  DLL+'zlib.dll.C32'
[3]  ZSTREAM+'(I I I I I I I I I I I I I I I I)'
[4]  a We will need the version to pass to the other calls:
[5]  ⑈NA'I4 ',DLL,'|zlibVersion'
[6]  'strncpy'⑈NA'dyalog32|STRNCPY >0T I4 U4'
[7]  VERSION+strncpy 256 zlibVersion 256  a VERSION+'1.1.3'
[8]
[9]  ⑈NA'I ',DLL,'|deflateInit_ =',ZSTREAM,' I <0T I'
[10] ⑈NA'I ',DLL,'|deflate =',ZSTREAM,' I'
[11] ⑈NA'I ',DLL,'|deflateEnd =',ZSTREAM
[12]
[13] ⑈NA'I ',DLL,'|inflateInit_ =',ZSTREAM,' <0T I'
[14] ⑈NA'I ',DLL,'|inflate =',ZSTREAM,' I'
[15] ⑈NA'I ',DLL,'|inflateEnd =',ZSTREAM
[16]
[17] a Standard utilities to reserve/free memory
[18] ⑈NA'I4 kernel32|GlobalAlloc U4 U4'
[19] ⑈NA'I4 kernel32|GlobalLock I4'
[20] ⑈NA'U4 kernel32|GlobalFree U4'
[21]
[22] a Use the Dyalog32 DLL to move stuff to/from reserved memory
[23] a MEMCPY(destination-address, source-address, number-of-bytes).
[24] 'put'⑈NA'dyalog32|MEMCPY I4 <I1[] U4'
[25] 'get'⑈NA'dyalog32|MEMCPY >I1[] I4 U4'
v

```

The outer layer ...

```

v r←bufs DeflateIV data;handle;availIn;stream;out;status
[1]  ⌘ Deflate an integer vector in units of <bufs>
[2]  ⌘ Moves data into memory buffer and compresses it
[3]
[4]  handle←DeflateInit Z_DEFAULT_COMPRESSION bufs bufs
[5]  availIn←0
[6]  r←0
[7]
[8]  :While 1
[9]    :If 0=availIn
[10]      stream←(bufs\pdata)+data
[11]      :If 0=ostream
[12]        :Leave
[13]      :EndIf
[14]      data←(ostream)+data
[15]    :EndIf
[16]    (status availIn out)←handle Deflate((0=availIn)/stream)Z_NO_FLUSH
[17]    'Invalid stream'␣SIGNAL(~status<Z_STREAM_END Z_OK)␣11
[18]    r,←out
[19]  :EndWhile
[20]
[21]  :While Z_STREAM_END≠status
[22]    (status availIn out)←handle Deflate 0 Z_FINISH
[23]    r,←out
[24]  :EndWhile
[25]
[26]  status←DeflateEnd handle
v

```

Basically, this sets up a 'session' by creating a temporary namespace and initialising the DLL with an appropriate 'zstream' which is a deceptively simply array of values and flags. It then uses the session progressively to perform the deflation and cleans up at the end.

Have a look at the workspace on the website for the low-level calls – these do some nice stuff with defined operators to keep the interface clean. Improvements and enhancements welcome – I will probably be using this for image data in PDF files (these are excellent subjects for compression, as are NewLeaf tables) so look out for more on the same theme in the future.

TECHNICAL CORRESPONDENCE

From: Phil Last <phil.last@ntlworld.com>

May 2001

```
f00+{} ◊ f01+{} ◊ f02+{}
```

might be naff but I've used it on occasion to localise names when I actually found the need (or at least the usefulness) to have locally defined NILADIC functions in a dynamic one.

On Stefano's point and once again (I may even start getting paranoid here) using DEFINED operations, I've always kept my (Ep) operator handy.

```

v r←(f Ep)r
[1]  a En Passant
[2]  a Useful for running non explicit fns
      under dynamic or defined operators.
[3]  f r
v
```

It's redundant in these cases now as `□FREPLACE` and `□WC` both return results (as all functions always should have) but my first uses were

```
z←data◊□FREPLACE Ep If(×pdata)+tie,comp
```

and

```
z←name◊□WC Ep If(9×□NC name)+settings
```

Just thought I'd let you know.

Localising the effects of system functions in D.

From: John Scholes

April 2001

People have noticed that from within a Dyalog 'D' function, it is difficult to localise the effect of object-creating system functions such as `⎕FX`, `⎕WC` and `⎕CY`. Localisation in D takes place at assign time, whereas these system functions create objects as a side effect.

A (slightly naff) way around this is to have D localise a name prior to calling the system function. Notice that the dummy assignment must accommodate the system functions' refusal to overwrite an object of differing name class.

```
foo←{}                                a local class 3 name
_←⎕FX'z←foo ...'                      a fix local function
```

or:

```
f1←'' ⋄ _←⎕EX'f1'                     a local class 0 name
ref←⚡'f1'⎕WC'form'                   a make local form
```

A similar problem arises with system functions that don't return a result. A (rather awkward) code-around for this is to have 'execute' call the function and then return an ignored result:

```
_←⚡'wsid ⎕CY obid ⋄ 0'
```

This awkwardness seems (to me) to result from the clash between the procedural and declarative styles of programming. With the benefit of hindsight, we might prefer that `⎕FX` and `⎕WC` don't create global workspace names but return function and object *references* suitable for assignment.

At Play With J: Boggle

by Eugene McDonnell

Boggle

I suppose many of you have played as a child with a set of blocks, wooden cubes about an inch and a half on a side, with pictures, letters, numbers, and designs on the faces. Did you ever set the alphabet faces in a row to spell a word? Suppose you had such a set of blocks in which all of the faces had letters on them, and that you had a tray divided by partitions to form rows and columns, giving cells into which the blocks just fitted. Now if you jumble up your blocks in a bag, then take out a block at a time and with your eyes closed, put it securely in one of the cells at random until they are all full, you will find when you then open your eyes that the letters that are face up will be in all different orientations.

Now if you are given the task of finding among these as many words of four or more letters formed among blocks that are connected either by an edge or a corner within three minutes, you will have some idea of how to play a word game I very much like. The game comes in two forms; the original game, the one I cut my teeth on, is called *Boggle™*, and is played on a 4-by-4 tray. The other game, which I now favour, and which appeared several years after Boggle was introduced, is called *Big Boggle™*, and is played on a 5-by-5 tray. The blocks are miniature, about five-eighths of an inch on a side and made of plastic, as is the tray. There is also a clear plastic dome which fits snugly over the tray, permitting the whole to be turned upside down, shaken, and turned upright, so that with a little jiggling each block nestles upright in one of the cells. The game comes with a three-minute egg-timer and a little sheet of instructions.

The letters on each of the sixteen Boggle cubes and the 25 Big Boggle cubes are given by the columns of the tables below. They are shown as lower-case here, but in the game they appear as capitals. The letter shown as 'q' is actually the digraph 'qu', and if used in a word counts as two letters:

aaaaacdddeeeeee
abcfoieeieehilil
ebhfomilsgirormn
ejoktolrthntstnn
goppttrvtnsvtqr
nossxwxywuwtyuz

aaaaaaaaabccccdddeeffgino
fdeaeaeajceeehhdhnmiiopoo
ieeageefkeiilllhhsoiprroo
rnefmegiqlipnnnlstirrtrt
snernemrxsplsoooosttsvrut
ynmsneuszttttrtrtrutywywu

Here are the rules supplied with the game:

Object: To list, within 3 minutes, as many words of the highest point value as you can find among the random assortment of letters in the cube grid.

Preparation: Each player should have a pencil and a piece of paper. Drop the letter cubes into the dome and place the grid, open side down, over the dome. Turn the domed grid right-side up, vigorously shake the cubes around, and maneuver the grid until each cube falls into place. Then, as one player removes the dome, another player starts the timer.

Playing: When the timer starts, each player searches the assortment of letters for words of *four letters or more*. When you find a word, write it down.

Words are formed from *adjoining letters*. Letters must join in the *proper sequence* to spell a word. They may join horizontally, vertically or diagonally to the left, right, or up-and-down. No letter cube, however, may be used more than once within a single word.

Type of words allowed: The only words that are allowed are those that can be found in a standard English dictionary. You may look for any type of word – noun, verb, adjective, adverb, etc. Plural nouns are acceptable as are all verb tenses. Words within words are also allowed, e.g., *master*: mast, aster.

Type of words not allowed: Proper nouns, abbreviations, contractions, hyphenated words, and foreign words that are not in an English dictionary.

Scoring and winning: When the timer runs out, everyone must stop writing. Each player in turn then reads aloud his or her list of words. Any word that appears on more than one player's list must be crossed off *all* lists, including that of the reader. The same word found by a player in different areas of the grid may not be counted for multiple credit.

After all players have read their lists, each player scores his or her remaining words, differing point values accorded to the words according to their lengths, as follows:

no. of letters	4	5	6	7	8+
Points	1	2	3	5	11

The winner is a) the player whose words have earned the most points, or b) the first to reach 50 points, 100 points or whatever score is considered by all to be a reasonable target.

I usually play until at least one player has reached or passed 100 points. I've played the game with three or four players, but prefer the two-person game. My wife and I have established several additional house rules. Since I frequently wrote down spurious words, my wife insisted that there be a penalty for such. Thus, any word may be challenged. If it is not found in the dictionary the player is given a score of -1 for it. If it *is* in the dictionary, the player gets an additional point for it. We began by using our huge unabridged *Merriam-Webster* dictionary, but this, my wife claimed, gave me an unfair advantage, since I frequently wrote down archaic Scottish words and the like that were in this dictionary but not in smaller ones. We then began using the *Concise Oxford Dictionary* (COD), but had to give up on that, too, since it favoured English words and English spellings. My wife was tired of me putting down words like *twae* and *nous* that are unknown on our side of the Atlantic. We now use the *American Heritage Dictionary* (AHD), Ken Iverson's favourite. Here is a sample grid:

```
t i n e
n i n t
o c n a
r e t l
```

These letters are shown in normal position; in practice they can have any of the four possible orientations. Try your hand at finding words in this grid. Remember, you have just three minutes. You can see the words I found, unaided by computer, at the end of this paper.

The Problem

Twenty years ago, when I was Recreational APL editor of *APL Quote Quad*, I received a letter from Robert Ashworth, of Carbondale, Illinois, asking if I could write a column on the Boggle game. I was agreeable to the extent of posing these problems to my readers (this was before Big Boggle was born, so assumes the smaller Boggle situation):

- a. Find the number of paths of length 4 in the grid.
- b. Find the number of paths of length 5.
- c. Write a suite of functions that, given a 4-by-4 character table, finds all words of length 4 and 5, using the rules of Boggle. Assume the existence of two tables *w4* and *w5* containing all the acceptable words of length 4 and 5, respectively.

I thought this was the most difficult problem I'd ever proposed. Furthermore, at the time I had only vague ideas of how to go about solving it. Recently, nagged by this unfinished business, I revisited the problem, with a degree of success.

A Boggle path is a sequence of distinct connected cells, connected in the Boggle sense. An interior cell is connected to the eight surrounding cells. An edge cell, not on a corner, is connected to the five surrounding cells. A corner cell is connected to the three surrounding cells. It may be suitable at times of to use the cell's list indices:

```

]m=:i.4 4
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15

```

and at other times their row-column indices:

```

]q=:<"1 (4 4#.: m)
+---+---+---+---+
|0 0|0 1|0 2|0 3|
+---+---+---+---+
|1 0|1 1|1 2|1 3|
+---+---+---+---+
|2 0|2 1|2 2|2 3|
+---+---+---+---+
|3 0|3 1|3 2|3 3|
+---+---+---+---+

```

Normalizing any 3-by-3 portion of the row-column index grid by subtracting the central item from each of the items, shows that two cells are connected if the maximum magnitude of the difference of their row-column indices is 1.

```

[] -&.> (<1 1)"_ { ] )3 _3{.q
+---+---+---+---+
|_1 _1|_1 0|_1 1|
+---+---+---+---+
|0 _1|0 0|0 1|
+---+---+---+---+
|1 _1|1 0|1 1|
+---+---+---+---+

```

The only way I know to find the number of paths for different cases is by constructing the paths. So to solve problems a and b above implies finding the paths themselves.

An easy but expensive way is to find a superset of the paths, by taking all the combinations of sixteen things taken four at a time, that is, $4!16$ or 1,820, then get each of the 24 permutations of every combination, giving $(!4)*(4!16)$ or 43,680 four-item lists, then select from the table all rows giving Boggle-connected paths. The verb `comb` is due to Roger Hui, and it and its component parts are given at the end

of this paper. The function `paths` has syntax `r =: n path k` and gives the paths of length `n` in a `k`-by-`k` grid.

```
paths =: dyad define
NB. find all paths of length x. in a grid of size y. * y. .
NB. px is a table of all the permutations of length x. .
    px =. (i.!x.)A.i.x.
NB. cxn is a table of all the combinations of y. things taken x. at a time.
    cxn =. x. comb *: y.
NB. pc is a table of all the permutations of each of the combinations.
    pc =. ,/px{"2 1 cxn
NB. (i{mpc) is 1 if (i{cxn) is Boggle-connected and 0 otherwise.
    mpc =. y. okf pc
NB. cxno is all the paths of length x. in a y.*y. grid.
    cxno =. /:- mpc#pc
)
```

This is the `okf` function:

```
okf =: 13 : '1=>./,|2-/\(2#x.)#:#:y.'"1
```

Given a list `y.`, this takes the row-column representation `((2#x.)#:#:y.)` of each item, the difference of pairs of successive representations `(2-/\)`, their magnitudes `(|)`, ravel these `(,)`, finds their maximum `(>./)`, compares this to 1 `(1=)`, yielding 1 if the list is Boggle-connected, and 0 otherwise.

For example, the number of paths of length 3 in a 4-by-4 grid is given by:

```
#3 paths 4
408
```

Here are four successive items from `pc`:

```
37 38 39 40{pc
2 0 4 1
2 1 0 4
2 1 4 0
2 4 0 1
```

And here is the result of applying `4 okf` to each of these rows:

```
4 okf 37 38 39 40{pc
0 1 1 0
```

If you look again at `m` you can verify that `2 0 4 1` and `2 4 0 1` are not Boggle-connected, but `2 1 0 4` and `2 1 4 0` are:

	m			
0	1	2	3	
4	5	6	7	
8	9	10	11	
12	13	14	15	

Don't bother to use this function. It takes an unbearably long time as the path lengths get just a little larger. It is intended only to let you know how my thinking was going.

An easier way to get the paths is to build them up starting with the n^2 paths of length 1, and extending these only with promising items. At this point I showed my astuteness by sending a message to Roger Hui explaining what I was doing, and asking him if anything bright in a combinatorial way occurred to him. I received, in rapid succession, four replies, written while he was babysitting his son Nicholas as he was in the middle of moving from Toronto to Vancouver. I'll pass over the first three messages because, as usual with Roger, one idea suggested a better idea. Here is his last effort:

```
rimb  =: _1: ,. (_1: , ] , _1:) ,. _1:
tileb =: 3 3 &(,;._3)
nborsb =: (4 1 4#1 0 1)&#"1 @ (,/) @ tileb @ rimb @ i. @ ,~

initb =: ,. @ i. @ *:
extendb =: 8&#@] ,. [: , {:"1@] { [
testb =: *./"1@0&<:) *. i.@{:@$ -:"1 i."1-
stepb =: (testb # ]>@extendb
pathb =: 4 : '(nborsb x.) stepb^:(y.-1) initb x.'
```

The pathb dyad has syntax $z =: n \text{ pathb } k$, and yields a table with k columns, with rows giving all the paths of length k to be found in an n -by- n grid. It begins by using `initb` to form a seed table having all possible starting cells of paths, namely, a single column with values $i. *: n$. Each use of its `step` dyad, beginning with the seed table, extends its argument, a table of paths of length $k-1$, to form the table of paths of length k . The `stepb` dyad requires as its left argument a table with eight columns and as many rows as there are cells in the grid. Each row contains a list of all the neighbours of each cell. In the case of edge cells, which have only five neighbours, and corner cells, with only three, the lists are filled out with `_1` values. The `nborsb` monad builds this table by forming an n -by- n grid with `(i.@,~)` and using the `rimb` monad to border this with `_1s`. This is tessellated into raveled 3-by-3 squares with the `tileb` monad. `,` and `(,/)` makes the individual tables into one large table. To complete the table the central column is deleted with `(4 1 4#1 0 1)&#"1`. Here is what this table for the 4-by-4 case looks like:

nborsb 4									
_1	_1	_1	_1	1	_1	4	5		
_1	_1	_1	0	2	_4	5	6		
_1	_1	_1	1	3	5	6	7		
_1	_1	_1	2	_1	6	7	_1		
_1	0	1	_1	5	_1	8	9		
0	1	2	_4	6	8	9	10		
1	2	3	5	7	9	10	11		
2	3	_1	6	_1	10	11	_1		
_1	4	5	_1	9	_1	12	13		
_4	5	6	8	10	12	13	14		
5	6	7	9	11	13	14	15		
6	7	_1	10	_1	14	15	_1		
_1	8	9	_1	13	_1	_1	_1		
_8	9	10	12	14	_1	_1	_1		
9	10	11	13	15	_1	_1	_1		
10	11	_1	14	_1	_1	_1	_1		

The table for the 5-by-5 case is similar, but with 25 rows instead of 16.

At each step, the last item of each of the current paths is used to select the appropriate row from the neighbours table, that row is replicated eight times, and each neighbour is appended to one of the eight replicated rows, using the `extendb dyad`. The `testb monad` removes illegal rows from this extended table, ensuring that no item is duplicated, and no `_1`s are present in the result. This process continues for `k-1` steps, at the end of which we have obtained the desired table.

The `pathb` function executes `4 pathb 4` in 0.02 seconds, and `4 pathb 5` in 0.09 seconds on my 233mhz computer. With its help I found the answers to problems a and b:

```
#4 pathb 4
1764
#4 pathb 5
6712
```

The time for my `path` function to do `4 path 4` is 43.4 seconds. I don't have the patience to try cases involving longer paths.

Now it's time to solve problem c. I have carried about with me for about 15 years word lists from the *American Heritage Dictionary*. The original data came from the publisher Houghton-Mifflin on a single file on a magnetic tape which Joey Tuttle had mounted on a tape drive in the Toronto machine room of I. P. Sharp Associates and read into the Amdahl V8 then in use there. He processed this file in several ways, the one of most use to me being sorted lists of words all of the same length, which I have named Words02 through Words26. These are now resident on

a Macintosh computer in my home. With the tools you've seen developed, you could probably arrive at a solution yourself. Assume that you have the *b*, the ravel of the letters in the grid, *p* the table of paths, and *w* the list of words, all for a given length. Then

$r =: /: \sim \sim. (w\ e. p\ \{ b) \# w$

will give *r*, a sorted table of all the words of a given length, with no duplicates, satisfying a legal path on the grid. Before showing the results I obtained using the phrase above, I give first the words I found unaided by computer in this grid:

t i n e
n i n t
o c n a
r e t l

ante	alter	cental
cent	inner	lancer
coin	inter	lancet
core	lance	lanner
lane	later	recoin
late	nance	rennet
nice	nicer	rental
nine	octal	tanner
rent	renal	tannin
tine		tinner
tint		

All of these are in *COD*. All but "nance" and "recoin" are in *AHD*. And here are those found by the computer:

<i>anne</i>	alter	alnico
ante	ancon	cental
cent	anent	cetane
cero	anion	encina
coin	conti	encore
core	creon	innate
etna	enate	lancer
icon	inane	lancet
lane	inner	lanner
late	inter	latent
neon	lance	nocent
<i>nero</i>	later	octane
nice	nicer	octant
nine	renal	rectal
<i>nore</i>	renan	rennet
once	renin	rennin
<i>reni</i>	rente	rental
rent	tater	tanner
<i>tate</i>	tinct	tannic
tent		tannin
tine		tenant
tint		tinner

The results of me versus the computer are: 4-letter words, 11 vs 16; 5-letter words, 9 vs 16; and 6-letter words, 10 vs 22. The second table shows a defect of my word collection. The tape we got from Houghton Mifflin includes biographical and geographical entries, and these have not been removed. I've put in italics those words, which, by Boggle rules, are not legal words. Also, *COD* has some words not in *AHD* and vice-versa. For example, "cero" (a western Atlantic fish) is in *AHD*, but not in *COD*, and "recoin" is in *COD* but not *AHD*.

There are longer words in the list. How many of you found "continent"? And "continental"? And the 16-letter behemoth "intercontinental"? Of course, I contrived this case, choosing a suitable looking word from my word16 file, in order to fill a 4-by-4 grid completely.

Here is an incomplete table of the number of paths of given lengths in grids from size two to five:

paths =: 0 : 0

The number of paths of length k connecting distinct adjacent points in a square grid of $n * n$ points:

k\ n	2	3	4	5
1	4	9	16	25
2	12	40	84	144
3	24	160	408	768
4	24	496	1764	3768
5		1208	6712	17280
6		2240	22672	74072
7		2984	68272	296360
8		2384	183472	
9		784	436984	
)				

I've been unable to find the law of this table. Row 1 are the squares, of course. Row 2 is four times the alternate triangular numbers (3 10 21 36). Beyond that deponent witnesseth not. The columns headed 2 and 3 are complete, but even with Hui's much more efficient functions, larger cases for columns 4 and 5 are still unattainable with my computer (and my patience).

There are two ways the task can be made more efficient: first, instead of using `pathb`, which necessitates going back to the beginning for each case computed for a given grid size, the previous results can be stored, allowing case k to be computed by using the `stepb` function on the $k-1$ result; and second, by taking advantage of the symmetries of the square grid. The number of paths of a given length proceeding from any corner point are the same; similarly for symmetrically located edge points and interior points. The symmetries are evident if the number of paths beginning from each grid point are displayed. For example, here are the number of paths from each point for a 4-by-4 case and a 5-by-5 case:

```

4 4$#/.~{.:4 pathb 5
322 435 435 322
435 486 486 435
435 486 486 435
322 435 435 322
5 5$#/.~{.:5 pathb 6
1874 2752 2998 2752 1874
2752 3524 3672 3524 2752
2998 3672 3784 3672 2998
2752 3524 3672 3524 2752
1874 2752 2998 2752 1874

```

In general, for a grid of side n , containing n^2 points, there are only $g(n)$ distinct numbers of paths, where g is

$$g = :2\&!@>:@>.@-:$$

Here are the results of g applied to grid sizes one through 8:

```
(, :g) 1+i.8
1 2 3 4 5 6 7 8
1 1 3 3 6 6 10 10
```

So that, for the 4-by-4 case, only three values have to be computed, not 16, and for the 5-by-5 case, only six, not 25. The total number of cases for a given path length k in a grid of size n can be obtained by an inner product:

```
4 4$#/.~{.}:4 pathb 4
75 109 109 75
109 148 148 109
109 148 148 109
75 109 109 75
4 8 4+/. * 75 109 148
1764
#4 pathb 4
1764
```

When the actual paths are needed, the additional cases can be obtained from the abbreviated tables by using the appropriate indices to select from them, the indices being the permutations obtained from the ravel of the rotations and reversals of the table $i.(n.n)$.

Here is how this is done:

```

NB. get neighbours table
n4:=nborsb 4
NB. form monad to step previous case by bonding stepb's left argument
f:=n4&stepb
NB. initial case for abbreviated situation
] ip=:.,0 1 5
0
1
5
NB. Get all paths for abbreviated argument
$p42=:f ip
16 2
NB. Only sixteen cases of the 84 obtained using full argument
$4 pathb 2
84 2
NB. Use phrase 7.b.d8 to obtain desired permuted lists of all cell numbers
] allpcn=:,"2 (i.8)d8"0 2 i.4 4
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
12 8 4 0 13 9 5 1 14 10 6 2 15 11 7 3
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
3 7 11 15 2 6 10 14 1 5 9 13 0 4 8 12
12 13 14 15 8 9 10 11 4 5 6 7 0 1 2 3
15 11 7 3 14 10 6 2 13 9 5 1 12 8 4 0
3 2 1 0 7 6 5 4 11 10 9 8 15 14 13 12
0 4 8 12 1 5 9 13 2 6 10 14 3 7 11 15
NB. Get all length-2 paths (with possible duplicates)
$q42=:./p42{"1/allpcn
128 2
NB. Get ordered set of all length-2 paths
$s42 =: /:~ ~. ,/ q42
84 2
NB. compare ordered set to long-winded but accurate set
(4 pathb 2) -: s42
1

```

Appendix

Hui's combinations suite:

```

startc =. i.@-.-@-
countc =. <:@[ ! <:@[ + |.@startc
indexc =. ;@:((i.-))&.>)
recurc =. (countc#startc) ,. (indexc@countc{comb&.<:)
testc =. *@[ *. <
basisc =. i.@(<: , [])
comb =. basisc`recurc @. testc

```

J is for JPEG: Windows Bitmap to JPEG Baseline Compression

by Richard Dazeley (rdazeley@postoffice.utas.edu.au)

Abstract

JPEG baseline is one of the world's most used compression algorithms. This is because it is fast, offers good compression with little noticeable image degradation, and allows the user to specify the level of image quality. The algorithm uses a number compression techniques such as difference, run-length and Huffman coding. It also does a number of image manipulation functions such as colour space conversion and changing from the spatial to the frequency domain. This paper explains the algorithm in detail as well as giving code for a J implementation. The implementation given compresses 24-bit Windows bitmap images. Also provided is a brief explanation and code implementation of JFIF (JPEG File Interchange Format). The result is an implementation that runs in a similar time as implementations in compiler-based languages.

Introduction

In a world with limited data storage and bandwidth our ability to store and transfer information cheaply has become a major field of research and development. The ability to compress data significantly helps to reduce this expense making information more readily available to everyone. Digital images are one of the most costly information sources and have been the focus of a number of techniques. There are two major advantages to image compression over other sources such as text and speech. The first is that their "multi-dimensional characteristic implies that image data has more redundancies than one dimensional data such as speech" (Kou 1995, p6). Secondly an image is capable of losing a great deal of its information before the human viewer is capable of observing any degradation. However this is not possible at all in sources like text.

With the advent of the World Wide Web the number of transfers of photographic images has exploded and one of the most popular compression techniques for this type of image is JPEG. The JPEG (Joint Photographic Experts Group) standard specification (ISO/IEC IS 10918, or ITU-T T.81) defines four operation modes: sequential DCT based mode, sequential lossless mode, progressive DCT based mode, and hierarchical mode. The first mode is the simplest and is referred to as JPEG baseline and is what most people mean when they refer to JPEG.

This simpler but by no means trivial algorithm will be the focus of this paper. The algorithm will be applied to a standard 24-bit Windows bitmap file using the functional language J. While a compiled language such as C or Java may be better suited in some ways to an algorithm like this, an array based language such as J offers an interesting alternative as it provides a number of tools well suited to the matrix manipulation required.

Setting up the Image Data

There are four initial tasks that need to be performed prior to compression. The first is obviously to load the bitmap but then excess bytes need to be removed, the colour space converted and a suitable data structure built that allows the algorithm to operate smoothly. This process is controlled by the `setUp` function.

```
SetUp =: blocks@fillImage@shape@components@colourSpace@
        removeZeros
```

Load Bitmap

The process of loading the bitmap involves reading in the data, converting it, extracting some important information from the header and setting up our initial data structure. The following function and its subsidiaries performs these tasks.

```
loadBitmap =: boxElements@interpret@read@usage
```

There are two pieces of information that are important for this algorithm. The first is the path and file name of the bitmap to be compressed and the second is the quality level to perform the compression. The function `usage` allows the user the option of giving a value or using the default quality setting. The output of the function is two boxes: the first is the quality setting followed by the path and file name.

```
usage      =: ({:,{.)`'(quality&.) @. (#>2:)
quality    =: 80
```

The `loadBitmap` function then reads in the file data and converts it from characters to integers with the following functions:

```
read       =: >@{.;1!:1@{:
interpret  =: {.,charToInt&.>@{:
charToInt  =: a.&i.
```

The last stage is to extract the width, height and start of image information that is contained in the bitmap header. All of these pieces of information are 4 bytes in

size and start at the 18th, 22nd and 10th elements respectively. This is done with the following code.

```
boxElements =: (., (imageWidth; imageHeight; image) @>@{ :
image       =: fileStart). |
imageHeight =: (22+i.4)&group
imageWidth  =: (18+i.4)&group
fileStart   =: (10+i.4)&group
group       =: 256&#. @|. @: {
```

This information is collated into an array of four boxes in the format below.

Quality	Image Width	Image Height	Image Data
---------	-------------	--------------	------------

Removing Zeros

The Windows bitmap file can contain extra zeros at the end of each row of pixel data. This is because Windows allocates memory in DWORD (4 bytes) sized blocks. The number of extra zeros at the end of each row is the amount needed to fill out the row of bytes to a multiple of four. For instance, an image of one pixel width has three bytes to represent blue, green, and red (Windows bitmaps are the reverse of the more common RGB format) plus one zero to fill out the four bytes required. Therefore, a width of two pixels would have two zeros, three pixels would have three zeros and four pixels would have no zeros. In the following set of functions the `stencil` function builds an array of 1s based on the width of the image and appends to the tail a number of zeros. The `removeZeros` function duplicates the stencil for each row and performs a copy over the array of pixels. This copies only the elements we need and removes the excess zeros.

```
removeZeros =: }., <@ (pixels #~ ($@pixels $ stencil))
stencil     =: >: @ (0&*) @ (i. @ (3&*@width)), (0&*) @ i. @ numZeros
numZeros    =: (4&|) @ (4&-) @ (4&|) @ (3&*@width)
width       =: >@ (1&{)
pixels      =: >@ (3&{)
```

Colour Space Conversion

Windows bitmaps store each pixel's data using three colour channels: blue, green and red (BGR). JPEG, however, requires that an image uses the YUV or YCbCr colour space. One reason for this is because the human eye is much more sensitive to variances in luminance than those in chrominance. Therefore, significantly more

compression can be achieved on the horizontal and vertical chrominance channels without any visible degradation in the image (Taylor, 1999)¹.

Colour space conversion can be done by calculating the following equations:

$$\begin{aligned} Y &= 0.299 R + 0.587 G + 0.114 B \\ Cb &= -0.1687 R - 0.3313 G + 0.5 B + 128 \\ Cr &= 0.5 R - 0.4187 G - 0.813 B + 128 \end{aligned}$$

In J this was easily done by multiplying the reverse of BGR by the matrix YCbCr at rank 1. The rows of the resulting 3x3 matrix are then added together along with the array inc. The reshape function takes a number on the left and makes a matrix with that number of columns out of the array to the right.

```
YCbCr      =: 3 3 $ 0.299, 0.587, 0.114,
                0.1687, 0.3313, 0.5,
                0.5, 0.4187, 0.0813
inc         =: 0 128 128
colourSpace =: },<@,@(convert"1@|. "1@{3&reshape)@pixels)
convert     =: (inc&+)&:(+/"1)@{YCbCr&{* "1~))
reshape     =: (>.@($@)%[],)@$
```

Components and Blocks

The JPEG algorithm requires that an image is compressed within its component parts. In this script the source file is always regarded as having three components Y, Cb and Cr (A grey scale image would normally only have the luminance component). Therefore, our next step is to extract each component's data into separate arrays. The following code does this calling three functions which in turn call the selector function passing it an additional three-element array. This array gives which byte we are interested in for that component. This array is duplicated to the length of the number of pixels in the image. J's copy function is then used to select only the elements required for that component.

```
components =: },<@{Y;Cb;Cr)
selector    =: pixels@] (#~) ({($~) (3&*@numPixels@]))
Y           =: (1 0 0)&selector
Cb          =: (0 1 0)&selector
Cr          =: (0 0 1)&selector
numPixels   =: width * height
```

¹[Editor's Note] YUV, Y Cb Cr represent the same colour space where instead of decomposing a colour into its Red, Green and Blue components, Luminance (Y), Blue Chrominance (U or Cb) and Red Chrominance (V or Cr) are used?

The end result of the `components` function is three boxes, each containing an array representing a component extracted from each of the pixels. These three boxes are all contained within the image data box of our main data structure.

Within these components the JPEG compression operates on individual 8x8 blocks of data. Therefore, we need to cut each component into blocks of this size. To do this we first need to convert each component array of data into a matrix with the same width and height as the original image.

```
shape      =: },<@(dim (<@|.@$"1 >) pixels)
dim        =: height , width
```

You may notice here that after the array has been shaped into the same size as the original image it then flips the order of the rows. This is done because Windows bitmaps start at the bottom and scan upwards (Lamothe, 1998). Secondly, before we cut out our blocks we need to consider the case where the image's width or height is not divisible by 8, which would result in some non-standard size blocks. This is corrected by filling out the width and height of each component with zeros. The `fillImage` function does this by building a template matrix that is the desired minimum size. The desired minimum size is the same size or bigger than the source image and it is also divisible by 8 in both dimensions. This template is then laminated onto each component. J then fills out the component matrix with zeros to the same size as the template. The template matrix is then removed leaving the component with the correct dimensions.

```
fillImage  =: },<@(template fillComp (>@pixels))
fillComp   =: (;/ )@{:@{([, :])
template   =: i.@(newHeight,newWidth)
newWidth   =: blockWidth&ceiling@width
newHeight  =: blockHeight&ceiling@height
ceiling    =: ((| -) |) +]
```

We are now ready to cut the blocks which is done by repeatedly cutting the component into lots of 8 rows. Then by transposing the resulting matrix, repeating the cutting process and transposing it back, we have effectively been cutting in lots of 8 columns giving us our 8x8 blocks.

```
blocks     =: },<@(<@(<@trans@cut@trans@cut@>)@pixels)
cut        =: _8&(<\)
trans      =: |:@>
```

JPEG Baseline Compression

Now that the image data is structured appropriately we can perform the JPEG compression algorithm. The baseline algorithm is DCT (Discrete Cosine Transform) based and the basic system is shown in Figure 1. The encoder diagram contains three stages: an 8x8 FDCT (Forward DCT), a quantiser and a Huffman encoder. The FDCT converts the 64 samples that are in the spatial domain to a set of 64 values that are in the frequency domain. The first frequency is called the DC coefficient (zero frequency coefficient) and the rest are the AC coefficients (non-zero frequency coefficients). These values are re-ordered and quantised according to the quality specified by the user and then Huffman coded. Both the quantisation and Huffman coding is accomplished through the use of look-up tables. The Decoder is the inverse of the encoding process using a Huffman decoder, a dequantiser, and an 8x8 IDCT (Inverse DCT). The decoding process is not implemented here.

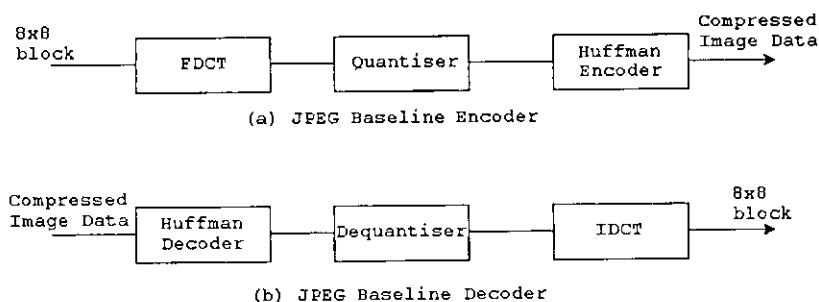


Figure 1: JPEG Baseline System Diagram (Kou, 1995)

Discrete Cosine Transform

The Discrete Cosine Transform converts the set of 64 samples from a block into a set of frequency coefficients. This is done through the application of the following equation. However, few JPEG encoders are actually implemented using this definition as it is very computational intensive and any accuracy gained is lost during quantisation. Instead they use one of a number of fast DCT algorithms. This application is using the AAN² method as implemented in the IJG JPEG-6a library.

²[Editor's Note] ANN: Artificial Neural Network

The FDCT given a block of samples: S_{yx} ($y, x = 0, 1, \dots, 7$) is defined by:

$$S_{vu} = \frac{1}{4} C_u C_v \sum_{x=0}^7 \sum_{y=0}^7 S_{yx} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

Also, included below is the IDCT, should the reader feel inclined to implement a JPEG decoder.

$$S_{yx} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 C_u C_v S_{vu} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

where

$$C_m = \begin{cases} 1/\sqrt{2} & \text{for } m = 0 \\ 1 & \text{otherwise} \end{cases}$$

To perform the FDCT using the AAN method we first need to subtract 128 from all the samples in each block.

```
FDCT      =: }, <@ (FDCTblock&.>@pixels)
FDCTblock =: (DCT@sub)&.>
sub       =: 128&(-/~)
```

The next step is to apply the transform's set of calculations by column and then by rows. In the graph below is a set of values needed to generate the fast transform.

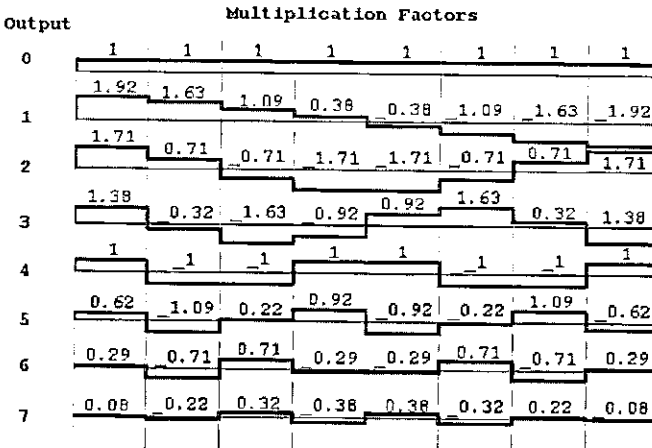


Figure 2: Fast DCT Multiplication Factors.

Basically 8 samples are entered (1 row or column) and 8 coefficients are generated. Each output element is a result of multiplying each of the input elements by its corresponding graph values and adding the results. The J implementation, though, has altered this slightly to minimise the amount of multiplications but still provides the same results.

```

DCT      =: (DCTcalc*1)&|. |: (DCTcalc*1)
DCTcalc  =: output @ step4 @ step3 @ step2 @ step1
step1    =: 4&{. (+,-&|. ) |. @(_4&{.)
step2    =: [, step2a ,step2b
step2a   =: (2&{. (+,-&|. ) |. @(_2&{. ))@ (4&{.)
step2b   =: +/@>@ (2&{<|.) @(_4&{.)
step3    =: [, t1, t2, t3
          t1 =: (0.707106781)&*+/@({10 11}&{)
          t2 =: (0.382683433)&*+/@({12 14}&{)
          t3 =: (0.707106781)&*@ (13&{)
step4    =: [,t4,t5, +/@p1, -/@p1
          t4 =: ({0.541196100)&*@ (12&{)) + {16&{)
          t5 =: ({1.306562965)&*@ (14&{)) + {16&{)
output   =: +/@p2, +/@p3, +/@p4, -/@p5,
          -/@p2, +/@p5, -/@p4, -/@p3
p1       =: 7 17 &{
p2       =: 8 9 &{
p3       =: 20 19 &{
p4       =: 11 15 &{
p5       =: 21 18 &{

```

Quantisation

The quantisation phase of the algorithm is now performed on the result of the FDCT and is the lossy part of the JPEG algorithm. This stage involves first building a Quantisation Matrix or QM. There are two QMs (luminance and chrominance) in this implementation, comprising 64 quantiser step-size parameters each based on the specified quality.

```

divisorsLuminance =: divisors@quantumLuminance
divisorsChrominance=: divisors@quantumChrominance
divisors          =: %@(((64$AANscaleFactor)*{8#AANscaleFactor)
                  *8)&*)
quantumLuminance  =: quantumLConst&quantum
quantumChrominance=: quantumCConst&quantum
qualityFactor    =: (200&-)@ (2&*)` (5000&%)@. (50&{<~))
quantum          =: (mnQm, mxQm)&interval@<.@ (100&{<~))@ (50&+
                  @({*qualityFactor@((min,max)&interval)&{)
interval         =: ({. @(>.) ) (+.>.) ({. @(<.) )
max              =: 100
min              =: 1
mxQm             =: 255
mnQm             =: 1

```

The quantumLConst, quantumCConst and AANscaleFactor are arrays and are given in Appendix A. The quality factor calculates a factor between 0 and 5000

according to the quality level specified at the start. The `quantum` function first calls `interval` then calculates the quantiser steps. The `interval` function is dyadic and forces any elements outside the interval specified by `min` and `max` to be in the interval. The result is fed into the function divisors which uses the `AANscaleFactor` to calculate the two QMs.

The resulting array of 64 coefficients from the FDCT are then divided by the relevant QM and rounded to the nearest integer. In the above code the QMs were also inverted so in the `qCalc` function below the result from the FDCT and the QMs are actually multiplied.

```
quantise=: }.,<@{<@{divisorsLuminance@>@{.qCalc >@{.@pixels)
, <@{divisorsChrominance@>@{. qCalc >@{(1&())@pixels)
, <@{divisorsChrominance@>@{. qCalc >@{:@pixels)}
qCalc    =: <@zigzag"1@round@([(*"1) (/,)@>@])
round    =: <.@(0.5+)
```

Zig-Zag Re-ordering

The next stage, entropy coding, would be a waste of time if performed on the data received from the quantiser. This is because the AC encoding uses run-length coding which relies on long streams of identical values and our matrix of coefficients will not provide this if read left to right and top to bottom as it currently would be. Therefore, the data is re-ordered using what is called zig-zag re-ordering. This ordering places the coefficients that are most likely to have the highest values first. The elements with these high values tend to be the low frequencies. This is because the human eye is less sensitive to high frequencies so the quantiser cuts a lot of them out. The diagram below shows the zig-zag ordering of an 8x8 block of frequencies.

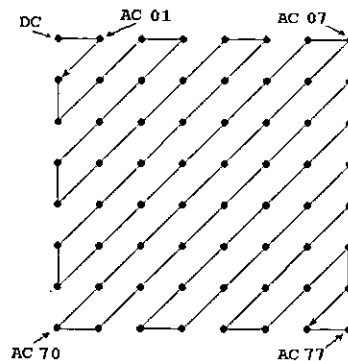


Figure3: Zig-Zag ordering (Kou, 1995)

This re-ordering was implemented simply by having an array with the order of the elements' indices and by using the from function. The array `jpegNaturalOrder` is provided in appendix A.

```
zigzag      =: jpegNaturalOrder&{
```

Entropy Coding

In JPEG the Quantised DCT coefficients are entropy coded using difference and run-length coding combined with Huffman coding. The DC component is coded separately from the AC part. The entropy encoder is able to use up to two DC and two AC Huffman tables as has been used in this implementation. The following function calls the appropriate DC and AC encoders for each component.

```
entropy      =: }:, <"2@((DCY ,. >@ACY)@{., (DCC ,.
>@ACC)@{1&{)
              ,: (DCC ,. >@ACC)@{:}@:>@pixels
```

Encoding DC Coefficient

There is a lot of locale correlation between blocks in photographic images. This is taken advantage of in JPEG by employing difference coding between the DC components called Differential Pulse Code Modulation (DPCM). To do this, first all the DC components are extracted from the blocks within a component. Then the difference between them is calculated. This value is then binary coded in a two step process. The first step is to assign a category to the DPCM and which is Huffman coded. Then if the difference is not zero, the second step is to assign additional bits to indicate the specific value and sign. The amount of extra bits is the same as the category number.

Below is a table that gives the category classifications in relation to the DC difference.

Category (SSSS)	DPCM Difference (DIFF Values)
0	0
1	-1,1
2	-3,-2,2,3
3	-7...-4, 4...7
4	-15...-8, 8...15
5	-31...-16, 16...31
6	-63...-32, 32...63
7	-127...-64, 64...127
8	-255...-128, 128...255
9	-511...-256, 256...511
10	-1023...-512, 512...1023
11	-2047...-1024, 1024...2047

Table 1: Difference magnitude categories for DC coefficients (Kou, 1995)

There are three additional rules that apply to the coding process here. The first is that the Huffman code words must be generated in such a way that no codeword has all its bits set to '1' or it will conflict with JFIF marker prefixes. The second is that the most significant bit of the appended difference magnitude is set to '0' for a negative DPCM and '1' for a positive difference (Kou, 1995).

The following code does this by finding the DPCM array and feeding it into a fork which calculates the Huffman coded category bitstream, as well as the specific difference value bitstream and joins the two.

```
DCY      =: <@ (SSSSY,DIFF) "0@difference
DCC      =: <@ (SSSSC,DIFF) "0@difference
```

The DPCM array is calculated using the difference function which works by extracting the DC elements (the first element from each box), appending a zero at the start, finding the difference between pairs and negating the result.

```
difference =: -@ (2:-/\)]@ (0&,)@ ({. @>@, )
```

The SSSS functions generate the bitstream for the category.

```

SSSSY      =: (DCbitsY&code) (|. @{.)
            (|. @, @#: @ (512&+) @ (DCvalsY&code))
SSSSC      =: (DCbitsC&code) (|. @{.)
            (|. @, @#: @ (2048&+) @ (DCvalsC&code))

```

The right prong of its fork first calls the `code` function and passes in a look-up table for the Huffman code. The `code` function picks out the relevant Huffman code by using the value generated by the `category` function. However, the value returned when converted to binary will be missing any preceding zeros. This is corrected by filling the value out with extra zeros and using the `code` function again to look-up the number of required bits for the category to select that many elements.

On inspection of table 1 it can be observed that the category for a given value is actually the number of bits it takes to represent it. The `category` function uses this characteristic by converting the value to binary and taking the shape of the binary number. There is one small problem with this though and that is that a DC value of zero is a special case. Due to the binary representation zero being '0' it, therefore, has the shape of 1 but it should be in category 0. To correct this we test the DC coefficient to see if it's not equal to zero. If it is not then we use the calculated value, otherwise we use the value 0.

```

code        =: ((~) category
category     =: (({.) @ ((~:0:), (;/@ $@#:))) "0

```

The `DIFF` function represents the specific DPCM value as a binary bitstream by simply using the `antibase2` function if the value is positive. If negative it first decrements the value and then finds the `antibase2`. There are some cases, though, where it may have an extra one at the head e.g. `_7` would generate `1 0 0 0` instead of the intended `0 0 0`. Therefore, `DIFF` also tests for this case and removes the head element if it occurs.

```

DIFF        =: ({}`}. @. ((#.@).)=0:)) @ #: @<: ` # : @. (]>0:)

```

Encoding AC Coefficient

Due to zig-zag ordering the high frequencies tend to be zero. This is utilised by using run-length coding on the AC part of the block. The code for the AC part is generated by repeatedly counting the number of zero coefficients until a non-zero one is encountered. This is then represented using the RS byte which is Huffman coded. The first four bits store the run-length of zeros and the last four hold the category value of the non-zero coefficient encountered after the run.

```

RS = binary 'R R R R S S S S'

```

There are two special cases which are handled through the use of two predefined bytes. The first is if the run-length has sixteen or more zeros. In this case a special byte called the ZRL (zero run length) is inserted. The second is if there are only zeros left until the end of the block. In this situations the byte called the EOB (end of block) is added.

```
ZRL = '1 1 1 1 0 0 0 0'
EOB = '0 0 0 0 0 0 0 0'
```

The precise value of the AC coefficient (called ZZ) is then added after the RS byte using the number of bits specified by the SSSS portion of the byte.

In the J implementation the DC component of a block is replaced by a zero and passed into the function data which builds a two row matrix. The first row gives the run-length of zeros the second gives the corresponding coefficient encountered after that run length. This is then passed into the RS and ZZ functions which generate the Huffman coded RS byte and the specific coefficient bit streams respectively. These are then joined into a single bit stream for each non zero coefficient.

```
ACY
=: (<@(>@hufCodeY@RS) (<@(>@{.,>@{:)"1@,.)>@ZZ)@data)"1@prep
ACC
=: (<@(>@hufCodeC@RS) (<@(>@{.,>@{:)"1@,.)>@ZZ)@data)"1@prep
prep=: (0&,)@)."1@>@,
```

The data function uses a function fixIndices to build an array of the index locations of non-zero coefficients. For the top row it takes this array of indices and calculates the difference between them minus one to give the run-length. The second row is built by using the from (I) function to select the non-zero coefficients. Both rows then call the EOB function which adds the EOB special byte if the last coefficient is zero. The fixIndices function uses the function indices to build the initial array.

It then finds the difference between them and inserts the ZRL special byte should any runs be 16 elements or longer by using the split function.

```
data    =: (<@(>EOB run@fixIndices), (>EOB fixIndices{}))
fixIndices =: (+/\)@ (16&split)@: -@ (2:-/\)@ (0&,)@ indices
indices =: ((i.64)&(#~))@ (0&~:)
split   =: ;@: (<g"0)
g       =: f^: (<{:)^: _
f       =: (,):) , ~&{:
EOB     =: ]` (, 0:)@. (({:@{)=0:)
run     =: ,@({)"0:@. (({=_1:),.)@<: -@ (2:-/\)@ (0:,))
```

The ZZ function takes the matrix generated by data and, using only the second row, generates the bit streams using the DIFF function used in the DC coding.

```
ZZ      =: <@DIFF&.>@<"0@{:@>
```

The RS function takes the run-length element from the first row, left-shifts it 4 places by multiplying it by sixteen and then adds a category value using the category function defined earlier.

```
RS      =: <@(((16&*)@{.)+(category@{:}))@>
```

The hufcode functions look-up the appropriate Huffman code and the number of bits required to represent that code and the trim function removes the excess elements.

```
hufCodeY =: <@trim"1@|:@{ACnumBitsY , |:@:|.1)@ACvalueY)@>
hufCodeC =: <@trim"1@|:@{ACnumBitsC , |:@:|.1)@ACvalueC)@>
ACvalueY =: (#:@(65536&+))@{ACvalsY&{(~)}
ACvalueC =: (#:@(65536&+))@{ACvalsC&{(~)}
ACnumBitsY =: ACbitsY&{(~)}
ACnumBitsC =: ACbitsC&{(~)}
trim      =: <@|. @({.{.})
```

JPEG File Interchange Format

The JFIF (JPEG File Interchange Format) is a minimal file format that allows the exchange of JPEG bitstreams between a variety of platforms and applications. The JFIF format is an extension to the older format JIF (JPEG Interchange Format) and allows for extra extensions. A JFIF format must have the JFIF APP0 marker directly after the SOI (Start of Image) marker (Hamilton, 1992).

While there are around 30 markers that specify coding information the most common and the ones used in this implementation are listed in table 2.

Description	Marker Symbol	Code Assignment (Hex)
Start Of Image	SOI	FFD8
Application-Specific JFIF	APP0	FFE0
Comment	COM	FFFE
Start Of Frame for Baseline	SOF	FFC0
Define Huffman Table(s)	DHT	FFC4
Define Quantisation Table(s)	DQT	FFDB
Start Of Scan	SOS	FFDA
End Of Image	EOI	FFD9

Table 2: Typical Markers for JPEG Baseline (Kou, 1995)

The implementation of the JFIF section basically involves appending header information to the beginning of the file and an EOI marker at the end of the file. The main function that does this is the `appendHeader` function. Its first operation is to call the function `bitStream` which takes all of the bitstream pieces from all the separate blocks and components and joins them into a single stream. It then uses the `reshape` function defined earlier to convert it into a matrix with rows of 8 bits and each row is then converted into a byte. It also inserts an extra byte with the value 0 after every byte that has a value of 255. This is done because the value 255 is reserved for markers. The `appendHeader` function then attaches the various header information which can be found in appendix A.

```

appendHeader =: (((.header1)&,)@
  (DQT,SOF,(((.header2)&,)@(;@bitStream))
bitStream    =: b@(254&a)@#.@(8&reshape)@
  ;@(pixels,.>@(4&{}),.>@(5&{}))
a             =: <@g*0
b             =: <@{}`(>:@{.,<:@{:}) @. (#>1:))@>
header1       =: (SOI,"1 JFIF","1 COM)
header2       =: (DHT,"1 SOS)
appendFooter =: EOI&{,~)

```

Saving the File

The process of writing the compressed JPEG image to file is performed by the `saveJpeg` function. The compressed data is passed in as the right argument and the path and file name entered by the user as the left argument. The function `outFile` takes the image data and appends the header and footer to the beginning

and end of the data and then calls the function `intToChar` to convert it all to characters ready for the write function to save the file.

```
saveJpeg      =: write~ outFile
outFile       =: intToChar@appendFooter@appendHeader
write         =: 1!:2<
intToChar     =: a.&({~)
```

Bringing It All Together

The various stages of the compression algorithm have been written and now just need to be brought together. The `compress` function below takes the loaded bitmap, sets up the data structure and calls the `FDCT`, `quantise` and `entropy` functions defined above.

```
compress      =: entropy@quantise@FDCT@setUp
```

This is called in the main function `JPGcompress` which loads the bitmap before it calls `compress` using what is entered on the right parameter and passes the compressed data to the `saveJpeg` function.

```
JPGcompress =: saveJpeg (compress@loadBitmap)
```

Running JPGcompress

The JPEG compression implementation above when entered works in almost the same time as other compression algorithms written in other languages. During testing, a number of images were compressed and opened in third party programs such as Internet browsers and image editing tools to ensure they displayed the image correctly.

To run the compression algorithm use the `JPGcompress` function using the following rule.

```
<path\filename.jpg> JPGcompress
<path\filename.bmp> [;<quality>]
```

The first path and file name is for the file you wish to write to and will overwrite any existing file or create a new file if one does not already exist. The second path and file name is for the bitmap file you wish to compress. The last part in brackets is optional. If not included the compression quality level will be 80, which gives a good quality image and decent compression. If you want a different level of compression then add the semicolon and integer representing the quality level.

The value 100 will not lose any information (and may be larger than the original image, depending on the image's detail) while the value 1 will give a very high level of compression but may result in an unrecognisable image.

Below are six copies of a photo of my son Uisce (Ish-Ka): the first is the original bitmap and the other five are examples of different quality levels.



Original Bitmap.
Size: 97.1 KB



Compressed at quality 100
Size: 75.7 KB



Compressed at quality 80
Size: 16.6 KB



Compressed at quality 50
Size: 10.0 KB



Compressed at quality 25
Size: 6.77 KB



Compressed at quality 1
Size: 2.39 KB

Conclusion

JPEG baseline is one of the world's most used compression algorithms. This is because it is fast, offers good compression with little noticeable image degradation and allows the user to specify the level of image quality. The purpose of this paper was to show one possible implementation for the algorithm in J. By using a

language such as J a number of areas of the implementation were simplified. The quantity of code was also reduced significantly. For instance, a Java implementation by James Weeks in 1998 took approximately 1500 lines of code.

References

Hamilton, Eric. "JPEG File Interchange Format" Version 1.02, C-Cube Microsystems, September 1, 1992, <http://www.jpeg.org/public/jfif.pdf>.

Kou, Weidong. "Digital Image Compression: Algorithms and Standards", Kluwer Academic Publishers, Boston, 1995.

Lamothe, Andre. "Windows® Game Programming For Dummies", IDG Books Worldwide February 1998.

Taylor, Nathan. "The inner workings of two key multimedia compression systems: MPEG and JPEG", *Australian Personal Computer*, Australian Consolidated Press, July 1999.

Weeks, James R. "JpegEncoder", Version 1.0a, Java program, BioElectroMech, 1998, <http://www.obrador.com>.

Hagey, Warren. "Huffman Coding", 1998, <http://educ.queensu.ca/~compsci/units/encoding/huffman.html>.

Stergiou, Chris. "What is a Neural Network", 1998, http://www.doc.ic.ac.uk/~nd/surprise_96/journal/vol1/cs11/article1.html

Appendix A

Header Information and Functions

Following is the header information that is appended to the start and end of the compressed image.

```

word          =: (256 256)&#:
SOI           =: 255 216
JFIF         =: 255 224 0 16 74 70 73 70 0 1 0 0 0 1 0 1 0 0
comment       =: 'JPEG Encoder programmed in J. Copyright 2000,
               Richard Dazeley and the University of Tasmania.'
COMlength     =: word@(2&+)@$comment
COMheader     =: 255 254
COM           =: COMheader,"1 COMlength,"1 charToInt comment
DQT          =: (DQTHeader&.)@(DQTtableY@>@{., 1:., DQTtableC@>@{.)
DQTHeader     =: 255 219 0 132 0
DQTtableY     =: jpegNaturalOrder&{@quantumLuminance
DQTtableC     =: jpegNaturalOrder&{@quantumChrominance
SOF           =: (SOFHeader&.)@SOFinfo
SOFHeader     =: 255 192 0 17
SOFinfo       =: (precision&.)@(word@height, word@width,
               numComponents, 1:, sampleFactor, 0:, 2:,
               sampleFactor, 1:, 3:, sampleFactor, 1:)
DHTtable      =: bitsDCLuminance, valDCLuminance,
               bitsACLuminance, valACLuminance,
               bitsDCchrominance, valDCchrominance,
               bitsACchrominance, valACchrominance
DHTdata       =: (word@(2&+)@$, "1]) DHTtable
DHTHeader     =: 255 196
DHT           =: (DHTHeader,"1 DHTdata)
SOS           =: 255 218 0 12 3 1 0 2 17 3 17 0 63 0

```

Tables and General Data

Following is a listing of the look-up tables and general data that has been used through the JPEG implementation.

```

precision           =: 8
blockWidth          =: 8
blockHeight         =: 8
numComponents       =: 3:
sampleFactor        =: 9:+8:
AANscaleFactor      =: 1.0 1.387039845 1.306562965 1.175875602
                    1.0 0.785694958 0.541196100 0.275899379
quantumLConst       =: 16 11 10 16 24 40 51 61 12 12 14 19 26 58
                    60 55 14 13 16 24 40 57 69 56 14 17 22 29
                    51 87 80 62 18 22 37 56 68 109 103 77 24
                    35 55 64 81 104 113 92 49 64 78 87 103 121
                    120 101 72 92 95 98 112 100 103 99

quantumCConst       =: 17 18 24 47 99 99 99 99 18 21 26 66 99 99
                    99 99 24 26 56 99 99 99 99 99 47 66 99 99
                    99 99 99 99 99 99 99 99 99 99 99 99 99 99
                    99 99 99 99 99 99 99 99 99 99 99 99 99 99

bitsDCLuminance     =: 0 0 1 5 1 1 1 1 1 0 0 0 0 0 0 0
valDCLuminance      =: 0 1 2 3 4 5 6 7 8 9 10 11
bitsDCChrominance   =: 1 0 3 1 1 1 1 1 1 1 1 0 0 0 0 0
valDCChrominance    =: 0 1 2 3 4 5 6 7 8 9 10 11
bitsACLuminance     =: 16 0 2 1 3 3 2 4 3 5 5 4 4 0 0 1 125

valACLuminance      =: 1 2 3 0 4 17 5 18 33 49 65 6 19 81 97 7 34
                    113 20 50 129 145 161 8 35 66 177 193 21
                    82 209 240 36 51 98 114 130 9 10 22 23 24
                    25 26 37 38 39 40 41 42 52 53 54 55 56 57
                    58 67 68 69 70 71 72 73 74 83 84 85 86 87
                    88 89 90 99 100 101 102 103 104 105 106
                    115 116 117 118 119 120 121 122 131 132
                    133 134 135 136 137 138 146 147 148 149
                    150 151 152 153 154 162 163 164 165 166
                    167 168 169 170 178 179 180 181 182 183
                    184 185 186 194 195 196 197 198 199 200
                    201 202 210 211 212 213 214 215 216 217
                    218 225 226 227 228 229 230 231 232 233
                    234 241 242 243 244 245 246 247 248 249
                    250

bitsACChrominance   =: 17 0 2 1 2 4 4 3 4 7 5 4 4 0 1 2 119

```

```

valACchrominance=: 0 1 2 3 17 4 5 33 49 6 18 65 81 7 97 113
19 34 50 129 8 20 66 145 161 177 193 9 35
51 82 240 21 98 114 209 10 22 36 52 225 37
241 23 24 25 26 38 39 40 41 42 53 54 55 56
57 58 67 68 69 70 71 72 73 74 83 84 85 86
87 88 89 90 99 100 101 102 103 104 105 106
115 116 117 118 119 120 121 122 130 131
132 133 134 135 136 137 138 146 147 148
149 150 151 152 153 154 162 163 164 165
166 167 168 169 170 178 179 180 181 182
183 184 185 186 194 195 196 197 198 199
200 201 202 210 211 212 213 214 215 216
217 218 226 227 228 229 230 231 232 233
234 242 243 244 245 246 247 248 249 250

DCvalsY      =: 0 2 3 4 5 6 14 30 62 126 254 510
DCbitsY      =: 2 3 3 3 3 3 4 5 6 7 8 9
DCvalsC      =: 0 1 2 6 14 30 62 126 254 510 1022 2046
DCbitsC      =: 2 2 2 3 4 5 6 7 8 9 10 11

ACvalsY      =: 10 0 1 4 11 26 120 248 1014 65410 65411 0 0 0 0 0 0
12 27 121 502 2038 65412 65413 65414 65415 65416 0 0
0 0 0 0 28 249 1015 4084 65417 65418 65419 65420
65421 65422 0 0 0 0 0 0 58 503 4085 65423 65424
65425 65426 65427 65428 65429 0 0 0 0 0 0 59 1016
65430 65431 65432 65433 65434 65435 65436 65437 0 0
0 0 0 0 122 2039 65438 65439 65440 65441 65442 65443
65444 65445 0 0 0 0 0 0 123 4086 65446 65447 65448
65449 65450 65451 65452 65453 0 0 0 0 0 0 250 4087
65454 65455 65456 65457 65458 65459 65460 65461 0 0
0 0 0 0 504 32704 65462 65463 65464 65465 65466
65467 65468 65469 0 0 0 0 0 0 505 65470 65471 65472
65473 65474 65475 65476 65477 65478 0 0 0 0 0 0 506
65479 65480 65481 65482 65483 65484 65485 65486
65487 0 0 0 0 0 0 1017 65488 65489 65490 65491 65492
65493 65494 65495 65496 0 0 0 0 0 0 1018 65497 65498
65499 65500 65501 65502 65503 65504 65505 0 0 0 0 0
0 2040 65506 65507 65508 65509 65510 65511 65512
65513 65514 0 0 0 0 0 0 65515 65516 65517 65518
65519 65520 65521 65522 65523 65524 0 0 0 0 0 2041
65525 65526 65527 65528 65529 65530 65531 65532
65533 65534 0 0 0 0

```

```

ACbitsY      =: 4 2 2 3 4 5 7 8 10 16 16 0 0 0 0 0 0 4 5 7 9 11 16
               16 16 16 16 0 0 0 0 0 0 5 8 10 12 16 16 16 16 16 16
               0 0 0 0 0 0 6 9 12 16 16 16 16 16 16 16 0 0 0 0 0 0
               6 10 16 16 16 16 16 16 16 16 0 0 0 0 0 0 7 11 16 16
               16 16 16 16 16 16 0 0 0 0 0 0 7 12 16 16 16 16 16 16
               16 16 0 0 0 0 0 0 8 12 16 16 16 16 16 16 16 16 0 0 0
               0 0 9 15 16 16 16 16 16 16 16 16 0 0 0 0 0 0 9 16
               16 16 16 16 16 16 16 16 0 0 0 0 0 0 9 16 16 16 16 16
               16 16 16 16 0 0 0 0 0 0 10 16 16 16 16 16 16 16 16
               16 0 0 0 0 0 0 10 16 16 16 16 16 16 16 16 16 0 0 0
               0 0 11 16 16 16 16 16 16 16 16 16 0 0 0 0 0 0 16 16
               16 16 16 16 16 16 16 16 0 0 0 0 0 0 11 16 16 16 16
               16 16 16 16 16 0 0 0 0

```

```

ACvalsC      =: 0 1 4 10 24 25 56 120 500 1014 4084 0 0 0 0 0 0 11
               57 246 501 2038 4085 65416 65417 65418 65419 0 0 0 0
               0 0 26 247 1015 4086 32706 65420 65421 65422 65423
               65424 0 0 0 0 0 0 27 248 1016 4087 65425 65426 65427
               65428 65429 65430 0 0 0 0 0 0 58 502 65431 65432
               65433 65434 65435 65436 65437 65438 0 0 0 0 0 0 59
               1017 65439 65440 65441 65442 65443 65444 65445 65446
               0 0 0 0 0 0 121 2039 65447 65448 65449 65450 65451
               65452 65453 65454 0 0 0 0 0 0 122 2040 65455 65456
               65457 65458 65459 65460 65461 65462 0 0 0 0 0 0 249
               65463 65464 65465 65466 65467 65468 65469 65470
               65471 0 0 0 0 0 0 503 65472 65473 65474 65475 65476
               65477 65478 65479 65480 0 0 0 0 0 0 504 65481 65482
               65483 65484 65485 65486 65487 65488 65489 0 0 0 0
               0 505 65490 65491 65492 65493 65494 65495 65496
               65497 65498 0 0 0 0 0 0 506 65499 65500 65501 65502
               65503 65504 65505 65506 65507 0 0 0 0 0 0 2041 65508
               65509 65510 65511 65512 65513 65514 65515 65516 0 0
               0 0 0 0 16352 65517 65518 65519 65520 65521 65522
               65523 65524 65525 0 0 0 0 0 0 1018 32707 65526 65527
               65528 65529 65530 65531 65532 65533 65534 0 0 0 0

```

```

ACbitsC      =: 2 2 3 4 5 5 6 7 9 10 12 0 0 0 0 0 0 4 6 8 9 11 12 16
               16 16 16 0 0 0 0 0 0 5 8 10 12 15 16 16 16 16 16 0 0
               0 0 0 0 5 8 10 12 16 16 16 16 16 16 0 0 0 0 0 0 6 9
               16 16 16 16 16 16 16 16 0 0 0 0 0 0 6 10 16 16 16
               16 16 16 16 0 0 0 0 0 0 7 11 16 16 16 16 16 16 16
               0 0 0 0 0 0 7 11 16 16 16 16 16 16 16 16 0 0 0 0 0
               8 16 16 16 16 16 16 16 16 16 0 0 0 0 0 0 9 16 16 16
               16 16 16 16 16 16 0 0 0 0 0 0 9 16 16 16 16 16 16
               16 16 0 0 0 0 0 0 9 16 16 16 16 16 16 16 16 16 0 0
               0 0 0 9 16 16 16 16 16 16 16 16 16 0 0 0 0 0 0 11 16
               16 16 16 16 16 16 16 16 0 0 0 0 0 0 14 16 16 16 16
               16 16 16 16 16 0 0 0 0 0 0 10 15 16 16 16 16 16 16
               16 16 0 0 0 0

```

```

jpegNaturalOrder=: 0 1 8 16 9 2 3 10 17 24 32 25 18 11 4 5 12
                   19 26 33 40 48 41 34 27 20 13 6 7 14 21 28
                   35 42 49 56 57 50 43 36 29 22 15 23 30 37
                   44 51 58 59 52 45 38 31 39 46 53 60 61 54
                   47 55 62 63

```

Inversion of Boolean Matrices

by Michel Dumontier (michel.dumontier@free.fr)

Summary: We give here the algorithm for the inversion of Boolean matrices, in the Boolean sense of course, for matrixes of any order. This article was first published in *Les Nouvelles d'APL* N° 35.

Introduction

In issue No. 8 of *Les Nouvelles d'APL*, Gérard Langlet [1] gives the following matrix of order 15:

GA

0	0	0	0	0	1	0	1	1	1	1	0	1	0	1
0	1	1	0	0	1	1	0	1	1	0	0	0	0	0
0	0	0	0	0	1	1	1	0	1	1	1	0	0	1
1	0	1	1	0	1	0	1	0	1	0	1	1	0	0
0	0	0	1	0	1	0	0	1	1	1	0	1	0	1
1	1	1	1	0	0	0	1	1	0	1	1	1	1	1
1	0	0	0	0	1	1	1	0	1	1	0	0	1	1
0	0	0	1	0	0	1	0	1	1	0	1	1	0	1
1	0	1	0	0	0	1	1	1	0	0	0	1	1	0
1	1	0	0	0	0	0	0	1	0	1	1	1	1	1
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	0	1	1	1	0	0	0
0	0	0	0	0	0	1	0	0	1	0	1	0	1	0
0	0	0	0	0	1	0	0	0	1	0	0	0	1	0
1	0	0	0	1	1	1	0	1	1	0	1	0	0	0

And asks:

1. Is it possible to find its inverse?
2. Is the solution unique?
3. What is the shortest program and, if possible, the quickest, able to produce in the general case such an inversion without error, if necessary on the largest possible matrices?

He added, certainly with mischief: the solution in the next issue... we talked over the phone very often and he knew that I had the solution and believed that I was going to give it.

Without waking up the cat that sleeps,* I got away with a dirty trick in the following issue without revealing the algorithm. I gave the inverted matrix by saying that the matrix in question is the 5355th root of the unit matrix (by raising the matrix to the power 5354, one obtains the inverse): a method that is certainly not the shortest!

Let us verify it here.

Here is the function that raises a matrix to its N^{th} power:

```

V Z←M MP N;MD;L;C;I;L2;IO
[1] IO←0 ∘ Z←M ∘ I←0 ∘ L←[2⊗N ∘ L2←2*L ∘ MD←L2|N
[2] 'Z←Z×.ΛZ'wh'L≥I+I+1' ∘ →(0=MD)/0 ∘ I←0
[3] 'Z←Z×.ΛM'wh'MD≥I+I+1'
V

```

```

GA MP 5355
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

```

Here is the function that let us find the inverse:

```

V Z←TU M;IO;I;U;N;P;N2
[1] Z←1⊗N+1↑ρM⊗U+(I N)∘.=I N∘ U←UΛU ∘P←M ∘N2←N×N
[2] BO:P←P×.ΛM ∘ Z←Z+1 ∘ →(N2=+/(,P)=,U)/0∘→BO
V
TU GA
5355

```

3 seconds to wait! Not so long (considering that it's performing 5354 Boolean matrix products!)

* For those who don't know: a French idiom meaning it is not necessary to reawaken old quarrels. *Let sleeping dogs lie* is probably the closest English equivalent.

The story is that the only weapon he had for matrix inversion was the following function. He said that mathematicians had not yet discovered what the inverse of a Boolean matrix (in the Boolean sense) is, nor did they know how to calculate it.

```

      v Z←BINV M
[1]   Z←CT<|M
      v

```

Let us try it on a small matrix:

```

      A←3 3p79p2
      A
0 1 0
1 1 0
0 1 1

```

```

      A≠. ^BINV A
1 0 0
0 1 0
0 0 1

```

Indeed that works fine, but on the matrix GA it does not work.

Here is the proof...

I see some of my readers are frowning: they believed it was necessary to replace the + of the numerical inner product by the Boolean or... No! It is the XOR you must use, v. ^ performs the transitive closure of a graph, provided that you apply it to the same matrix: B v. ^ B.

But back to business:

```

      GA≠. ^BINV GA
1 0 0 0 0 0 1 1 1 0 0 1 0 1 0
1 1 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 1 1 0 1 0 1 0 1 0
1 0 0 1 0 1 1 1 0 0 0 1 1 1 0
0 0 0 0 1 0 1 1 1 0 0 1 0 1 0
0 0 0 0 0 1 1 0 0 1 0 0 1 0 0
1 0 0 0 0 0 0 1 0 1 1 1 0 1 0
1 0 0 0 0 1 0 0 0 0 1 1 1 1 0
1 0 0 0 0 0 1 0 1 1 1 0 0 0 0
0 0 0 0 0 0 0 1 1 0 1 1 0 1 0
0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
1 0 0 0 0 0 0 0 0 1 0 1 0 1 0
0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 1 0 0 1 0 1 0 0 1 0
1 0 1 1 0 0 0 1 0 1 1 1 1 0 1

```

Calculating the inverse in the numerical sense of GA:

```

      4 1*GA
      0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0
      -0.4 0.6^-1.0 0.6^-1.0^-0.9 1.3 0.7^-0.3 1.3^-2.1 0.7^-1.3^-0.1 0.0
      1.0 0.0 0.0 0.0 1.0 1.7^-0.7^-1.3^-0.7^-1.7 1.3^-1.3 2.7^-1.3 0.0
      -1.0 0.0 0.0 0.0 0.0^-0.3 0.3 0.7 0.3 0.3^-0.7 0.7^-1.3 0.7 0.0
      0.8^-0.2^-2.0 0.8 0.0 0.5 1.7^-0.7^-1.1^-0.3^-2.5^-0.7 1.3^-2.1 1.0
      -1.0 0.0 1.0 0.0 0.0^-0.7^-0.3 0.3 0.7 0.7^-0.3 0.3^-1.7 1.3 0.0
      -0.8 0.2 0.0 0.2 0.0^-0.9 0.7 0.3 0.5 0.7^-1.1 0.3^-0.7^-0.3 0.0
      0.0 0.0 0.0 0.0^-1.0^-0.3 0.3 0.7 0.3 0.3^-0.7 0.7^-1.3 0.7 0.0
      0.0 0.0 1.0^-1.0 0.0 0.3^-1.3 0.3 0.7^-0.3 1.7 0.3^-0.7 1.3 0.0
      1.2 0.2^-1.0 0.2 0.0 0.5 0.3^-0.3^-0.9^-0.7 0.5^-0.3 1.7^-0.9 0.0
      0.0 0.0 0.0 0.0 1.0 0.0 0.0^-1.0 0.0 0.0 0.0 1.0^-1.0 0.0
      -0.2^-0.2 1.0^-0.2 0.0 0.2^-1.0 0.0 0.2 0.0 0.8 0.0 0.0 0.6 0.0
      0.0 0.0^-1.0 1.0 0.0^-1.0 1.0 0.0 0.0 1.0^-2.0 0.0 0.0^-1.0 0.0
      -0.2^-0.2 0.0^-0.2 0.0 0.2 0.0 0.0 0.2 0.0^-0.2 0.0 0.0 0.6 0.0
      0.8^-0.2 0.0^-0.2 0.0 1.2 0.0 0.0^-0.8^-1.0 0.8^-1.0 1.0^-0.4 0.0
  
```

This is not at all the same thing as the inverse in the Boolean sense.

Inversion of a Boolean matrix

We must once again use the algorithm of the inversion modulo p [2] and set $p=2$, but re-write the algorithm in origin 0.

```

      ▽ Z+M DIO2 P; IO; I; T; U; N; IT; MU; D
      [1] IO+0 T+φM φN+1+ρM
      [2] U+(IN)φ.=IN φ IT+INV T φD+|DET M
      [3] MU+θU φ Z+(N,N)ρ0 φ I+0 φ N1+N-1
      [4] INC:Z[I;]+(P|D×IP) (P|((IT)+.×MU[I;])×D)
      [5] →(N1≥I+I+1)/INC
      ▽
  
```

$P \leftarrow 2$
 $\square \leftarrow IGA + GA \quad DIO2 \quad P$
 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
 0 1 1 1 1 0 0 0 0 0 1 0 0 0 0
 1 0 0 0 1 1 0 0 0 1 0 0 0 0 0
 1 0 0 0 0 1 1 0 1 1 0 0 0 0 0
 0 1 0 0 0 0 1 0 1 1 0 0 0 1 1
 1 0 1 0 0 0 1 1 0 0 1 1 1 0 0
 0 1 0 1 0 1 0 1 1 0 1 1 0 0 0
 0 0 0 0 1 1 1 0 1 1 0 0 0 0 0
 0 0 1 1 0 1 0 1 0 1 1 1 1 0 0
 0 1 1 1 0 1 1 1 1 0 0 1 1 0 0
 0 0 0 0 1 0 0 1 0 0 0 0 1 1 0
 1 1 1 1 0 1 1 0 1 0 0 0 0 1 0
 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0
 1 1 0 1 0 1 0 0 1 0 1 0 0 1 0
 0 1 0 1 0 0 0 0 0 1 0 1 1 0 0

$GA \neq \wedge IGA$
 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

QED.

Properties: See [3].

We can process matrices of any order. (We work in J now). To keep the examples compact, we are going to work on a matrix of order 20 here.

The preceding matrix was invertible in the Boolean sense because its determinant was odd (it was 15); an even determinant in a Boolean matrix is equivalent to a null determinant in matrices of numbers (whether they are integers or reals). In other words, $2 \mid \det a$ would be 0 in this case.

```

a
0 1 0 1 1 0 0 1 0 0 1 0 0 1 1 1 1 0 0 0
0 0 1 0 1 1 1 0 1 0 0 0 0 0 0 0 1 1 1 0
0 0 0 1 1 0 0 1 1 1 0 0 1 0 1 1 1 0 1 0
1 1 0 0 1 1 0 1 1 0 1 0 1 1 1 0 0 0 1 1
0 1 1 1 1 0 1 0 1 0 0 1 0 0 1 1 1 1 0 1
1 1 1 1 0 1 1 1 0 1 0 1 0 1 1 0 0 0 1 0
1 0 0 0 0 1 1 1 1 1 0 1 1 1 0 1 1 1 1 1
0 1 0 1 0 1 1 0 0 1 1 1 0 0 0 0 0 1 0 1
1 0 0 0 1 0 1 1 1 0 0 0 1 0 0 0 1 1 0 0
0 1 0 1 1 0 1 1 0 0 1 1 1 0 1 1 0 0 1 1
0 1 1 0 0 1 0 1 1 1 1 0 0 0 0 1 0 0 1 1
1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 0 0 1 1
1 1 1 0 0 1 1 0 0 1 0 0 0 1 0 1 1 1 1 0
1 0 1 1 0 1 0 0 0 1 1 1 1 0 0 0 1 1 0 0
0 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 0 0
0 1 1 1 1 1 0 1 0 1 1 1 1 1 0 1 0 0 1 0
0 1 0 1 0 0 0 1 1 0 1 1 1 1 0 1 1 0 0
0 0 0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 1
1 1 1 0 1 1 1 1 1 0 0 0 1 1 1 0 1 0 0 1
1 0 1 0 1 0 0 0 1 0 1 0 0 1 1 1 1 1 0

```

```

det a
12371

```

It's possible to invert it in the Boolean sense:

```

+b=:a dio p
1 1 1 1 0 1 1 0 0 1 1 0 0 1 1 0 1 1 1 0
1 1 1 0 1 1 1 1 0 0 0 0 1 0 0 1 1 1 0 1
1 0 1 0 0 0 1 1 1 1 1 1 1 0 1 1 0 1 0 0
0 0 1 0 1 1 1 1 1 0 1 0 1 0 1 1 0 1 1 1
0 1 0 1 1 0 0 1 1 1 0 1 0 0 0 0 0 0 1 0
1 1 0 0 0 1 0 1 0 0 0 1 0 0 0 1 0 1 1 1
1 0 0 0 0 1 0 0 1 0 1 0 1 1 0 0 0 1 0 0
0 1 0 1 1 0 0 1 1 1 0 1 1 1 1 1 1 0 1 0
1 1 1 1 0 1 0 1 1 0 1 0 1 1 0 0 1 0 1 0
1 1 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1
1 0 1 1 0 0 0 0 0 1 0 0 1 1 0 1 1 0 0 1
0 1 1 0 1 0 1 0 1 1 0 0 0 0 0 1 1 0 1 0
1 0 1 1 1 0 0 1 1 0 1 1 1 1 1 1 0 0 1 0
0 1 1 0 0 0 0 1 0 1 1 1 1 1 1 1 0 1 1 0
0 1 0 1 0 0 0 0 1 1 0 0 1 0 0 1 0 0 0 1
0 1 1 1 0 1 1 0 0 0 1 0 0 0 0 0 0 0 1 0
1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 0 1 1 1 0
1 0 0 0 1 0 0 1 0 0 1 0 0 0 0 0 0 1 0 0
0 0 1 1 0 0 0 0 0 1 1 0 1 1 0 0 0 1 0 1
0 0 1 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 1 1

```

and to invert it in the numerical sense.

```

c =:%.a

```


Let us apply to it the inner product of the matrix a on v as if it were a vector of integers to give a vector of integers w :

$$+w = .a +/ . * v$$

$$4 \ 4 \ 5 \ 3 \ 5 \ 4 \ 7 \ 4 \ 5 \ 5 \ 3 \ 6 \ 6 \ 8 \ 6 \ 5 \ 3 \ 5 \ 5$$

That obviously gives integers as we would expect. (They are sums of numbers that are either 0 or 1.)

Let us apply the inner product of the inverse of a on the vector w and then we have to find the original vector v .

$$c +/ . * w$$

$$0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0$$

QED1.

But we are going to make the Boolean matrix work in the Boolean ocean, by using Boolean inversion.

Let us take again our vector v and apply it to the Boolean matrix product to give the vector bw .

Note: bw is a Boolean vector as opposed to the numerical vector n obtained previously.

$$+bw = :has \sim :/ . * .v$$

$$0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1$$

Let us apply the Boolean matrix product of the inverse of a in the Boolean sense (matrix b) on the vector bw produced and then we would have to recover the original vector v .

$$b \sim :/ . * .bw$$

$$0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0$$

Let us compare it to the vector v

$$v$$

$$0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 1 \ 0 \ 0$$

QED2.

Let us compare the vector w obtained previously and the vector bw .

The vector bw represents the parity of w .

^w
4 4 5 3 5 4 7 4 5 5 5 3 6 6 8 6 5 3 5 5

^{bw}
0 0 1 1 1 0 1 0 1 1 1 1 0 0 0 0 1 1 1 1

Oh, what fun! (When one is retired and has plenty of time.)

References

- [1] Gérard Langlet. Travaux pratiques en APL, *Les Nouvelles d'APL* N° 8 p 97.
- [2] Michel J.Dumontier. Inversion matricielle modulo p, Systèmes linéaires modulo p. *Les Nouvelles d'APL* N° 35 pp. 45-54. Translated as Matrix Inversion modulo p, Linear systems modulo p, *Vector* 18.1.
- [3] Properties of Boolean Matrices generated by Gray functions. Michel J. Dumontier *APL-CAM Journal*, Vol 14, N°2.

Matrix inversion modulo p Linear systems modulo p

by Michel Dumontier (*michel.dumontier@free.fr*)

Summary

The two problems announced in the title are linked. To solve the one is to solve the other. On the other hand solving the first leads to the resolution of another problem that had not yet been solved [1] before this present attempt. This article was first published in *Les Nouvelles d'APL* N° 35.

Let A be a matrix of integers of order n , p an integer and V a vector of integers of n elements all less than p . Given $W = p \mid A + \text{###} V$, is there a matrix B such that $p \mid B + \text{###} W$ restores the original vector V ?

The answer is yes, if A is well formed (but not in the way one normally thinks – see [2]). We give here a recipe for the calculation of B under this assumption. We will call B the *inverse modulo p of A* (the term *inverse of A modulo p* would be more ambiguous).

Introduction

Such a system can be solved manually up to order 3. For any order n , despite reading through a large bibliography and searching the web, nowhere have we found a solution to this problem. As usual, using APL and J tools (that, a long time ago, I christened 'faitout-vite' in French, possibly SpeedyGee in English), I found the general solution in one day, first of all in APL. As the numbers grow very quickly, eventually I had to resort to J, which lets one work in extended precision, that is integers without limit. As they say, if you want it done, do it yourself, and I think we should not hesitate when we have tools such as APL and J at our disposal.

The work here presented served as preparation for a paper where the techniques developed are put to a practical application in the exciting field of cryptology. [2]

We will deal in turn first with the APL solution and then the J solution.

The APL solution leads to a deadlock when the matrix A to invert (modulo p) is of a order high enough, because we would need numbers larger than the limit of APL integers. However, we will not pass in silence over this work we had to do.

1. APL solution

As we need the integer matrix determinant and inversion, we pick up our dusty old notebooks and write these functions in APL.

Calculation of the adjoint matrix:

```

      ▽ Z←ADJOINT A
[1]   Z←⊖COFACTORS A
      ▽

```

Calculation of the cofactors:

```

      ▽ CO←COFACTORS A;I;J;⊖IO
[1]   CO←1 0 ⊙ I←1
[2]   INC←J←0
[3]   IT←J+J+1
[4]   CO←CO, A COFACTOR I, J
[5]   →(J<(ρA)[1])/IT
[6]   I←I+1
[7]   →(I≤1+ρA)/INC
[8]   CO←(ρA)ρCO
[9]
      ▽

```

Calculation of a cofactor:

```

      ▽ Z←A COFACTOR V;SIGN;⊖IO
[1]   ⊖IO←1 ⊙ SIGN←-1*+/V
[2]   Z←SIGN*DET A MINOR V
[3]
      ▽

```

Calculation of the determinant:

```

      ▽ Z←DET M ;⊖IO
[1]   ⊖IO←1 ⊙ Z←M[1;]+.×(COFACTORS M)[1;]
      ▽

```

Calculation of a minor:

```

      ▽ Z←A MINOR V;Q;⊖IO
[1]   ⊖IO←1 ⊙ Q←(V[1]≠1(ρA)[1])/[1]A
[2]   Z←(V[2]≠1(ρA)[2])/Q
[3]
      ▽

```

Calculation of the inverse:

```

      ▽ Z←INV M
[1]   →((DET M)=0)/FIN
[2]   Z←(ADJOINT M)÷DET M 0 →0
[3]   FIN:'DET =0'
      ▽

```

Thus we are well prepared and are sure that we are working with integers.

We can now write the function that calculates the inverse modulo p of a matrix.

Calculation of the inverse matrix modulo p

Taking into account successive vectors from the first to the last line of the unit matrix of order n , we have the next solution. We call it DIO in honour of the Greek mathematician Diophantus.

```

      ▽ Z←M DIO P;□IO;I;T;U;N;IT;MU;D
[1]   □IO←10 T←0M 0N+1pM
[2]   U←(1N)0.=1N 0 IT←INV T 0D←DET M
[3]   MU←0U 0 Z←(N,N)00 0 I←1
[4]   INC:Z[I;]←(P|D×1P)1|(P|((IT)+.×MU[I;])×D)
[5]   →(N>I+I+1)/INC
[6]
      ▽

```

(This is the function I searched for everywhere and eventually I had to write for myself as well.)

Test of the function

Let us choose $p=256$.

We take a random integer matrix of order 3, whose elements are in the range -255 to 255

```

      A
226  -31 150
199  -38  57
 94   -8  71

```

Let us calculate the matrix B inverse modulo 256 of A

```

      B←B+A DIO P
62 233 93
189 154 72
188 174 141

```

Let us take a random vector V whose elements are less than 256.

$\square + V + 3 \times 255$
229 7 145

Let us apply to it $P|A+. \times V$

$\square + W + P|A+. \times V$
71 66 21

The goal of the game is to recover V by applying $P|B+. \times W$.

$P|B+. \times W$
229 7 145

Success!

Do not rejoice too soon!

Unfortunately already at order 6, the calculations will overflow, causing the inverse to be wrong.

$\square + A + 6 \times 36 \times 255$
33 192 116 135 55 11
173 173 238 97 132 211
8 13 135 171 1 97
17 106 175 150 237 215
134 23 166 106 178 232
194 66 12 187 83 161

$A \text{ DIO } P$
60 156 224 257 136 257
11 257 202 124 257 257
257 109 257 55 257 257
54 71 257 42 68 173
229 141 77 202 257 67
148 257 257 14 77 257

Let us start again with another (because a single element of 257 is enough to make the calculation of b wrong).

$A + 6 \times 36 \times 255$
131 214 7 108 184 118
76 83 224 112 96 82
124 105 100 135 204 119
64 230 248 113 159 225
176 207 142 103 92 89
15 52 110 115 78 67

```

      A DIO P
257 257 257 257 257 257
257 257 257 257 257 257
257 257 257 257 257 257
257 257 110 257 29 257
257 257 257 257 257 257
257 257 79 257 45 257

```

It's about time to gear up and switch to J, the only one which can save us, since it can store and perform calculations on integers as large as we need.

2. J Solution

Let us rewrite our routines.

For the determinant, no sweat:

```
det=: -/ . *
```

Luckily for us, this train is special-cased in the latest releases of the interpreter produced by Jsoftware and we don't have to hide the beauty of the mathematical definition behind a thousand lines of efficient, but ugly, code.

J does not have among its primitives the equivalent of APL's vertical mirror, so let us re-write it in tacit form as:

```
mv=: |." _1@]
```

We will trust the inverse matrix calculated by the J primitive %.

```

dio=: 3 : 0
:
t =: |: mv x.
n =: 0{$x.
u =: (i.n) =/ i.n
mu=: |.u
it=: %. t
d =: det x.
b =: (n,n)$0
i =: 0
while. i<n do.
b =: (y.|d*i.y.) i. (y. |((it)+/ .* (i{mu))*d) (i)) b
i =: i+1
continue.
end.
)

```

Now we can try our teeth with the first matrix that did not work in APL.

```

a
33 192 116 135 55 11
173 173 238 97 132 211
8 13 135 171 1 97
17 106 175 150 237 215
134 23 166 106 178 232
194 66 12 187 83 161

```

```

a:=x: a      NB. Conversion into integers without limit.
p :=:256x
b :=:a dio p

```

```

b
175 50 54 130 174 65
46 166 248 202 13 54
198 72 47 82 9 185
121 146 33 53 87 179
123 139 24 136 171 182
186 153 237 33 96 228

```

Let us pick a random vector v:

```

v :=: 67256
v
126 123 172 3 127 184

```

Let us calculate w and verify that we get v back:

```

w :=:p|a+/ . *v
w
52 116 27 189 213 88
p|b+/ . *w
126 123 172 3 127 184

```

In order to show that it works on matrices of high order, just to whet our appetite for the paper on cryptology, let us take a matrix of order 10 and do a little test.

```

+a=:10 10$100?256
37 149 215 150 243 141 37 250 104 36
144 64 124 118 245 32 50 81 160 32
166 158 204 63 121 99 51 7 229 108
36 241 104 33 225 23 41 18 93 64
34 199 116 89 115 206 237 166 54 173
231 63 219 120 129 153 208 192 117 242
161 112 210 175 179 251 243 217 73 137
131 26 105 147 223 112 186 221 182 204
180 189 4 225 133 118 16 181 124 170
173 50 233 220 226 138 35 114 252 54

```

```

+b=:a dio p
168 146 70 169 227 106 107 63 184 81
38 129 50 213 66 33 80 54 219 157
42 75 118 174 151 243 131 72 126 140
44 206 38 39 196 223 76 214 6 241
74 98 77 108 76 137 198 112 215 147
87 209 5 217 234 46 96 214 236 223
80 31 80 237 212 111 6 81 40 234
78 177 51 40 80 39 202 42 145 55
35 123 194 69 112 0 144 149 2 18
60 12 17 23 13 222 78 51 166 227

```

```

+v=:10?256
234 79 158 41 220 6 242 228 125 56

```

```

+w=:p|a+/ . *v
241 162 174 217 22 28 170 135 124 136

```

```

p|b+/ . *w
234 79 158 41 220 6 242 228 125 56

```

NB. In both matrices, one can replace any number n (especially those that are greater than 128) by the negative number $n - 256$.

Let us do it on a line of a taken randomly.

```

u =: _25 63 _37 120 _127 _103 _48 _64 117 _14
u =: x: u
a =: u (5)} a
a
37 149 215 150 243 141 37 250 104 36
144 64 124 118 245 32 50 81 160 32
166 158 204 63 121 99 51 7 229 108
36 241 104 33 225 23 41 18 93 64
34 199 116 89 115 206 237 166 54 173
_25 63 _37 120 _127 _103 _48 _64 117 _14
161 112 210 175 179 251 243 217 73 137
131 26 105 147 223 112 186 221 182 204
180 189 4 225 133 118 16 181 124 170
173 50 233 220 226 138 35 114 252 54

```

- 1) The transformed matrix b is identical:

```

a dio p
168 146 70 169 227 106 107 63 184 81
38 129 50 213 66 33 80 54 219 157
42 75 118 174 151 243 131 72 126 140
44 206 38 39 196 223 76 214 6 241
74 98 77 108 76 137 198 112 215 147
87 209 5 217 234 46 96 214 236 223
80 31 80 237 212 111 6 81 40 234
78 177 51 40 80 39 202 42 145 55
35 123 194 69 112 0 144 149 2 18
60 12 17 23 13 222 78 51 166 227

```

- 2) the transformed vector w is the same:

```

+w=:p|a+/ . * v
241 162 174 217 22 28 170 135 124 136

```

We shall not pass over in silence the ulterior motive that some may have had reading this paper...

It's no accident that we have chosen $p=256$ because the residue modulo 256 is a number in the range 0 to 255 and can represent the index in origin 0 of any character of a computer in the $\square AV$.

If b is the inverse modulo p of a , it is necessary to know that it is symmetrical: then a is the inverse modulo p (the same p of course) of b .

Let us verify this by working on the same vector v again but let us use b instead of a as the transformation matrix. Except in case of an unlikely coincidence, we should not have the same w .

$$\begin{array}{r}
 v \\
 234 \ 79 \ 158 \ 41 \ 220 \ 6 \ 242 \ 228 \ 125 \ 56 \\
 +w = p|b+/. * v \\
 69 \ 1 \ 205 \ 61 \ 73 \ 100 \ 88 \ 168 \ 186 \ 247
 \end{array}$$

The w transformed with a was:

$$241 \ 162 \ 174 \ 217 \ 22 \ 28 \ 170 \ 135 \ 124 \ 136$$

It does not matter if the w calculated is not the same, we are going to find our v by the magical operation:

$$\begin{array}{r}
 p|a+/. * w \\
 234 \ 79 \ 158 \ 41 \ 220 \ 6 \ 242 \ 228 \ 125 \ 56
 \end{array}$$

One concludes immediately that the matrices a and b can be kept secret separately. Person B for example, who knows how to invert modulo p , sends to his correspondent (A) the matrix a that he can use to transform messages for B's eyes only. B reconstitutes the messages with b but can send messages to A by using b . A will then reconstitute the message with a and does not need to know b .

Of course what is sent down the wire will not be the numeric vector w but the characters situated at the indices contained in w in the $\square AV$ (or maybe something cleverer – see [2]). In other words, the person A would transmit the message $T[w]$ (reads naturally T indexed by w), where T is the atomic vector in our example.

A last paragraph on an interesting property of the transformation: of course one could take matrices a whose elements are greater than -255 and less than 255 but it would be as if he took the matrix $256|a$.

References

- [1] Michel J. Dumontier. Inversion des matrices Booléennes. *Les Nouvelles d'APL* N° 35 p.55. Translated as Inversion of Boolean Matrices, *Vector* 18.1
- [2] Michel J. Dumontier. Un nouveau chiffrement court échappant à la cryptanalyse. *Les Nouvelles d'APL* N° 35 p.65. Translated as MUCIPHER: A new short cipher that escapes cryptanalysis, *Vector* 18.2.

Index to Advertisers

Dyadic Systems Ltd	4
HMW Systems	2
Strand Software	6
Vector Back Numbers	5

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Subscribing to Vector

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

UK private membership	£12
-----------------------------	-----

Overseas private membership£14

Airmail supplement (not needed for Europe)£4

UK Corporate membership	£100
-------------------------------	------

Corporate membership overseas	£135
-------------------------------------	------

Sustaining membership£430

Non-voting UK member (student/OAP/unemployed only) .£6

□ □ □ □ □ □

PAYMENT – in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

☐ I authorise you to debit my Visa/Mastercard/JCB account

[illegible]

for the membership category indicated above,

☐ annually, at the prevailing rate, until further notice

☐ one year's subscription only

Data Protection Act:
The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Signature: _____

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

Fax: +44 (0) 1653 697719

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2001/2002 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973 0036 acam@tcsco.net	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 trcas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 100430.740@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
Education	Vacant post	
Administration	Rowena Small 01223-570850 trcas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho, Ray Cannon (01252-874697), Marc Griffiths, Bob Hockstra (01483-771028), Jon Sandles (01904-612882)	

Typeset by APL-385 with MS Word for Windows
Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

APL2000
Rapid Application Development
6610 Rockledge Drive
Suite 502
Bethesda MD 20817
USA
Email: sales@apl2000.com
Web: www.apl2000.com

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com

Dutch APL Association
Postbus 1341
3430BH Nieuwegein
Netherlands
Tel: +31 347 342 337